

BehaviorTree.CPP Technical Report

August 21, 2026

Prepared by:
Framework Team

Prepared for:
ASA Team

Sumário

Lista de figuras		vi
Lista de tabelas		vii
I	Introdução	2
I.1	O que é a BehaviorTree.CPP	2
I.2	Objetivo e público-alvo	2
I.3	Versão documentada	3
I.4	Convenções do documento	3
I.5	Organização do manual	4
II	Arquitetura geral	6
II.1	Princípios de projeto	6
II.2	Organização do código-fonte	6
II.3	O modelo de execução: o <i>tick</i>	7
II.4	Memória e reatividade	8
II.5	O ciclo de vida de um nó	9
II.6	As três famílias de nós	10
II.7	Da configuração à árvore: os quatro passos	10
II.8	Um cenário para acompanhar o manual	11
III	O modelo de nós	12
III.1	TreeNode : o contrato	12
III.2	Estado, notificação e concorrência	13
III.3	O identificador único (UID)	15
III.4	As três aridades	15
	III.4.1 LeafNode : nenhum filho	15
	III.4.2 ControlNode : <i>N</i> filhos	15
	III.4.3 DecoratorNode : exatamente um filho	16
III.5	O tipo de um nó, em tempo de compilação	17
III.6	Percorrer a árvore	17
III.7	Erros e exceções	18
III.8	No cenário da porta	19
IV	Portas e <i>blackboard</i>	20
IV.1	Declarando portas	20
IV.2	Lendo e escrevendo	21
IV.3	A resolução de um nome de porta	22
IV.4	Conversão de texto: convertFromString	23
IV.5	O <i>blackboard</i>	24
	IV.5.1 Encadeamento entre pai e filho	25
	IV.5.2 O travamento de tipo	25
	IV.5.3 Inspeção	26
IV.6	Valores padrão	26
IV.7	No cenário da porta	27
V	Nós de controle	28
V.1	Visão de conjunto	28

V.2	Sequence: o E lógico	28
V.3	SequenceStar: o E que não esquece	29
V.4	ReactiveSequence: o E que reavalia tudo	30
V.5	Fallback e ReactiveFallback: o OU lógico	31
V.6	Parallel: <i>M</i> de <i>N</i>	31
V.7	IfThenElse e WhileDoElse	33
V.8	Switch<N>: o desvio por valor	33
V.9	ManualSelector: o operador no laço	34
V.10	Escolhendo o nó de controle	34
V.11	No cenário da porta	34
VI	Decoradores	35
VI.1	Os que transformam o resultado	35
VI.2	Os que repetem	36
VI.3	Os que impõem tempo	37
VI.4	Os que guardam o acesso	38
VI.5	SimpleDecoratorNode: um decorador sem herança	39
VI.6	Escrevendo um decorador	39
VI.7	No cenário da porta	40
VII	Nós de ação e de condição	41
VII.1	ConditionNode: perguntas, não comandos	41
VII.2	SyncActionNode: a ação que termina no <i>tick</i>	42
VII.3	StatefulActionNode: pedido e resposta	43
VII.4	AsyncActionNode: uma <i>thread</i> por execução	44
VII.5	CoroActionNode: suspender em vez de bloquear	45
VII.6	Como escolher, na prática	46
VII.7	Ações auxiliares embutidas	47
VII.8	Argumentos que não são portas	47
VII.9	No cenário da porta	48
VIII	O XML e a fábrica	49
VIII.1	A fábrica	49
	VIII.1.1 registerNodeType<T>()	49
	VIII.1.2 O construtor escolhido	50
	VIII.1.3 Registro em tempo de execução	50
VIII.2	O formato XML	51
	VIII.2.1 A validação	51
	VIII.2.2 <include>: dividir em arquivos	52
VIII.3	Da carga à árvore	53
VIII.4	Subárvores e a fronteira de dados	54
	VIII.4.1 <SubTree>: isolamento com remapeamento	54
	VIII.4.2 <SubTreePlus>: três formas de ligar	55
VIII.5	Plugins	56
VIII.6	O objeto Tree	57
VIII.7	Gerando o modelo para o Groot	58
VIII.8	No cenário da porta	58
IX	Observação e depuração	59
IX.1	StatusChangeLogger: a base comum	59
IX.2	StdCoutLogger: o traço no terminal	60

IX.3	FileLogger: o traço binário	60
IX.4	MinitraceLogger: a linha do tempo	61
IX.5	PublisherZMQ e o Groot	62
IX.6	As ferramentas de linha de comando	63
IX.7	Depuração sem <i>logger</i>	63
IX.8	Uma estratégia de depuração	64
IX.9	No cenário da porta	64
X	A infraestrutura de suporte	65
X.1	BT::Any: o valor de tipo apagado	65
X.2	SimpleString: <i>small object optimization</i>	66
X.3	TimerQueue: temporizadores	66
X.4	SharedLibrary: carga dinâmica	67
X.5	StrCat e demangle	68
X.6	Código de terceiros embutido	68
X.7	O que não está aqui	69
XI	O sistema de compilação	70
XI.1	O CMakeLists.txt	70
	XI.1.1 As opções	70
	XI.1.2 A biblioteca que muda de conteúdo	71
	XI.1.3 Fontes por plataforma e tipo de biblioteca	71
	XI.1.4 Avisos e instalação	72
XI.2	ROS 1 e ROS 2	72
XI.3	O empacotamento Conan 2	73
	XI.3.1 O Makefile de conveniência	74
	XI.3.2 Consumindo o pacote	74
XI.4	Compilando o repositório diretamente	75
XI.5	Resumo das dependências	75
XII	Apêndice: uma aplicação completa	76
XII.1	A árvore	76
XII.2	O tipo próprio e a sua conversão	77
XII.3	Uma ação assíncrona sem <i>thread</i>	77
XII.4	Nós que precisam do mundo	78
XII.5	Uma ação com entrada e saída	79
XII.6	O main()	79
XII.7	Compilando	80
XII.8	A saída	81
XII.9	Sete coisas para reparar nesse traço	82
XII.10	Exercícios de leitura	83
XIII	Apêndice: catálogo dos nós embutidos	85
XIII.1	Nós de controle	85
XIII.2	Decoradores	85
XIII.3	Ações	86
XIII.4	Resumo por tipo	86
XIII.5	IDs que não são o nome da classe	86
XIII.6	Como este catálogo foi verificado	87
	Referências	89

Glossário	90
Índice remissivo	92

Lista de figuras

1	Camadas do código-fonte e sentido das dependências. Nada em utils/ conhece o conceito de árvore.	7
2	Um <i>tick</i> percorre apenas o caminho que a lógica dos nós de controle abre. A porta fechada faz a Sequence falhar sem executar a segunda ação, e o Fallback passa à alternativa.	8
3	A mesma árvore com e sem memória. À esquerda a bateria é conferida uma única vez; à direita, a cada <i>tick</i> — e uma queda de bateria interrompe o movimento em curso.	9
4	Ciclo de vida de um nó. As setas sólidas são iniciativa do próprio nó; as tracejadas, do pai (ou do Tree).	9
5	A hierarquia fundamental. O nível intermediário fixa a aridade; a política fica nas folhas da hierarquia (não da árvore).	10
6	Os quatro passos de toda aplicação. Os três primeiros ocorrem uma vez, na carga; o quarto é o laço da aplicação.	10
7	A árvore do cenário. A subárvore DoorClosed — inversor, RetryUntilSuccessful e travessia — está detalhada na Seção XII.	11
8	Os dois caminhos de uma entrada. O caminho de cima passa pelo <i>blackboard</i> ; o de baixo, quando o atributo não está entre chaves, converte o texto diretamente e nunca toca no <i>blackboard</i> .	23
9	Um caminho de decisão para escolher o nó de controle. A segunda pergunta — “preciso reavaliar o que já deu certo?” — é a que separa as variantes reativas.	34
10	Escolha da classe base de uma ação. O caminho da direita é sempre preferível ao de baixo.	46
11	As seis etapas da carga. As três primeiras acontecem uma vez por arquivo; as três últimas, uma vez por nó.	53
12	A fronteira entre <i>blackboards</i> . Só as chaves listadas no mapa de remapeamento atravessam; todas as outras são locais à subárvore.	56
13	Qual instrumento usar para cada sintoma. Os três primeiros não exigem nenhuma dependência externa.	64
14	As quatro sondagens que alteram o conteúdo e a ABI da biblioteca. Nenhuma delas é uma opção declarada; todas dependem do que estiver instalado.	71

Lista de tabelas

1	Os quatro valores de <code>NodeStatus</code> e o papel de cada um.	7
2	As exceções da biblioteca e o critério de escolha entre elas, segundo os comentários do cabeçalho.	18
3	As formas que um atributo de porta pode assumir e o que cada uma significa. O "=" não é escrito no XML: é o que <code>assignDefaultRemapping()</code> grava ao configurar um nó criado diretamente em C++, sem <i>parser</i> — como fazem os testes da biblioteca.	22
4	Os nós de controle registrados pela fábrica. Todos devolvem <code>RUNNING</code> enquanto o filho ativo não conclui.	28
5	A renomeação entre a v2 e a v3, conforme <code>docs/MigrationGuide.md</code> . Um <code><Sequence></code> de um arquivo da v2 era reativo; na v3 é o contrário.	31
6	Os decoradores registrados pela fábrica, por família. O <code>SubTree</code> e o <code>SubTreePlus</code> são tecnicamente decoradores, mas pertencem ao Seção VIII.4.	35
7	As classes base de folha e o que cada uma pede em troca.	41
8	As regras que <code>VerifyXML()</code> aplica, em <code>src/xml_parsing.cpp</code> .	52
9	As ferramentas de <code>tools/</code> . O <code>bt3_recorder</code> exige <code>ZeroMQ</code> ; os outros dois, não.	63
10	As conversões numéricas de <code>SafeAny::details::convertNumber</code> e o que cada uma confere antes do <code>static_cast</code> .	66
11	As bibliotecas embutidas. As quatro primeiras somam mais de 9 000 linhas — várias vezes o tamanho do núcleo da <code>BehaviorTree.CPP</code> .	68
12	As opções declaradas em <code>CMakeLists.txt</code> . Note que todas são <code>ON</code> por padrão.	70
13	Os alvos do <code>Makefile</code> do <i>fork</i> . O padrão é <code>BUILD_TYPE=Debug</code> e <code>SHARED=True</code> ; ambos são sobrescrevíveis na linha de comando.	74
14	As dependências e o custo de dispensar cada uma. No empacotamento deste <i>fork</i> , apenas as duas primeiras estão presentes.	75
15	Nós de controle. [†] registrado apenas se a biblioteca foi compilada com <code>ncurses</code> — não é o caso nesta configuração (Seção V.9).	85
16	Decoradores. <code>SubTree</code> e <code>SubTreePlus</code> têm <code>type()</code> igual a <code>SUBTREE</code> , mas herdam de <code>DecoratorNode</code> .	86
17	Ações embutidas. Nenhuma delas devolve <code>RUNNING</code> .	86
18	Contagem por tipo, conforme <code>factory.manifests()</code> . Os dois nós de subárvore contam como <code>SubTree</code> , e não como <code>Decorator</code> , por causa da ordem dos testes em <code>getType<T>()</code> (Seção III.5).	87
19	IDs de registro que diferem do nome da classe. O primeiro é o único em que a diferença é um erro de digitação (Seção VI.2).	87

Aviso

Este relatório foi construído com o auxílio de inteligência artificial generativa e revisado manualmente por um grupo de especialistas humanos.

Todo o conteúdo técnico foi conferido contra o código-fonte da versão documentada (Seção I.3). A responsabilidade pelo texto final é da equipe revisora.

I Introdução

I.1 O que é a BehaviorTree.CPP

A **BehaviorTree.CPP** [1] é uma biblioteca em C++ para construir e executar *árvores de comportamento* (*behavior trees*) — a estrutura de decisão que substituiu a máquina de estados finitos em boa parte da robótica moderna e dos jogos. Ela não é um planejador nem um controlador: é a máquina que decide, a cada instante, *qual* das ações que você escreveu deve ser executada agora.

A decisão de projeto que a distingue de uma biblioteca convencional, e que governa praticamente todo o restante deste manual, é a *separação entre estrutura e comportamento*: o *comportamento* de cada folha é implementado em C++, mas a *estrutura* da árvore — quais nós existem, como se aninham, com que parâmetros e ligados a quais dados — é descrita em um arquivo **XML**. Em tempo de carga, um *parser* lê esse arquivo e constrói a árvore de objetos correspondente. A consequência prática é direta e poderosa: **reconfigurar um comportamento não exige recompilar a aplicação** — basta editar o arquivo.

Essa escolha só funciona porque três mecanismos do núcleo a sustentam, e os três reaparecerão em todas as seções deste manual:

- **Fábrica por nome** (**BehaviorTreeFactory**) — permite criar um nó a partir de uma *string* de identificação lida do arquivo, o ID de registro.
- **Sistema de portas** (*ports*) — declara estaticamente quais entradas e saídas cada nó tem, e conecta os atributos do XML às entradas do blackboard que os alimentam.
- **O tick e o NodeStatus** — um único protocolo de execução, com quatro valores possíveis, ao qual absolutamente todo nó obedece, dos embutidos aos seus.

Na prática, toda aplicação da biblioteca é uma árvore de objetos **TreeNode** cuja raiz é devolvida dentro de um **Tree** — a estrutura que detém a posse dos nós, a pilha de *blackboards* e os manifestos. A topologia completa dessa árvore é declarada em um arquivo XML; o *parser* a constrói em tempo de carga e a mantém viva enquanto o objeto **Tree** existir.

Por que — por que árvores de comportamento e não máquinas de estados

Uma máquina de estados finitos codifica a lógica nas *transições*: para N estados há até N^2 arestas, e acrescentar um estado obriga a rever as arestas que chegam nele. A árvore de comportamento move a lógica para os *nós de controle* internos, e o resultado é composicional: uma subárvore que funciona continua funcionando ao ser pendurada em outro lugar, porque tudo o que ela promete ao pai é um dos três resultados de um *tick*. É essa propriedade — e não o desenho em forma de árvore — que a torna escalável.

I.2 Objetivo e público-alvo

Este documento é um **manual técnico de referência** voltado a desenvolvedores que pretendem *usar* a BehaviorTree.CPP para coordenar um sistema ou *estender* a biblioteca com nós próprios. Pressupõe familiaridade com C++ moderno; não é um tutorial de usuário final nem uma introdução teórica a árvores de comportamento.

O manual privilegia as *decisões arquiteturais* — o porquê de cada mecanismo — em vez da exposição exaustiva de código. O objetivo é que o leitor compreenda *como pensar* dentro da biblioteca, de modo que o código-fonte se torne legível por consequência. Onde o comportamento observado diverge do que a documentação da própria biblioteca afirma, o manual descreve o comportamento *observado* e registra a divergência.

I.3 Versão documentada

Este manual descreve a BehaviorTree.CPP na versão **3.5.6** (de 3 de fevereiro de 2021, conforme o `CHANGELOG.rst`), no *fork* empacotado para consumo interno como `behaviortree.cpp.asa/3.5.6`, cuja diferença principal em relação ao repositório original (`BehaviorTree/BehaviorTree.CPP`) é a adoção de uma receita **Conan 2** sobre o `CMakeLists.txt` existente (Seção XI). Todos os trechos de código, nomes de porta, constantes e valores padrão citados ao longo do texto foram conferidos contra essa árvore de fontes.

O *namespace* é `BT`; os cabeçalhos instalam-se sob `behaviortree_cpp_v3/` e a biblioteca gerada chama-se `libbehaviortree_cpp_v3`.

Quatro características desse empacotamento são consequência do estado atual do *build* e merecem ser conhecidas desde já, porque afetam o planejamento de uma integração:

- **As co-rotinas estão desligadas.** A receita desabilita a busca por Boost, o que faz o `CMakeLists.txt` definir `BT_NO_COROUTINES`; `CoroActionNode` não existe no binário (Seção VII.5).
- **O PublisherZMQ não é compilado.** Sem ZeroMQ, o *logger* que alimenta o Groot ao vivo fica de fora, assim como a ferramenta `bt3_recorder` (Seção IX.5).
- **O ManualSelectorNode não é compilado.** Ele depende de *ncurses*, também desabilitado (Seção V.9).
- **O padrão de linguagem é C++14**, fixado em `CMAKE_CXX_STANDARD` (Seção XI.1).

Onde o texto descreve algo que existe no código mas não está operacional nessa configuração — como o `CoroActionNode` ou o `PublisherZMQ` — isso é dito explicitamente no ponto correspondente.

Armadilha — a v3 não é a v4

Existe uma versão 4 da biblioteca, com mudanças incompatíveis de *API* e de XML: `NodeConfiguration` virou `NodeConfig`, o `SubTreePlus` virou o `SubTree` padrão, o `tickRoot()` deu lugar a `tickOnce()/tickWhileRunning()` e o estado `IDLE` deixou de ser repostado entre *ticks* da mesma forma. Praticamente toda resposta encontrada na Internet hoje pressupõe a v4. **Nada neste manual vale para ela**; o texto todo descreve a v3.5.6.

I.4 Convenções do documento

Nomes de classes, funções e identificadores aparecem em fonte monoespçada (`TreeNode`, `tick()`). Todo o código da biblioteca vive no *namespace* `BT`; o prefixo (`BT::NodeStatus`) é indicado quando relevante. Termos técnicos consagrados em inglês (*tick*, *blackboard*, *port*) são mantidos no original, e os principais estão definidos no glossário, ao final.

Blocos de código. Uma convenção importa mais que as outras: **quando a legenda de um bloco traz um caminho de arquivo**, como `(src/controls/sequence_node.cpp)`, **o conteúdo é transcrição do código-fonte** — eventualmente condensado, com as omissões marcadas por `// ...`, mas nunca reescrito. Blocos sem caminho são ilustrativos: mostram como usar um mecanismo, não como ele está escrito.

Caixas de destaque. Além da caixa cinza de aviso que abre o manual — preliminar e de ocorrência única —, três tipos de quadro interrompem o texto, sempre com o mesmo papel:

Regra — o que é uma Regra

Uma norma da biblioteca — algo que o desenvolvedor precisa fazer, ou não fazer, para que o mecanismo funcione.

Armadilha — o que é uma Armadilha

Um comportamento real do código que contraria a expectativa razoável: um valor padrão surpreendente, um nome que não é o que parece, um comentário de cabeçalho desatualizado, uma funcionalidade que existe pela metade. São o resultado da conferência do texto contra o fonte, e vale lê-las mesmo em diagonal — esta biblioteca tem um número incomum delas.

Por que — o que é um Por quê

A motivação de uma decisão de projeto — a pergunta que o mecanismo responde. São opcionais para usar a biblioteca, e essenciais para estendê-la.

Figuras. Todas as figuras deste manual são desenhos TikZ escritos dentro do próprio capítulo. Não há imagens importadas: reger uma figura é editar o `.tex` ao lado do texto que ela ilustra, e o `make` não depende de nenhuma ferramenta externa para isso.

I.5 Organização do manual

A Seção II estabelece a visão arquitetural: o modelo de execução, o vocabulário do *tick*, a organização do código e o cenário que acompanha o manual inteiro. As seções seguintes descem ao detalhe, na ordem em que os conceitos se apoiam uns nos outros:

- Seção III — o `TreeNode` e a hierarquia de classes: o que significa *tick*, *halt* e `IDLE`.
- Seção IV — portas e *blackboard*: como os dados circulam.
- Seção V — os nós de controle, um a um.
- Seção VI — os decoradores, um a um.
- Seção VII — as quatro formas de escrever uma ação, e quando usar cada uma.
- Seção VIII — o formato XML, a fábrica, as subárvores e os *plugins*.
- Seção IX — observação e depuração: *loggers*, Groot e as ferramentas de linha de comando.
- Seção X — a infraestrutura de suporte (`Any`, `TimerQueue`, `SharedLibrary`, terceiros).

- Seção XI — o sistema de compilação, as opções do CMake e o empacotamento Conan 2.

A **Seção XII** fecha o manual com uma aplicação completa — arquivo XML, `main()` e comandos de *build* — que exercita quase todos os mecanismos discutidos. Quem prefere partir do concreto pode lê-lo primeiro e usar as remissões para voltar aos capítulos. A **Seção XIII** traz o catálogo completo dos nós que a fábrica registra sozinha, com as portas de cada um.

Ao final estão o **glossário** dos termos da biblioteca e um **índice remissivo** de classes, portas e IDs de registro.

II Arquitetura geral

Esta seção apresenta a BehaviorTree.CPP de cima: primeiro as decisões de projeto que moldam tudo, depois a estrutura estática (hierarquia de classes e organização do código) e, por fim, o comportamento dinâmico (o *tick* e a propagação de estados). O detalhamento de cada peça fica para as seções seguintes; aqui interessa o vocabulário e a lógica do conjunto.

II.1 Princípios de projeto

Quatro decisões arquiteturais permeiam a biblioteca. Mantê-las em mente torna o restante do manual previsível.

1. **Composição dirigida por dados.** Estrutura em XML, comportamento em C++ (Seção I.1). Favorece experimentação rápida e reaproveitamento de subárvores, ao custo de um núcleo de *runtime* mais elaborado — o preço que a biblioteca paga uma vez para que toda aplicação se beneficie.
2. **Tudo é um `TreeNode`.** Da raiz à menor condição, todos compartilham a mesma maquinaria de execução, estado e notificação de mudança. Essa uniformidade significa que um mecanismo aprendido uma vez se aplica em toda a árvore — e que um *logger* escrito uma vez observa qualquer nó, inclusive os seus.
3. **Um único protocolo de retorno.** Todo *tick* devolve um `NodeStatus`. É a estreiteza dessa interface que torna as subárvores compostáveis: o pai não sabe nem precisa saber se o filho é uma condição trivial ou uma subárvore de duzentos nós.
4. **Execução sem *threads* por padrão.** A árvore é percorrida na *thread* de quem chamou `tickRoot()`. Concorrência é opcional, localizada e explícita — só aparece dentro de um `AsyncActionNode`, de um `TimerQueue` ou de um *logger* de rede.

Regra — o laço de execução é seu

A biblioteca não tem *main loop*, relógio nem agendador. Quem decide a frequência dos *ticks* é a aplicação, tipicamente com um laço `while (status == RUNNING) { tickRoot(); sleep(...); }`. Nunca chame `tickRoot()` em laço apertado sem pausa: a árvore não é autolimitada, e uma condição barata será reavaliada milhões de vezes por segundo.

II.2 Organização do código-fonte

O repositório separa cabeçalhos públicos (`include/behaviortree_cpp_v3/`) de implementação (`src/`), com dependência fluindo de cima para baixo. Na fundação, `utils/` reúne infraestrutura genérica que não sabe nada de árvores (tipo apagado, *string view*, carga de bibliotecas dinâmicas). Sobre ela, o *núcleo* define os tipos básicos, o *blackboard* e o `TreeNode`. Acima vêm as três famílias de nós concretos, e no topo a fábrica, o *parser* XML e os *loggers*.

A direção única das dependências é o que permite, por exemplo, usar `BT::Any` ou `BT::SharedLibrary` isoladamente, e o que explica por que o estudo da biblioteca segue naturalmente essa mesma ordem ascendente — que é a ordem dos capítulos deste manual.

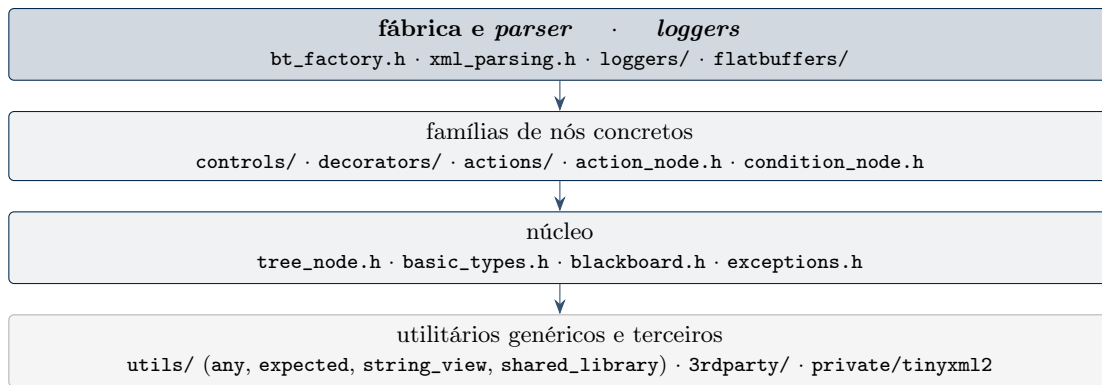


Figura 1: Camadas do código-fonte e sentido das dependências. Nada em `utils/` conhece o conceito de árvore.

Armadilha — `utils/` não é só código próprio

Quatro dos arquivos em `include/behaviortree_cpp_v3/utils/` são bibliotecas de terceiros embutidas (*vendored*), não código do projeto: `any.hpp` (*linb::any*), `expected.hpp` (*nonstd::expected*), `string_view.hpp` (*nonstd::string_view*) e boa parte de `shared_library.h` (POCO). Somam mais de 3 500 linhas — mais que todo o núcleo. Ao medir ou auditar a biblioteca, separe-os (Seção X.6).

II.3 O modelo de execução: o *tick*

Definida a estrutura, resta o comportamento no tempo. E aqui a biblioteca é deliberadamente minimalista: existe **uma** operação, o *tick*, e ela devolve **um** de quatro valores.

```

1  /// Enumerates the possible types of nodes
2  enum class NodeType
3  {
4      UNDEFINED = 0, ACTION, CONDITION, CONTROL, DECORATOR, SUBTREE
5  };
6
7  /// Enumerates the states every node can be in after execution during a particular
8  /// time step.
9  /// IMPORTANT: Your custom nodes should NEVER return IDLE.
10 enum class NodeStatus
11 {
12     IDLE = 0, RUNNING, SUCCESS, FAILURE
13 };

```

Listagem 1: Os dois enumerados que sustentam todo o modelo, em `include/behaviortree_cpp_v3/basic_types.h`.

A leitura correta dos quatro valores separa *resultado* de *estado*:

Valor	Papel	Significado
SUCCESS	resultado	terminei, e deu certo
FAILURE	resultado	terminei, e não deu certo
RUNNING	resultado	ainda não terminei; me chame de novo
IDLE	estado	não estou em execução; nunca fui <i>ticado</i> ou fui <i>halted</i>

Tabela 1: Os quatro valores de `NodeStatus` e o papel de cada um.

Regra — nunca devolva IDLE

Um `tick()` de usuário deve devolver `SUCCESS`, `FAILURE` ou `RUNNING`. `IDLE` é o estado em que um nó *está* antes do primeiro *tick* e depois de um `halt`, e todos os nós de controle embutidos tratam um filho que devolve `IDLE` como erro de programação: lançam `LogicError("A child node must never return IDLE")`.

A propagação. Um *tick* entra pela raiz e desce pela árvore segundo a lógica de cada nó de controle — que é justamente o que distingue uma *Sequence* de um *Fallback*. Nem todo nó é visitado a cada *tick*: um *Fallback* para no primeiro filho que devolve `SUCCESS`, e os irmãos à direita simplesmente não são chamados naquela passagem. O resultado sobe de volta, transformado a cada nível.

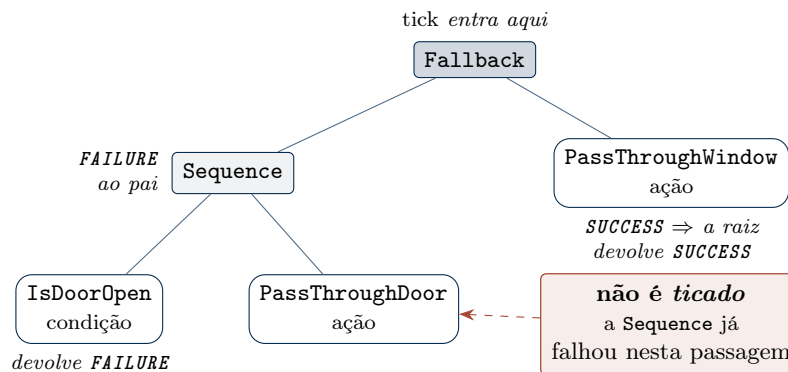


Figura 2: Um *tick* percorre apenas o caminho que a lógica dos nós de controle abre. A porta fechada faz a *Sequence* falhar sem executar a segunda ação, e o *Fallback* passa à alternativa.

O `RUNNING` e a duração. Se todo *tick* tivesse de terminar dentro de si mesmo, uma ação de dois segundos travaria a árvore por dois segundos. O `RUNNING` é a resposta: a ação devolve `RUNNING`, o controle sobe até a aplicação, e no *tick* seguinte a árvore retoma. É por isso que a frequência dos *ticks* importa — ela é a granularidade com que a árvore pode mudar de ideia.

II.4 Memória e reatividade

Do `RUNNING` nasce a pergunta central de todo projeto de árvore: *quando um filho fica `RUNNING`, o que acontece com os irmãos que já tinham terminado?* A biblioteca oferece as duas respostas, e escolher errado é a fonte número um de comportamento inesperado.

- Um nó com memória guarda o índice do filho corrente e, no *tick* seguinte, **retoma dali**. Os filhos que já devolveram `SUCCESS` não são reexecutados.
- Um nó reativo **recomeça do primeiro filho** a cada *tick*. Os que já tinham dado `SUCCESS` são reavaliados; se algum mudou de ideia, a árvore reage imediatamente.

Por que — por que as duas variantes coexistem

Reatividade não é gratuita: reavaliar tudo a cada *tick* custa tempo e exige que *todas* as condições à esquerda sejam baratas e sem efeito colateral. E há lógicas em que reavaliar é simplesmente errado —

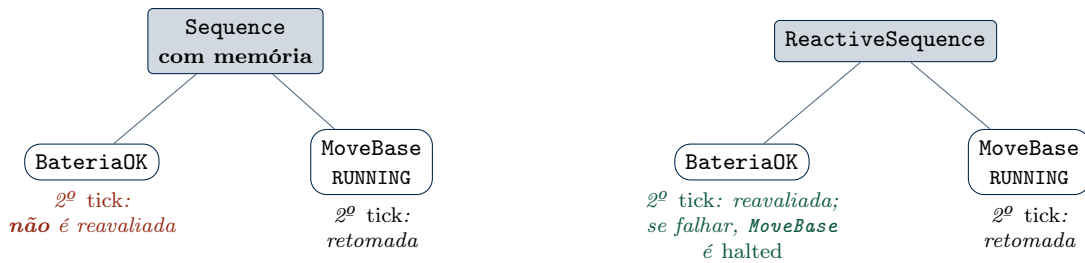


Figura 3: A mesma árvore com e sem memória. À esquerda a bateria é conferida uma única vez; à direita, a cada *tick* — e uma queda de bateria interrompe o movimento em curso.

uma sequência de montagem não deve reabrir a garra porque a condição “garra aberta” voltou a ser falsa *por causa* do passo seguinte. O par memória/reatividade é a forma de expressar essa diferença na estrutura, e não no código das folhas.

Regra — um único filho assíncrono nos nós reativos

Os cabeçalhos de `ReactiveSequence` e `ReactiveFallback` são explícitos: “*to work properly, this node should not have more than a single asynchronous child*”. Com dois filhos que devolvem `RUNNING`, o primeiro deles impede o segundo de ser alcançado a cada *tick* — e, no caso da `ReactiveSequence`, o código ainda *halts* todos os irmãos à direita do que ficou `RUNNING` (Seção V.4).

II.5 O ciclo de vida de um nó

Juntando *tick*, resultado e *halt*, o ciclo de vida de qualquer nó cabe num diagrama de quatro estados. Ele vale para todos os nós, dos embutidos aos seus, e conhecê-lo dispensa a leitura de metade do código-fonte.

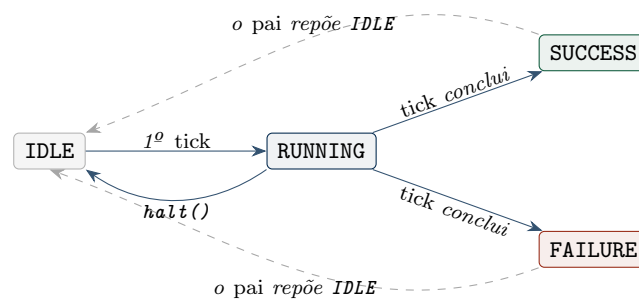


Figura 4: Ciclo de vida de um nó. As setas sólidas são iniciativa do próprio nó; as tracejadas, do pai (ou do `Tree`).

Três consequências merecem destaque desde já, e serão detalhadas na Seção III:

1. **Quem devolve a IDLE é o pai, não o nó.** Um `DecoratorNode` repõe o filho a IDLE assim que ele conclui; os nós de controle chamam `haltChildren()` ao encerrar um ciclo; e `Tree::tickRoot()` repõe a raiz quando ela devolve `SUCCESS` ou `FAILURE`.
2. **Um nó em SUCCESS ou FAILURE não é reexecutado** enquanto não voltar a IDLE. O aviso está no topo de `action_node.h` e é a razão de tantas chamadas a `haltChildren()` espalhadas pelos nós de controle.

3. `halt()` só faz sentido para quem devolve `RUNNING`. `SyncActionNode` e `ConditionNode` declaram `halt()` como final e ele apenas repõe o estado — não há o que interromper.

II.6 As três famílias de nós

A hierarquia de classes é curta e cada nível acrescenta exatamente uma capacidade: `TreeNode` define o protocolo; `LeafNode`, `DecoratorNode` e `ControlNode` definem a aridade (nenhum filho, um filho, N filhos); e as classes concretas definem a política.

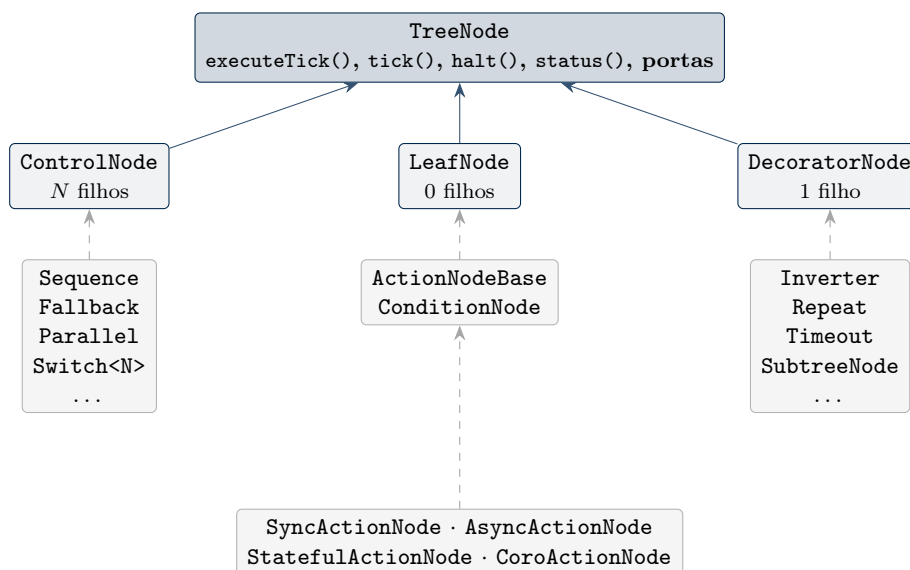


Figura 5: A hierarquia fundamental. O nível intermediário fixa a aridade; a política fica nas folhas da hierarquia (não da árvore).

Note que `SubtreeNode` é um *decorador*: uma subárvore inteira, vista de fora, é apenas um nó de um único filho — e é exatamente essa uniformidade que permite pendurá-la em qualquer lugar (Seção VIII.4).

II.7 Da configuração à árvore: os quatro passos

A construção de uma árvore é sempre a mesma sequência, e vale fixá-la agora porque cada capítulo detalha um de seus passos.

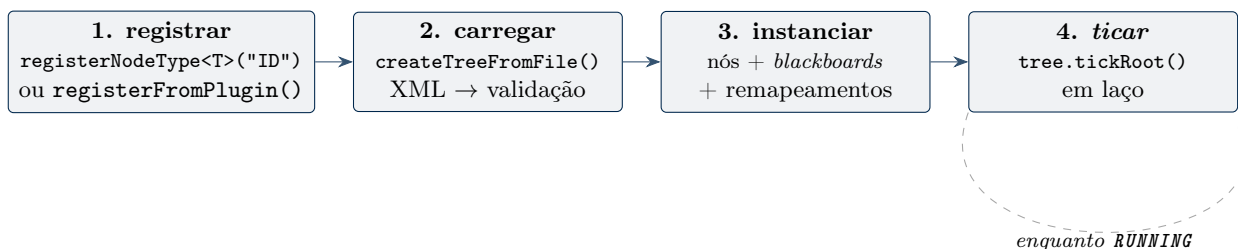


Figura 6: Os quatro passos de toda aplicação. Os três primeiros ocorrem uma vez, na carga; o quarto é o laço da aplicação.

Os passos 1 a 3 estão na Seção VIII; o passo 4 é o *tick* desta seção. O detalhe que mais surpreende quem

vem de outras bibliotecas é que *os três primeiros passos são caros e o quarto é barato*: instanciar a árvore lê XML, resolve tipos e aloca; *ticar* percorre ponteiros.

Regra — o Tree é o dono, e não é copiável

Tree guarda os `shared_ptr` de todos os nós, a pilha de *blackboards* e os manifestos. Ele é *move-only*: copiar está explicitamente removido (= `delete`). Quando o objeto sai de escopo, o destrutor chama `haltTree()` e destrói todos os nós — guardar um `TreeNode*` que sobreviva ao **Tree** é um *dangling pointer*.

II.8 Um cenário para acompanhar o manual

As seções seguintes descem, uma a uma, às peças da biblioteca. Para que elas não fiquem sendo um catálogo de mecanismos isolados, vale fixar desde já um **cenário concreto**, ao qual cada capítulo voltará para mostrar a fatia que lhe cabe. O cenário é o exemplo `t05_crossdoor` do próprio repositório, e é pequeno de propósito:

Um robô precisa atravessar uma porta. Se ela já estiver aberta, ele passa. Se não, tenta abri-la — até quatro vezes, porque a fechadura emperra — e então passa. Se nada disso funcionar, ele passa pela janela. Ao final, feche a porta.

Uma condição, cinco ações, um *fallback*, um *decorator* de repetição, um inversor e uma subárvore — o suficiente para exercitar quase toda a biblioteca, e pequeno o bastante para caber em meia página de XML.

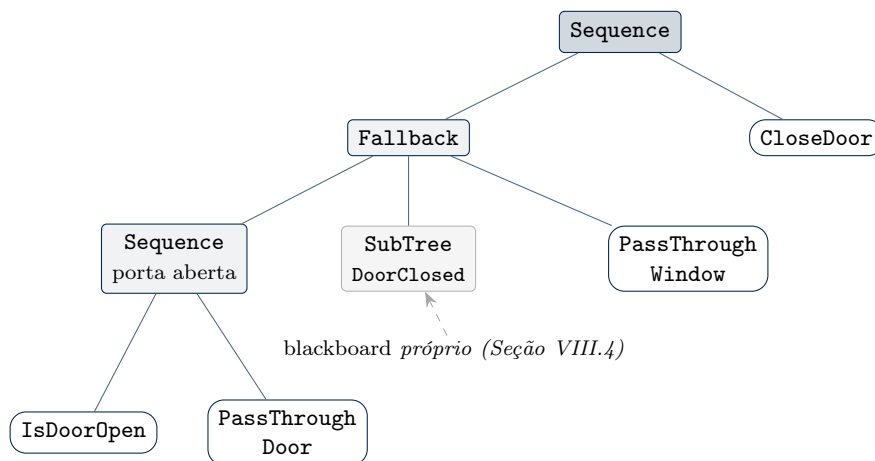


Figura 7: A árvore do cenário. A subárvore `DoorClosed` — inversor, `RetryUntilSuccessful` e travessia — está detalhada na Seção XII.

Cada capítulo termina com um parágrafo curto — “no cenário da porta...” — ligando o que acabou de ser explicado a esse desenho. O arquivo XML inteiro, o `main()` que o carrega e os comandos de *build* estão na Seção XII. Quem preferir partir do concreto pode lê-lo agora e voltar.

III O modelo de nós

Esta seção descreve a classe `TreeNode` e as três subclasses que fixam a aridade. É o capítulo mais denso do manual, e o mais rentável: todo o resto — nós de controle, decoradores, ações, *loggers* — é aplicação direta do que está aqui.

III.1 TreeNode: o contrato

`TreeNode` é abstrata e magra. Ela não sabe quantos filhos tem (isso é das subclasses), não sabe o que faz (isso é do usuário) e não sabe quando será chamada (isso é da aplicação). O que ela define é o *contrato*: como se executa um nó, como se descobre em que estado ele está e como se avisa o mundo quando esse estado muda.

```
1 class TreeNode
2 {
3     public:
4         typedef std::shared_ptr<TreeNode> Ptr;
5
6         TreeNode(std::string name, NodeConfiguration config);
7         virtual ~TreeNode() = default;
8
9         /// The method that should be used to invoke tick() and setStatus();
10        virtual BT::NodeStatus executeTick();
11
12        /// The method used to interrupt the execution of a RUNNING node.
13        /// Only Async nodes that may return RUNNING should implement it.
14        virtual void halt() = 0;
15
16        bool isHalted() const;
17        NodeStatus status() const;
18        const std::string& name() const; // nome da INSTANCIA
19        virtual NodeType type() const = 0;
20
21        /// Blocking function that will sleep until the setStatus() is called with
22        /// either RUNNING, FAILURE or SUCCESS.
23        BT::NodeStatus waitValidStatus();
24
25        StatusChangeSubscriber subscribeToStatusChange(StatusChangeCallback callback);
26
27        uint16_t UID() const;
28        const std::string& registrationName() const; // ID usado no XML
29        const NodeConfiguration& config() const;
30
31        template <typename T> Result getInput (const std::string& key, T& destination) const;
32        template <typename T> Optional<T> getInput(const std::string& key) const;
33        template <typename T> Result setOutput(const std::string& key, const T& value);
34
35    protected:
36        /// Method to be implemented by the user
37        virtual BT::NodeStatus tick() = 0;
38
39        void setStatus(NodeStatus new_status);
40        // ...
41};
```

Listagem 2: A interface pública essencial, em `include/behaviortree_cpp_v3/tree_node.h` (condensada).

Três pares de métodos merecem ser lidos com atenção, porque a distinção entre os membros de cada par é onde mora a maior parte dos enganos.

`tick()` versus `executeTick()`. `tick()` é `protected` e puramente virtual: é o que *você* escreve. `executeTick()` é público e virtual: é o que *o pai* chama. A implementação base é de três linhas e explica exatamente a divisão de trabalho:

```
1 NodeStatus TreeNode::executeTick()
2 {
3     const NodeStatus status = tick();
4     setStatus(status);
5     return status;
6 }
```

Listagem 3: `executeTick()` base, em `src/tree_node.cpp`.

Ou seja: `executeTick()` é o *envelope* que garante que o estado publicado do nó reflita o que o `tick()` devolveu. Quem sobrescreve `executeTick()` está mudando o *protocolo*, não o comportamento — e por isso são poucos os que fazem isso, todos na biblioteca: `DecoratorNode` (para repor o filho a `IDLE`), `SyncActionNode` (para proibir `RUNNING`), `AsyncActionNode` (para lançar a *thread*) e `CoroActionNode` (para retomar a *corotina*).

Regra — sobrescreva `tick()`, não `executeTick()`

Num nó de usuário, `tick()` é o ponto de extensão. Três das quatro classes de ação declaram `executeTick()` como `final` justamente para impedir o engano — e o comentário no cabeçalho de `AsyncActionNode` é explícito: “*This method spawn a new thread. Do NOT remove the ‘final’ keyword*”.

`name()` versus `registrationName()`. O primeiro é o nome da *instância* — o atributo `name` do XML, opcional. O segundo é o ID de registro, a *string* pela qual a fábrica sabe construir aquele tipo. Um mesmo `registrationName()` aparece em muitas instâncias com `name()` diferentes. Quando o XML omite `name`, o *parser* usa o próprio ID como nome da instância — o que faz os dois coincidirem e alimenta a confusão.

`status()` versus `isHalted()`. O segundo não tem nada de sofisticado:

```
1 bool TreeNode::isHalted() const
2 {
3     return status_ == NodeStatus::IDLE;
4 }
```

Listagem 4: `isHalted()`, em `src/tree_node.cpp`.

Armadilha — `isHalted()` não significa “foi interrompido”

O nome sugere um evento (“alguém chamou `halt()` em mim”); o código diz um estado (“estou em `IDLE`”). Um nó recém-construído, que nunca foi *ticado*, responde `true` a `isHalted()`. Além disso, essa leitura de `status_` é feita **sem tomar o *mutex***, ao contrário de `status()` — o que é uma leitura sem sincronização quando usada de outra *thread*. Prefira comparar `status() == NodeStatus::IDLE` explicitamente.

III.2 Estado, notificação e concorrência

O estado de um nó não é um simples membro: escrevê-lo dispara duas notificações.

```

1 void TreeNode::setStatus(NodeStatus new_status)
2 {
3     NodeStatus prev_status;
4     {
5         std::unique_lock<std::mutex> UniqueLock(state_mutex_);
6         prev_status = status_;
7         status_ = new_status;
8     }
9     if (prev_status != new_status)
10    {
11        state_condition_variable_.notify_all();
12        state_change_signal_.notify(std::chrono::high_resolution_clock::now(), *this,
13                                   prev_status, new_status);
14    }
15 }

```

Listagem 5: `setStatus()`, o coração do modelo, em `src/tree_node.cpp`.

Três consequências:

1. **A transição só é notificada se houver transição.** Um nó que devolve `RUNNING` dez *ticks* seguidos gera *um* evento, não dez. É o que torna viável ligar um *logger* a uma árvore *ticada* a 100 Hz.
2. **Há uma *condition variable* por nó**, usada por `waitValidStatus()` — uma espera bloqueante até o nó sair de `IDLE`. É o mecanismo que permite a uma *thread* externa sincronizar-se com um nó; na prática, quase ninguém a usa.
3. **O *callback* roda na *thread* de quem chamou `setStatus()`**. Para um `AsyncActionNode`, isso é a *thread* da ação, não a do *tick*. Um *logger* precisa ser seguro sob concorrência — e é por isso que `PublisherZMQ` protege o seu *buffer* com *mutex* (Seção IX.5).

O Signal. A notificação usa uma implementação própria e minúscula do padrão observador, em `utils/signal.h`. O assinante recebe um `shared_ptr` e o sinal guarda um `weak_ptr`: quando o assinante morre, a próxima notificação percebe o ponteiro expirado e o remove da lista.

```

1 void notify(CallableArgs... args)
2 {
3     for (size_t i = 0; i < subscribers_.size(); )
4     {
5         if (auto sub = subscribers_[i].lock()) { (*sub)(args ...); i++; }
6         else { subscribers_.erase(subscribers_.begin() + i); }
7     }
8 }

```

Listagem 6: A remoção preguiçosa de assinantes, em `include/behaviortree_cpp_v3/utils/signal.h`.

Regra — o *subscriber* precisa continuar vivo

`subscribeToStatusChange()` devolve um `shared_ptr`. Se você descartá-lo, o *callback* morre na hora — silenciosamente, sem erro. É exatamente por isso que `StatusChangeLogger` guarda um `std::vector` de assinaturas como membro, e por que os *loggers* do exemplo da Seção XII são variáveis locais do `main()` que vivem até o fim.

Armadilha — o Signal não é seguro sob concorrência

`notify()` e `subscribe()` manipulam o mesmo `std::vector` sem *mutex*, e `notify()` é chamado de dentro de `setStatus()` — que, num `AsyncActionNode`, roda na *thread* da ação. Assinar um nó enquanto

a árvore está sendo *ticada* é uma condição de corrida. Ligue todos os *loggers* **antes** do primeiro `tickRoot()`.

III.3 O identificador único (UID)

Cada nó recebe, na construção, um `uint16_t` sequencial. É esse número — e não o nome — que os *loggers* binários e o Groot usam para identificar um nó numa transição.

```
1 static uint16_t getUID()
2 {
3     static uint16_t uid = 1;
4     return uid++;
5 }
```

Listagem 7: O contador de UID, em `src/tree_node.cpp`.

Armadilha — o contador de UID é global, não por árvore

São quatro problemas num contador de cinco linhas. **(a)** O contador é `static` do processo: a segunda árvore criada continua a numeração da primeira, e os UID de uma árvore não começam em 1. **(b)** Destruir uma árvore *não* devolve os números; um programa que reconstrói a árvore a cada minuto esgota os 65 535 valores. **(c)** No estouro, o contador volta a zero *sem aviso* e dois nós vivos passam a ter o mesmo UID — com o que o *logger* binário e o Groot passam a atribuir transições ao nó errado. **(d)** O incremento não é atômico: construir árvores em duas *threads* é uma corrida. Nenhum desses casos é verificado em tempo de execução.

III.4 As três aridades

Sobre `TreeNode`, três subclasses fixam quantos filhos um nó pode ter. Não acrescentam política nenhuma — apenas armazenamento e as operações de *halt* correspondentes.

III.4.1 LeafNode: nenhum filho

É a mais simples da biblioteca: herda tudo e não acrescenta nada além do construtor. Serve de raiz comum a `ActionNodeBase` e `ConditionNode` (Seção VII), e é o ponto onde “não tenho filhos” vira um fato do sistema de tipos em vez de uma convenção.

III.4.2 ControlNode: *N* filhos

```
1 class ControlNode : public TreeNode
2 {
3     protected:
4         std::vector<TreeNode*> children_nodes_;
5
6     public:
7         void addChild(TreeNode* child);
8         size_t childrenCount() const;
9         const std::vector<TreeNode*>& children() const;
10 }
```

```

11     virtual void halt() override;
12     void haltChildren();
13     [[ deprecated("deprecated: please use explicitly haltChildren() or haltChild(i)") ]]
14     void haltChildren(size_t first);
15     void haltChild(size_t i);
16
17     virtual NodeType type() const override final { return NodeType::CONTROL; }
18 };

```

Listagem 8: ControlNode, em include/behaviortree_cpp_v3/control_node.h (condensada).

O detalhe que importa está em `haltChild()`:

```

1     void ControlNode::haltChild(size_t i)
2     {
3         auto child = children_nodes_[i];
4         if (child->status() == NodeStatus::RUNNING)
5         {
6             child->halt();
7         }
8         child->setStatus(NodeStatus::IDLE);
9     }

```

Listagem 9: O *halt* de um filho, em src/control_node.cpp.

Duas coisas acontecem aí, e confundi-las custa caro. `halt()` só é chamado se o filho estiver `RUNNING` — é o cancelamento propriamente dito, que propaga para baixo. Mas o `setStatus(IDLE)` é *incondicional*: mesmo um filho que terminou em `SUCCESS` volta a `IDLE`. É esse segundo efeito, e não o primeiro, que os nós de controle usam ao encerrar um ciclo — sem ele, o nó jamais seria executado de novo (Seção II.5).

Armadilha — os filhos são ponteiros crus, e o pai não é dono

`children_nodes_` é um `std::vector<TreeNode*>`. A posse dos nós é do `Tree`, que guarda `shared_ptr` em `Tree::nodes`. Um `ControlNode` construído à mão, fora do *parser*, só é válido enquanto alguém mantiver os filhos vivos — `addChild()` não faz nenhuma verificação e nem sequer rejeita `nullptr`. O erro só aparece depois, em `applyRecursiveVisitor()`, como `LogicError("One of the children of a DecoratorNode or ControlNode is nullptr")`.

III.4.3 DecoratorNode: exatamente um filho

`DecoratorNode` é a única das três que sobrescreve `executeTick()`, e a razão é sutil:

```

1     NodeStatus DecoratorNode::executeTick()
2     {
3         NodeStatus status = TreeNode::executeTick();
4         NodeStatus child_status = child()->status();
5         if ( child_status == NodeStatus::SUCCESS || child_status == NodeStatus::FAILURE )
6         {
7             child()->setStatus(NodeStatus::IDLE);
8         }
9         return status;
10    }

```

Listagem 10: `DecoratorNode::executeTick()`, em src/decorator_node.cpp.

Por que — por que o decorador repõe o filho a `IDLE`

Um decorador como `RepeatNode` precisa executar o mesmo filho várias vezes. Mas um nó que terminou em `SUCCESS` não é reexecutado enquanto não voltar a `IDLE`. Se a reposição ficasse a cargo de cada

decorador, qualquer decorador de usuário que a esquecesse teria um filho que executa uma única vez e depois some. Colocá-la no `executeTick()` da classe base resolve o problema para todos de uma vez — inclusive para os que ainda não foram escritos.

`setChild()` é a contrapartida de `addChild()`, e essa sim verifica: atribuir um segundo filho lança `BehaviorTreeException("Decorator [nome] has already a child assigned")`. No caminho do XML, porém, quem garante a aridade é a validação do *parser* (Seção VIII.2.1), não esta exceção.

III.5 O tipo de um nó, em tempo de compilação

`TreeNode::type()` é virtual, o que custa uma indireção e obriga a ter um objeto. Para os casos em que o tipo é conhecido estaticamente — o registro na fábrica, por exemplo — a biblioteca oferece uma versão resolvida pelo compilador:

```
1  template <typename T>
2  inline NodeType getType()
3  {
4      if( std::is_base_of<ActionNodeBase, T>::value ) return NodeType::ACTION;
5      if( std::is_base_of<ConditionNode, T>::value )  return NodeType::CONDITION;
6      if( std::is_base_of<SubtreeNode, T>::value )    return NodeType::SUBTREE;
7      if( std::is_base_of<SubtreePlusNode, T>::value ) return NodeType::SUBTREE;
8      if( std::is_base_of<DecoratorNode, T>::value )  return NodeType::DECORATOR;
9      if( std::is_base_of<ControlNode, T>::value )    return NodeType::CONTROL;
10     return NodeType::UNDEFINED;
11 }
```

Listagem 11: `getType<T>()`, em `include/behaviortree_cpp_v3/behavior_tree.h`.

A ordem dos testes não é decorativa: `SubtreeNode` é um `DecoratorNode`, e por isso precisa ser testado antes — senão toda subárvore seria classificada como decorador no manifesto e no Groot. É essa função, e não `type()`, que a fábrica usa ao montar o manifesto de um nó registrado (Seção VIII.1).

III.6 Percorrer a árvore

Três funções livres, declaradas em `behavior_tree.h`, cobrem as necessidades de quem precisa varrer uma árvore já construída — e são a base de todos os *loggers*:

- `applyRecursiveVisitor(root, visitor)` — chama o visitante em cada nó, descendo por `ControlNode::children()` ou `DecoratorNode::child()` conforme o caso. Há duas sobrecargas, `const` e `non-const`.
- `printTreeRecursively(root)` — despeja a árvore indentada em `std::cout`. Depuração rápida, sem dependência nenhuma.
- `buildSerializedStatusSnapshot(root, buffer)` — preenche um `vector` de pares (UID, status) com o estado de toda a árvore.

Armadilha — buildSerializedStatusSnapshot não linka como declarada

O cabeçalho declara a função recebendo `const TreeNode*`; o `src/behavior_tree.cpp` a define recebendo `TreeNode*`. São assinaturas diferentes: a declarada nunca é definida, e a definida nunca é declarada. Chamá-la com um ponteiro `const` compila e falha na ligação com *undefined reference*. Nada na biblioteca a chama — o `PublisherZMQ` monta o seu *snapshot* à mão, com `applyRecursiveVisitor` — e por isso o defeito passa despercebido.

A travessia usa `dynamic_cast`. Tanto o visitante quanto o `printTreeRecursively` descobrem a aridade com `dynamic_cast`, não com `type()`. É uma escolha consciente: `type()` diria `SUBTREE` para um `SubtreeNode`, e o visitante precisa saber que ele é estruturalmente um decorador para descer ao filho. O custo é irrelevante, porque a travessia acontece na carga, não a cada *tick*.

III.7 Erros e exceções

A biblioteca sinaliza erro por exceção, com uma hierarquia de três níveis em `exceptions.h`:

Classe	Quando é lançada
<code>BehaviorTreeException</code>	base de todas; herda de <code>std::exception</code>
<code>LogicError</code>	erro que “provavelmente exige refatorar o código”
<code>RuntimeError</code>	erro ligado a dados ou condições de execução

Tabela 2: As exceções da biblioteca e o critério de escolha entre elas, segundo os comentários do cabeçalho.

As três aceitam um número variável de argumentos, concatenados por `StrCat` — daí o padrão `throw RuntimeError("Missing parameter [", NUM_CYCLES, "] in RepeatNode")` que se repete por toda a biblioteca.

Armadilha — a fronteira `LogicError/RuntimeError` não é respeitada

O critério documentado é claro, mas o código não o segue. Um filho que devolve `IDLE` — erro de programação — é `LogicError`, correto; mas o mesmo erro aparece como `std::logic_error` cru em `IfThenElseNode`, `WhileDoElseNode` e `StatefulActionNode`, *fora* da hierarquia da biblioteca. Um `catch (BehaviorTreeException&)` não pega esses três. Para capturar tudo, use `std::exception`.

Regra — `tick()` pode lançar, mas prefira `FAILURE`

Uma exceção que escapa de um `tick()` sobe até o `tickRoot()` da aplicação, deixando a árvore num estado intermediário: os nós que já mudaram de estado permanecem como estavam, e nada foi *halted*. Isso é aceitável para erro de configuração (porta obrigatória ausente, tipo incompatível), que é precisamente o uso da biblioteca. Para falha de domínio — não achei o objeto, o motor não respondeu — devolva `FAILURE` e deixe a árvore decidir. Num `AsyncActionNode`, aliás, a exceção é capturada na *thread*, guardada num `std::exception_ptr` e relançada no *tick* seguinte (Seção VII.4).

III.8 No cenário da porta

Todo nó da Figura 7 é uma instância de uma dessas classes: o `Fallback` e as duas `Sequence` são `ControlNode`; o `Inverter` e o `RetryUntilSuccessful` da subárvore são `DecoratorNode`; o `SubTree` também é um decorador, apesar de `type()` devolver `SUBTREE`; e `IsDoorOpen` é um `ConditionNode`, enquanto as travessias são ações. Quando `PassThroughDoor` falha, é o `haltChildren()` da `Sequence` que devolve `IsDoorOpen` a `IDLE` — e é por isso que, no *tick* seguinte, a condição é reavaliada em vez de reaproveitada.

IV Portas e *blackboard*

O *tick* carrega controle, não dados: ele diz *quando* um nó executa, mas nada sobre *com que valores*. Esta seção descreve o segundo canal — o par porta (*port*)/blackboard — pelo qual a informação circula.

A analogia que a própria documentação usa é a certa: **uma porta de entrada é um argumento de função; uma porta de saída é um valor de retorno**. A diferença é que argumentos e retornos, aqui, são ligados por *nome* em tempo de carga, e não por posição em tempo de compilação.

IV.1 Declarando portas

Um nó anuncia as suas portas por um método estático de nome fixo:

```
1 class MoverPara : public BT::SyncActionNode
2 {
3     public:
4         MoverPara(const std::string& name, const BT::NodeConfiguration& config)
5             : BT::SyncActionNode(name, config) {}
6
7         // O nome e a assinatura são obrigatórios: a biblioteca detecta este método
8         // por SFINAE, e não por herança.
9         static BT::PortsList providedPorts()
10        {
11            return { BT::InputPort<Pose2D>("goal", "destino, em x;y;theta"),
12                    BT::InputPort<double>("tolerancia", 0.25, "raio de aceitacao [m]"),
13                    BT::OutputPort<std::string>("resultado") };
14        }
15
16        BT::NodeStatus tick() override;
17    };
```

Listagem 12: Um nó com uma entrada, uma saída e um valor padrão.

`PortsList` é um `unordered_map<std::string, PortInfo>`, e `PortInfo` guarda quatro coisas: a direção, o `std::type_info` do tipo declarado, uma função de conversão a partir de texto e dois campos textuais (descrição e valor padrão). As três funções de construção — `InputPort`, `OutputPort` e `BidirectionalPort` — são casos particulares de `CreatePort<T>()`, que difere apenas na direção.

Regra — porta com tipo, sempre que possível

`InputPort("chave")` sem parâmetro de *template* é válido e cria uma porta de tipo `void`: sem `type_info` e sem conversor. Ela funciona — o `getInput<T>()` converte na leitura — mas abre mão de duas verificações que a biblioteca faria de graça: a checagem de consistência de tipo entre dois nós que compartilham a mesma entrada do *blackboard* (Seção IV.5.2) e a informação de tipo que o Groot exige. Use `InputPort<T>("chave")`.

A detecção é por SFINAE. A fábrica descobre se um nó tem portas testando se a expressão `T::providedPorts()` é válida e devolve `PortsList`:

```
1 template <typename T, typename = void>
2 struct has_static_method_providedPorts: std::false_type {};
3
4 template <typename T>
5 struct has_static_method_providedPorts<T,
6     typename std::enable_if<std::is_same<decltype(T::providedPorts()),
7     PortsList>::value>::type>
```

```
8 | : std::true_type {};
```

Listagem 13: A detecção do método estático, em `include/behaviortree_cpp_v3/basic_types.h`.

Armadilha — um `providedPorts()` privado é o mesmo que nenhum

A detecção é sensível a acesso: se `providedPorts()` for `private` ou `protected`, a expressão não é válida no contexto do *trait*, a substituição falha e o nó é registrado **com a lista de portas vazia** — sem erro de compilação. Não é hipótese: `SubtreeNode` e `SubtreePlusNode` declaram `providedPorts()` na seção `private`, e os seus manifestos são, de fato, vazios. As portas `__shared_blackboard` e `__autoremap` que eles anunciam ali nunca chegam ao manifesto; funcionam porque o *parser* lê esses atributos diretamente do XML (Seção VIII.4). Num nó seu, o mesmo engano produz o erro "Possible typo? ... the manifest of this node does not contain a port with this name" na carga.

IV.2 Lendo e escrevendo

Dentro do `tick()`, as portas são acessadas por dois métodos de `TreeNode`. `getInput()` tem duas formas: uma que escreve num destino e devolve um `Result`, e outra que devolve um `Optional<T>`.

```
1 BT::NodeStatus MoverPara::tick()
2 {
3     // Forma 1: destino + Result. Conveniente quando ha um valor de fallback.
4     double tol = 0.25;
5     getInput("tolerancia", tol);           // ignora a falha de proposito
6
7     // Forma 2: Optional<T>. Preferivel quando a porta e obrigatoria, porque
8     // res.error() traz a mensagem exata do que faltou.
9     auto res = getInput<Pose2D>("goal");
10    if (!res) {
11        throw BT::RuntimeError("faltou a porta [goal]: ", res.error());
12    }
13    Pose2D destino = res.value();
14    // ...
15    setOutput("resultado", std::string("cheguei"));
16    return BT::NodeStatus::SUCCESS;
17 }
```

Listagem 14: As duas formas de ler uma entrada, e a escrita de uma saída.

`Optional<T>` é um apelido para `nonstd::expected<T, std::string>` — ou seja, ou o valor, ou uma mensagem de erro; e `Result` é `Optional<void>`. O comentário no cabeçalho explica a escolha do nome: “we use the name *Optional* instead of *expected* because it is more intuitive for users that are not up to date with ‘modern’ C++”.

Armadilha — `Optional` não é `std::optional`

Apesar do nome, `BT::Optional<T>` carrega uma mensagem de erro no caso negativo e *não* tem `value_or()` com a semântica que se espera de `std::optional`. Confundir os dois leva a código que compila e ignora a razão da falha — justamente a informação mais útil ao depurar uma árvore que não carrega.

Regra — leia as portas dentro do `tick()`, não no construtor

O `NodeConfiguration` é fixado na construção e nunca muda, mas o *conteúdo* do *blackboard* muda a cada *tick*. Ler uma entrada no construtor captura o valor que existia antes de qualquer nó ter escrito nele — tipicamente, nenhum. Vários nós embutidos deixam isso explícito com o membro

`read_parameter_from_ports_`: quando construídos pelo XML, adiam a leitura para o primeiro `tick()` (Seção VI.2).

IV.3 A resolução de um nome de porta

Este é o mecanismo central do capítulo, e vale segui-lo passo a passo. Quando o XML diz `<MoverPara goal="{destino}" />`, o *parser* guarda em `config().input_ports` o par `("goal", "{destino}")`. Na leitura, `getInput("goal", ...)` faz:

```

1  auto remap_it = config_.input_ports.find(key);
2  if (remap_it == config_.input_ports.end()) // ① a porta nao foi configurada
3      return nonstd::make_unexpected(StrCat("getInput() failed because ...", key, "]);
4
5  auto remapped_res = getRemappedKey(key, remap_it->second);
6  if (!remapped_res) { // ② nao e ponteiro: e literal
7      destination = convertFromString<T>(remap_it->second);
8      return {};
9  }
10 const auto& remapped_key = remapped_res.value(); // ③ e ponteiro
11 const Any* val = config_.blackboard->getAny( (std::string)remapped_key );
12 if (val && val->empty() == false) {
13     if (std::is_same<T, std::string>::value == false && val->type() == typeid(std::string))
14         destination = convertFromString<T>( val->cast<std::string>() ); // ④
15     else
16         destination = val->cast<T>();
17     return {};
18 }
19 return nonstd::make_unexpected(StrCat("getInput() failed because it was unable "
20                                     "to find the key [", key, "] remapped to [",
21                                     remapped_key, "]);

```

Listagem 15: A resolução de um nome, em `include/behaviortree_cpp_v3/tree_node.h` (condensada).

E `getRemappedKey()` decide, em três linhas, o que é ponteiro e o que é literal:

```

1  Optional<StringView> TreeNode::getRemappedKey(StringView port_name,
2                                              StringView remapping_value)
3  {
4      if( remapping_value == "=" ) return {port_name};
5      if( isBlackboardPointer( remapping_value ) ) return {stripBlackboardPointer(remapping_value)};
6      return nonstd::make_unexpected("Not a blackboard pointer");
7  }

```

Listagem 16: O que conta como “ponteiro para o *blackboard*”, em `src/tree_node.cpp`.

No XML	Interpretação	De onde vem o valor
<code>msg="oi"</code>	literal	convertido do próprio texto
<code>msg="{k}"</code>	ponteiro	entrada <code>k</code> do <i>blackboard</i>
<code>msg="\${k}"</code>	ponteiro (forma antiga)	entrada <code>k</code> do <i>blackboard</i>
<code>msg</code> ausente	padrão da porta, se houver	convertido do texto do padrão
(interno) <code>"=</code>	ponteiro homônimo	entrada de mesmo nome da porta

Tabela 3: As formas que um atributo de porta pode assumir e o que cada uma significa. O `"=` não é escrito no XML: é o que `assignDefaultRemapping()` grava ao configurar um nó criado diretamente em C++, sem *parser* — como fazem os testes da biblioteca.

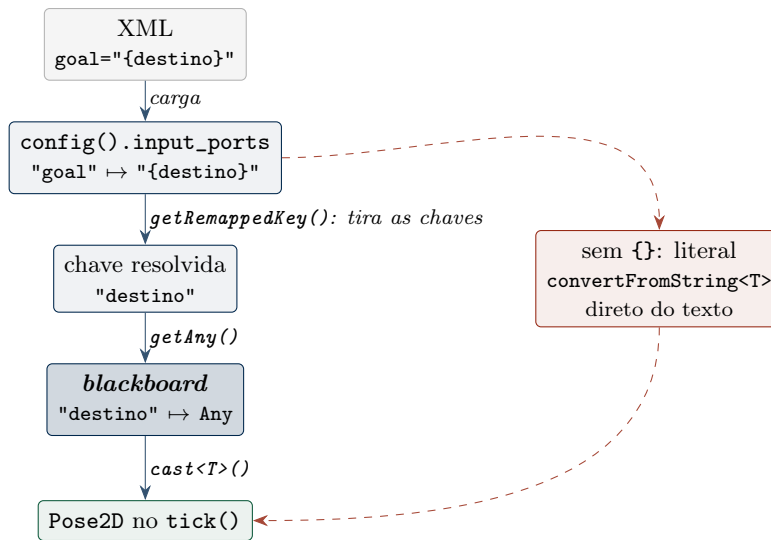


Figura 8: Os dois caminhos de uma entrada. O caminho de cima passa pelo *blackboard*; o de baixo, quando o atributo não está entre chaves, converte o texto diretamente e nunca toca no *blackboard*.

Armadilha — o literal é reconvertido a *cada tick*

No caminho literal, `convertFromString<T>()` é chamado toda vez que `getInput()` é chamado — não há memorização. Para um `int` é irrelevante; para um tipo cuja conversão faça *parsing* pesado, ou aloque, o custo aparece no perfil como tempo gasto no `tick()` da folha. Se o valor é constante e caro, leia-o uma vez e guarde num membro.

IV.4 Conversão de texto: `convertFromString`

Toda a ponte entre o XML (texto) e o C++ (tipos) passa por uma única função *template*:

```

1 template <typename T> inline
2 T convertFromString(StringView /*str*/)
3 {
4     auto type_name = BT::demangle( typeid(T) );
5     std::cerr << "You (maybe indirectly) called BT::convertFromString() for type ["
6               << type_name << "], but I can't find the template specialization.\n";
7     throw LogicError(std::string("You didn't implement the template specialization "
8                                "of convertFromString for this type: ") + type_name );
9 }
  
```

Listagem 17: O caso geral e a mensagem que ele produz, em `include/behaviortree_cpp_v3/basic_types.h`.

A biblioteca especializa a função para `std::string`, `const char*`, `int`, `unsigned`, `long`, `unsigned long`, `float`, `double`, `bool`, `std::vector<int>`, `std::vector<double>` e para os próprios `NodeStatus`, `NodeType` e `PortDirection`. Para qualquer outro tipo, é você quem escreve — e a especialização precisa estar **dentro** do *namespace* `BT`:

```

1 struct Position2D { double x, y; };
2
3 namespace BT
4 {
5     template <> inline Position2D convertFromString(StringView str)
6     {
7         auto parts = splitString(str, ','); // utilitario da propria lib
8         if (parts.size() != 2) {
9             throw RuntimeError("invalid input");
9         }
9     }
  
```

```

10     }
11     Position2D out;
12     out.x = convertFromString<double>(parts[0]);
13     out.y = convertFromString<double>(parts[1]);
14     return out;
15 }
16 } // namespace BT

```

Listagem 18: A conversão de um tipo próprio, no padrão usado por todos os exemplos do repositório.

A convenção de separar campos por ponto e vírgula não é imposta pela biblioteca: ela vem das especializações de `std::vector<int>` e `std::vector<double>`, e todos os exemplos a seguem por consistência.

Armadilha — `StringView` não é terminada em NUL, mas o código age como se fosse

As conversões numéricas são implementadas assim: `return std::stoi(str.data());`. Um `string_view` não garante terminação em NUL, e `str.data()` devolve um ponteiro para o meio do *buffer* original. Na prática funciona nos dois casos de uso reais — o atributo do *tinyxml2* e a `std::string` do *blackboard* são terminados — mas quebra silenciosamente quando a *view* é uma fatia, que é exatamente o que `splitString()` produz: `convertFromString<double>(parts[0])` lê *além* do ponto e vírgula. O `std::stod` para no primeiro caractere inválido, e por isso o resultado sai certo; a leitura fora dos limites, porém, é real.

Armadilha — `bool` aceita seis grafias e recusa todo o resto

`convertFromString<bool>` testa, nesta ordem, o tamanho da *string*: 1 caractere aceita "0" e "1"; 4 caracteres aceitam "true", "TRUE" e "True"; 5 caracteres aceitam "false", "FALSE" e "False". Qualquer outra coisa — "yes", "1 " com espaço, "tRue" — lança `RuntimeError("convertFromString(): invalid bool conversion")`.

Por que — por que `double` mexe no *locale*

As conversões de `float` e `double` salvam o `LC_NUMERIC` corrente, forçam "C", chamam `std::stod` e restauram. Sem isso, uma aplicação rodando em *locale* `pt_BR` interpretaria "1.5" como 1 — o separador decimal seria a vírgula. É a correção da *issue* #120 do projeto. O efeito colateral é que a conversão faz duas chamadas a `setlocale` por leitura, e `setlocale` é global ao processo: converter um `double` em duas *threads* ao mesmo tempo é uma corrida.

IV.5 O *blackboard*

O *blackboard* é a memória compartilhada da árvore: um dicionário de chaves textuais para valores de tipo apagado, com um *mutex* próprio e um ponteiro fraco para o *blackboard* do pai.

```

1 struct Entry {
2     Any value;
3     const PortInfo port_info;
4 };
5
6 mutable std::mutex mutex_;
7 std::unordered_map<std::string, Entry> storage_;
8 std::weak_ptr<Blackboard> parent_bb_;
9 std::unordered_map<std::string, std::string> internal_to_external_;

```

Listagem 19: O estado interno, em `include/behaviortree_cpp_v3/blackboard.h`.

Cada entrada guarda *dois* campos: o valor e a `PortInfo` da porta que a criou. É o segundo que sustenta a verificação de tipo descrita adiante.

Regra — o *blackboard* nasce por `create()`, nunca por `new`

O construtor é `protected` de propósito: um *blackboard* é sempre um `shared_ptr`, obtido por `Blackboard::create(pai)`. A ligação com o pai é um `weak_ptr`, então a pilha inteira só permanece válida enquanto o `Tree` — que guarda todos eles em `blackboard_stack` — estiver vivo.

IV.5.1 Encadeamento entre pai e filho

O encadeamento não é uma busca hierárquica: uma chave só sobe para o pai se houver um remapeamento *explícito* para ela.

```
1  const Any* getAny(const std::string& key) const
2  {
3      std::unique_lock<std::mutex> lock(mutex_);
4      if( auto parent = parent_bb_.lock())
5      {
6          auto remapping_it = internal_to_external_.find(key);
7          if( remapping_it != internal_to_external_.end())
8              return parent->getAny( remapping_it->second ); // delega ao pai
9      }
10     auto it = storage_.find(key);
11     return ( it == storage_.end()) ? nullptr : &(it->second.value);
12 }
```

Listagem 20: A consulta com encadeamento, em `blackboard.h`.

Por que — por que o isolamento é o padrão

Numa árvore grande, chaves como `"goal"` ou `"result"` aparecem em dezenas de subárvores escritas por pessoas diferentes. Se todas compartilhassem um espaço de nomes único, cada reutilização de subárvore seria uma colisão em potencial. A biblioteca escolhe o oposto: por padrão a subárvore não enxerga *nada* do pai, e o que precisa atravessar a fronteira é declarado, chave por chave, no XML (Seção VIII.4). O preço é o esquecimento silencioso — uma chave não remapeada não dá erro, apenas não é encontrada.

IV.5.2 O travamento de tipo

Uma entrada criada por uma porta tipada memoriza esse tipo, e uma escrita posterior com tipo diferente é recusada:

```
1  const auto locked_type = port_info.type();
2  Any temp(value);
3  if( locked_type && *locked_type != typeid(T) && *locked_type != temp.type() )
4  {
5      bool mismatching = true;
6      if( std::is_constructible<StringView, T>::value ) // veio como texto?
7      {
8          Any any_from_string = port_info.parseString( value ); // tenta converter
9          if( any_from_string.empty() == false ) { mismatching = false;
10              temp = std::move(any_from_string); }
11      }
12      if( mismatching )
13      {
14          debugMessage();
15          throw LogicError( "Blackboard::set() failed: once declared, the type of a "
16                          "port shall not change. Declared type [" , demangle(locked_type),
```

```

17         "] != current type [" , demangle(typeid(T)),"]" );
18     }
19 }

```

Listagem 21: A verificação de tipo em `Blackboard::set()` (condensada).

A ressalva do meio é o que faz o `SetBlackboard` funcionar: escrever a *string* `"-1;3"` numa entrada declarada como `Position2D` não é erro — a `PortInfo` sabe converter, porque guardou o conversor no momento em que a porta foi declarada com `InputPort<Position2D>`.

Armadilha — a mesma verificação, feita de dois jeitos incompatíveis

`Blackboard::set()` compara *os objetos* (`*locked_type != typeid(T)`), que é a forma correta. `Blackboard::setPortInfo()`, no mesmo arquivo, compara *os ponteiros* (`old_type != info.type()`). Comparar ponteiros de `std::type_info` funciona dentro de um binário, mas não há garantia de unicidade quando o tipo atravessa a fronteira de uma biblioteca compartilhada — exatamente o caso de um plugin carregado com `dlopen`. O sintoma é um falso `"the type of a port shall not change"` entre tipos idênticos, vindo de lados diferentes da ligação dinâmica (Seção VIII.5).

IV.5.3 Inspeção

Dois métodos servem à depuração. `debugMessage()` imprime cada chave, o tipo e se o valor está preenchido ou remapeado — é o que produz a saída comentada no exemplo `t06` do repositório:

```

1 move_result (std::string) -> full
2 move_goal (Pose2D) -> full
3 -----
4 output (std::string) -> remapped to parent [move_result]
5 target (Pose2D) -> remapped to parent [move_goal]

```

Listagem 22: Saída de `debugMessage()` nos dois *blackboards* do exemplo `t06_subtree_port_remapping`.

Armadilha — `getKeys()` e `debugMessage()` têm defeitos reais

`getKeys()` percorre `storage_` sem tomar o *mutex*, ao contrário de todos os outros métodos da classe — e ainda devolve `StringView` apontando para as chaves do mapa, que morrem se a entrada for removida. `debugMessage()`, por sua vez, faz `port_type = &(entry.value.type())` quando a porta não tem tipo declarado; `Any::type()` desreferencia um ponteiro que é `nullptr` num `Any` vazio — e entradas vazias são exatamente o que `setPortInfo()` cria. São ferramentas de depuração, não de produção.

IV.6 Valores padrão

Uma porta pode declarar um valor padrão, passado como segundo argumento: `InputPort<unsigned>("failure_threshold", 1, "descrição")`. O valor é guardado *como texto* (via `BT::toStr`) e aplicado pelo *parser* na construção do nó:

```

1 // use default value if available for empty ports. Only inputs
2 for (const auto& port_it: manifest.ports)
3 {
4     const std::string& port_name = port_it.first;
5     const PortInfo& port_info = port_it.second;
6     auto direction = port_info.direction();

```

```

7 |     if( direction != PortDirection::OUTPUT &&
8 |        config.input_ports.count(port_name) == 0 &&
9 |        port_info.defaultValue().empty() == false)
10 |    {
11 |        config.input_ports.insert( { port_name, port_info.defaultValue() } );
12 |    }
13 | }

```

Listagem 23: A aplicação do padrão, em `src/xml_parsing.cpp`.

Armadilha — o padrão só existe pelo caminho do XML

A aplicação do valor padrão está no *parser*, não no `TreeNode`. Um nó instanciado diretamente em C++, sem passar pelo XML, recebe um `NodeConfiguration` vazio e o `getInput()` da porta com padrão *falha*. Some-se a isso que uma porta de *saída* nunca recebe padrão (a condição `direction != OUTPUT` é explícita) e que um padrão vazio é indistinguível de padrão ausente — não há como declarar "" como valor padrão.

IV.7 No cenário da porta

O exemplo `t05` do repositório é deliberadamente austero neste ponto: o estado da porta vive em duas variáveis `static` do arquivo `crossdoor_nodes.cpp`, e não no *blackboard*. O comentário do fonte admite: “*For simplicity, in this example the status of the door is not shared using ports and blackboards*”. É uma escolha didática que vale conhecer para não copiar: assim que houver dois robôs, ou duas instâncias da árvore, o estado global vira um defeito. A Seção XII mostra a versão com portas.

V Nós de controle

Os nós de controle são o lugar onde a lógica mora. Uma folha diz *o que* fazer; um nó de controle diz *em que ordem, sob que condição e até quando*. São nove famílias — treze IDs de registro, contando as cinco variações do `Switch` — e cabem numa página. Mas as diferenças entre eles são finas, e é delas que dependem os comportamentos observados.

V.1 Visão de conjunto

ID no XML	Sucesso quando	Falha quando	Memória
<code>Sequence</code>	todos têm sucesso	um falha	sim, zerada ao falhar
<code>SequenceStar</code>	todos têm sucesso	um falha	sim, mantida ao falhar
<code>ReactiveSequence</code>	todos têm sucesso	um falha	não
<code>Fallback</code>	um tem sucesso	todos falham	sim, zerada ao ter sucesso
<code>ReactiveFallback</code>	um tem sucesso	todos falham	não
<code>Parallel</code>	M têm sucesso	N falham	parcial (<i>skip list</i>)
<code>IfThenElse</code>	o ramo escolhido	o ramo escolhido	sim
<code>WhileDoElse</code>	o ramo escolhido	o ramo escolhido	não
<code>Switch2...6</code>	o caso escolhido	o caso escolhido	sim (qual ramo roda)
<code>ManualSelector</code>	escolha do operador	escolha do operador	sim

Tabela 4: Os nós de controle registrados pela fábrica. Todos devolvem `RUNNING` enquanto o filho ativo não conclui.

Antes de descer aos detalhes, três observações que valem para todos:

1. **Nenhum deles usa *threads*.** Inclusive o `ParallelNode`: “paralelo” aqui quer dizer que vários filhos estão *logicamente* em curso, não que executem simultaneamente. A travessia é sequencial dentro de um mesmo *tick*.
2. **Quase todos começam com `setStatus(NodeStatus::RUNNING)`.** Isso publica a transição para os *loggers* antes de qualquer filho ser *ticado* — é o que faz o traço de execução mostrar o pai entrando antes dos filhos. As duas exceções são os reativos, que não o fazem.
3. **Um filho que devolve `IDLE` é erro fatal.** Todos lançam, na mesma linha do `switch`: “A child node must never return `IDLE`” (Seção II.3).

V.2 Sequence: o E lógico

`Sequence` executa os filhos da esquerda para a direita e para no primeiro que falha. É o E lógico da árvore.

```
1 NodeStatus SequenceNode::tick()
2 {
3     const size_t children_count = children_nodes_.size();
4     setStatus(NodeStatus::RUNNING);
5
6     while (current_child_idx_ < children_count)
7     {
8         TreeNode* current_child_node = children_nodes_[current_child_idx_];
9         const NodeStatus child_status = current_child_node->executeTick();
10
11         switch (child_status)
```

```

12     {
13         case NodeStatus::RUNNING: return child_status;
14         case NodeStatus::FAILURE:
15         {
16             // Reset on failure
17             haltChildren();
18             current_child_idx_ = 0;
19             return child_status;
20         }
21         case NodeStatus::SUCCESS: current_child_idx_++; break;
22         case NodeStatus::IDLE: throw LogicError("A child node must never return IDLE");
23     }
24 }
25 // The entire while loop completed. This means that all the children returned SUCCESS.
26 if (current_child_idx_ == children_count)
27 {
28     haltChildren();
29     current_child_idx_ = 0;
30 }
31 return NodeStatus::SUCCESS;
32 }

```

Listagem 24: `SequenceNode::tick()`, em `src/controls/sequence_node.cpp`.

O laço é a memória: enquanto um filho devolve `SUCCESS`, o índice avança e o *tick* continua *no mesmo tick*. Só o `RUNNING` devolve o controle à aplicação. Ao concluir — por sucesso total ou por falha — o nó chama `haltChildren()` e zera o índice, o que devolve todos os filhos a `IDLE` e os torna executáveis de novo no próximo ciclo.

Armadilha — o `reset_on_failure` do cabeçalho não existe

O comentário de `sequence_node.h` diz: “If a child returns `FAILURE`, stop the loop and return `FAILURE`. Restart the loop only if (`reset_on_failure == true`)”. Não há membro, porta ou opção com esse nome em lugar nenhum da biblioteca. O `docs/MigrationGuide.md` explica a origem: na v2 existia um `SequenceStar` com esse parâmetro, e a variante `reset_on_failure=true` foi promovida a `Sequence` na v3 — o comentário sobreviveu à promoção. O comportamento real é incondicional: ao falhar, a `Sequence` *sempre* recomeça do primeiro filho. Quem quer o contrário usa `SequenceStar`.

V.3 SequenceStar: o E que não esquece

A diferença cabe em duas linhas de código, e muda tudo. No caso `FAILURE`:

```

1 case NodeStatus::FAILURE:
2 {
3     // DO NOT reset current_child_idx_ on failure
4     for(size_t i=current_child_idx_; i < childrenCount(); i++)
5     {
6         haltChild(i);
7     }
8     return child_status;
9 }

```

Listagem 25: O tratamento da falha em `src/controls/sequence_star_node.cpp`.

Duas mudanças em relação à `Sequence`: o índice não é zerado, e apenas os filhos *a partir* do corrente são devolvidos a `IDLE` — os já concluídos permanecem em `SUCCESS`. No *tick* seguinte, o nó retoma exatamente do filho que falhou.

Por que — para que serve uma sequência que não recomeça

Para procedimentos irreversíveis. “Abra a garra, aproxime, feche a garra”: se a aproximação falhar, recomeçar do zero reabriria a garra — desfazendo trabalho que já estava feito. A `SequenceStar` é a forma de dizer “o que já deu certo, ficou certo”. É o padrão em máquinas de montagem e em qualquer sequência com efeitos colaterais no mundo.

Armadilha — a memória da `SequenceStar` sobrevive à falha, mas não ao *halt*

`SequenceStar::halt()` zera o índice. Como `ControlNode::haltChild()` só chama `halt()` num filho que esteja `RUNNING`, e a `SequenceStar` está em `FAILURE` quando o pai a recolhe, na prática a memória sobrevive — que é o comportamento desejado. Mas basta que ela seja *halted* enquanto `RUNNING` — por um `Timeout` que expira, por um irmão à esquerda de uma `ReactiveSequence` que passa a falhar, ou por `Tree::haltTree()` — para que o progresso todo se perca sem aviso. Se a retomada precisa ser garantida, guarde-a numa entrada do *blackboard*, não no nó.

V.4 ReactiveSequence: o E que reavalia tudo

```
1  for (size_t index = 0; index < childrenCount(); index++)
2  {
3      const NodeStatus child_status = children_nodes_[index]->executeTick();
4      switch (child_status)
5      {
6          case NodeStatus::RUNNING:
7          {
8              running_count++;
9              for(size_t i=index+1; i < childrenCount(); i++) { haltChild(i); }
10             return NodeStatus::RUNNING;
11         }
12         case NodeStatus::FAILURE: { haltChildren(); return NodeStatus::FAILURE; }
13         case NodeStatus::SUCCESS: { success_count++; } break;
14         case NodeStatus::IDLE: throw LogicError("A child node must never return IDLE");
15     }
16 }
17 if( success_count == childrenCount()) { haltChildren(); return NodeStatus::SUCCESS; }
18 return NodeStatus::RUNNING;
```

Listagem 26: `ReactiveSequence::tick()`, em `src/controls/reactive_sequence.cpp` (condensada).

Não há índice: o laço começa em zero a cada *tick*. Um filho que já tinha dado `SUCCESS` é reexecutado, e se agora falhar, todo o restante é *halted*. É o mecanismo de guarda por excelência: condições à esquerda, trabalho à direita.

Regra — o padrão guarda-e-trabalho

A forma canônica de usar uma `ReactiveSequence` é pôr N condições baratas nas primeiras posições e uma única ação assíncrona na última. Toda condição vira, assim, uma *precondição contínua*: se qualquer uma deixar de valer, a ação é interrompida no mesmo *tick*.

Armadilha — o *halt* dos irmãos à direita é o que quebra dois assíncronos

O laço `for(i=index+1; ...) haltChild(i)` explica por que a `ReactiveSequence` não suporta dois filhos assíncronos: assim que o primeiro deles devolve `RUNNING`, o segundo é *halted* — perdendo todo o progresso — e recomeçado do zero assim que o primeiro terminar. Com duas ações longas em sequência reativa, a segunda pode nunca progredir. O cabeçalho avisa (Seção II.4); o código é a razão.

Note ainda o contador `running_count`: ele é incrementado e nunca lido. É código morto, sobrevivente de uma versão anterior em que a `ReactiveSequence` era mais parecida com o `ParallelNode`.

V.5 Fallback e ReactiveFallback: o OU lógico

Fallback é o espelho exato da **Sequence**: percorre os filhos e para no primeiro que *tem sucesso*; se todos falharem, falha. As duas implementações são idênticas com **SUCCESS** e **FAILURE** trocados, inclusive nas chamadas a `haltChildren()` e no zeramento do índice.

O mesmo vale para **ReactiveFallback** em relação à **ReactiveSequence** — inclusive o *halt* dos irmãos à direita e a mesma restrição de um único filho assíncrono. A leitura de uma **ReactiveFallback** é “tente a primeira estratégia; se ela falhar, a segunda; e reavalie a preferência a cada *tick*”.

Armadilha — os nomes da v2 significam o oposto na v3

Esta é a incompatibilidade mais perigosa que um XML antigo pode carregar, porque os nomes continuam válidos e mudaram de sentido. O `docs/MigrationGuide.md` traz a tabela de renomeação:

Nome na v2	Nome na v3	É reativo?
Sequence	ReactiveSequence	sim
SequenceStar (reset_on_failure=true)	Sequence	não
SequenceStar (reset_on_failure=false)	SequenceStar	não
Fallback	ReactiveFallback	sim
FallbackStar	Fallback	não
Parallel	Parallel	sim (v2) / não (v3)

Tabela 5: A renomeação entre a v2 e a v3, conforme `docs/MigrationGuide.md`. Um `<Sequence>` de um arquivo da v2 era reativo; na v3 é o contrário.

Por que — por que o padrão deixou de ser reativo

O texto do guia é explícito quanto à motivação: *“I don’t think reactive ControlNodes should be used mindlessly by default”*. A v2 fazia da reatividade o padrão, e o resultado prático era que a maioria dos usuários usava um nó reativo onde queria um com memória — sem perceber, até que uma condição à esquerda mudasse no meio de uma operação. A v3 inverteu a escolha e deu nome explícito à reatividade. Um **FallbackStar** num XML antigo, portanto, não dá erro de sintaxe: dá **"Node not recognized"** na carga, porque o ID desapareceu — o que, neste caso, é a falha benigna.

V.6 Parallel: M de N

ParallelNode *tica* todos os filhos a cada passagem e conta quantos já concluíram de cada jeito. Duas portas governam o resultado:

```

1 static PortsList providedPorts()
2 {
3     return { InputPort<unsigned>(THRESHOLD_SUCCESS,
4         "number of children which need to succeed to trigger a SUCCESS" ),
5         InputPort<unsigned>(THRESHOLD_FAILURE, 1,
6         "number of children which need to fail to trigger a FAILURE" ) };
7 }
```

Listagem 27: As portas de **ParallelNode**, em `include/behaviortree_cpp_v3/controls/parallel_node.h`.

Armadilha — success_threshold é obrigatório; o outro, não

Repare que só `failure_threshold` tem valor padrão (1). Um `<Parallel>` escrito sem `success_threshold` compila, carrega e falha no primeiro *tick* com `RuntimeError("Missing parameter [success_threshold] in ParallelNode")` — em tempo de execução, não de carga. Escreva sempre os dois.

O nó mantém uma *skip list* com os índices dos filhos que já concluíram, para não os *ticar* de novo dentro do mesmo ciclo:

```
1 for (unsigned int i = 0; i < children_count; i++)
2 {
3     bool in_skip_list = (skip_list_.count(i) != 0);
4     NodeStatus child_status = in_skip_list ? children_nodes_[i]-->status()
5                                     : children_nodes_[i]-->executeTick();
6     switch (child_status)
7     {
8         case NodeStatus::SUCCESS:
9             if( !in_skip_list ) { skip_list_.insert(i); }
10            success_childred_num++;
11            if (success_childred_num == success_threshold_) {
12                skip_list_.clear(); haltChildren(); return NodeStatus::SUCCESS;
13            }
14            break;
15        case NodeStatus::FAILURE:
16            if( !in_skip_list ) { skip_list_.insert(i); }
17            failure_childred_num++;
18            // It fails if it is not possible to succeed anymore or if
19            // number of failures are equal to failure_threshold_
20            if ((failure_childred_num > children_count - success_threshold_)
21                || (failure_childred_num == failure_threshold_)) {
22                skip_list_.clear(); haltChildren(); return NodeStatus::FAILURE;
23            }
24            break;
25        case NodeStatus::RUNNING: break; // do nothing
26        default: throw LogicError("A child node must never return IDLE");
27    }
28 }
29 return NodeStatus::RUNNING;
```

Listagem 28: O laço principal, em `src/controls/parallel_node.cpp` (condensado).

A condição de falha tem duas metades: a explícita (`failure_count == failure_threshold`) e a implícita — se já falharam tantos que os restantes não bastam para atingir o limiar de sucesso, não há por que continuar. Antes do laço, duas verificações de sanidade lançam `LogicError` se algum limiar for maior que o número de filhos (“*Can never succeed*” / “*Can never fail*”).

Armadilha — um Parallel não paraleliza nada

O nome vem da literatura de árvores de comportamento, não de concorrência. Os filhos são *ticados* um após o outro, na mesma *thread*, dentro do mesmo *tick*. Se dois deles forem `SyncActionNode` de 100 ms cada, o *tick* leva 200 ms. Concorrência real exige que os filhos sejam assíncronos (Seção VII); o `Parallel` apenas permite que várias ações assíncronas estejam *pendentes* ao mesmo tempo.

Armadilha — os limiares são lidos a cada tick, e podem mudar

Quando construído pelo XML, `read_parameter_from_ports_` é verdadeiro e os dois limiares são relidos no início de *cada tick()*. Se estiverem remapeados para o *blackboard*, um nó anterior pode alterá-los no meio de um ciclo — com a *skip list* já parcialmente preenchida e as contagens recomeçando do zero. O resultado é bem definido, mas raramente é o que se esperava.

V.7 IfThenElse e WhileDoElse

Os dois exprimem a mesma ideia — condição, ramo verdadeiro, ramo falso — com reatividades opostas.

IfThenElseNode aceita 2 ou 3 filhos e **não é reativo**: uma vez que a condição decidiu, o `child_idx_` fixa o ramo, e a condição só volta a ser avaliada quando o ramo terminar. Com apenas 2 filhos, a falha da condição é a falha do nó — equivalente a ter **AlwaysFailure** como terceiro filho.

WhileDoElseNode é **reativo**: a condição é reavaliada a cada *tick*, e se ela mudar de resultado, o ramo em curso é interrompido antes de o outro começar.

```
1  if (condition_status == NodeStatus::SUCCESS)    { haltChild(2);  
2                                                    status = children_nodes_[1]->executeTick(); }  
3  else if (condition_status == NodeStatus::FAILURE) { haltChild(1);  
4                                                    status = children_nodes_[2]->executeTick(); }
```

Listagem 29: A troca de ramo em `src/controls/while_do_else_node.cpp`.

Armadilha — WhileDoElse exige exatamente 3 filhos, apesar do que diz a mensagem

O cabeçalho anuncia “*must have exactly 2 or 3 children*” e a própria mensagem de erro repete “*WhileDoElse must have either 2 or 3 children*” — mas a condição que a dispara é `if(children_count != 3)`. Com dois filhos, o nó lança. Some-se a isso que a exceção é um `std::logic_error` cru, fora da hierarquia **BehaviorTreeException** (Seção III.7), e que ela só ocorre no primeiro *tick* — a validação do XML não confere aridade de nós de controle nomeados.

V.8 Switch<N>: o desvio por valor

SwitchNode é um *template* sobre o número de casos, registrado em cinco instâncias — **Switch2** a **Switch6**. Ele compara a porta **variable** com as portas `case_1...case_N` e executa o filho correspondente; o último filho, sempre presente, é o *default*.

```
1  <Switch3 variable="{var}" case_1="1" case_2="42" case_3="666" >  
2    <ActionA name="action_when_var_eq_1" />  
3    <ActionB name="action_when_var_eq_42" />  
4    <ActionC name="action_when_var_eq_666" />  
5    <ActionD name="default_action" />  
6  </Switch3>
```

Listagem 30: O exemplo do próprio cabeçalho `controls/switch_node.h`: um **Switch3** tem **quatro** filhos.

O nó guarda qual ramo estava **RUNNING** e, se a variável mudar entre *ticks*, *halta* o ramo antigo antes de iniciar o novo — é reativo quanto à seleção, ainda que não quanto ao resultado.

Armadilha — a comparação é textual, e isso importa

Tanto **variable** quanto os `case_i` são lidos como `std::string`, e a comparação é `variable == value`. Se a entrada do *blackboard* guardar um `double`, a conversão para texto passa por `std::to_string`, que produz “42.000000” — e `case_1="42"` **não** casa. Guarde `int` ou `std::string` na variável do *switch*, nunca ponto flutuante. Se **variable** não puder ser lida, o nó cai silenciosamente no *default*; não há erro.

Armadilha — o número de filhos é conferido só no primeiro *tick*

`if( childrenCount() != NUM_CASES+1 )` lança `LogicError("Wrong number of children in SwitchNode; must be (num_cases + default)")` — dentro do `tick()`. Um `Switch3` com três filhos carrega sem reclamar e explode na primeira execução. E, se o *default* for selecionado por índice `NUM_CASES` numa árvore com filhos a menos, o acesso a `children_nodes_[child_index]` seria fora dos limites — é justamente essa verificação que o impede.

V.9 ManualSelector: o operador no laço

`ManualSelectorNode` abre uma interface de texto em *ncurses* e pergunta ao operador qual filho executar — ou, opcionalmente, qual `NodeStatus` devolver diretamente. Serve para depurar uma árvore cujos ramos ainda não existem, e para exercitar caminhos raros.

Armadilha — não existe nesta configuração

O registro é condicionado a `#ifdef NCURSES_FOUND`, e `src/controls/manual_node.cpp` só entra na compilação se `BUILD_WITH_CURSES` estiver ligado e o CMake achar a biblioteca. A receita Conan deste *fork* desliga `BUILD_WITH_CURSES` (Seção XI.3), então um XML que use `<ManualSelector>` falha na carga com "Node not recognized" — e não com uma mensagem sobre *ncurses*.

V.10 Escolhendo o nó de controle

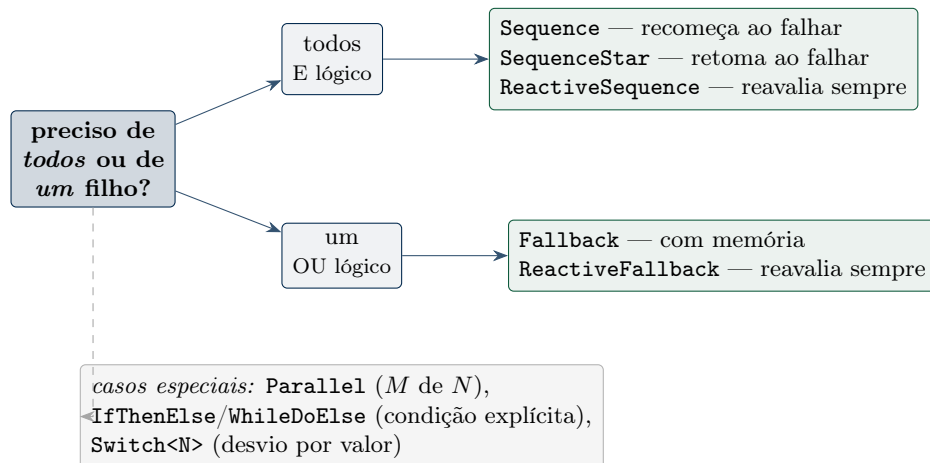


Figura 9: Um caminho de decisão para escolher o nó de controle. A segunda pergunta — “preciso reavaliar o que já deu certo?” — é a que separa as variantes reativas.

V.11 No cenário da porta

A raiz do exemplo é uma `Sequence` de dois filhos: o `Fallback` das estratégias e o `CloseDoor` final. O `Fallback` tenta, em ordem, passar pela porta já aberta, executar a subárvore `DoorClosed` e passar pela janela — exatamente o OU lógico da Tabela 4. E como as duas são com memória, uma vez que a travessia dê certo, o `CloseDoor` executa sem reavaliar `IsDoorOpen`. Trocar a raiz por uma `ReactiveSequence` mudaria o comportamento de forma sutil e indesejada: a condição da porta seria reavaliada durante o fechamento.

VI Decoradores

Um decorador tem exatamente um filho e existe para modificar alguma coisa a respeito dele: o resultado, o número de execuções, a duração ou a permissão para executar. São onze registrados pela fábrica com `NodeType::DECORATOR` — treze contando `SubTree` e `SubTreePlus`, que herdam de `DecoratorNode` mas se declaram `SUBTREE` — e agrupam-se naturalmente em quatro famílias.

Família	ID no XML	O que faz
resultado	<code>Inverter</code>	troca <code>SUCCESS</code> por <code>FAILURE</code> e vice-versa
	<code>ForceSuccess</code>	converte qualquer conclusão em <code>SUCCESS</code>
	<code>ForceFailure</code>	converte qualquer conclusão em <code>FAILURE</code>
repetição	<code>Repeat</code>	repete enquanto o filho tiver sucesso
	<code>RetryUntilSuccessful</code>	repete enquanto o filho falhar
	<code>KeepRunningUntilFailure</code>	nunca conclui, até o filho falhar
tempo	<code>Timeout</code>	<i>halta</i> o filho após <i>N</i> ms
	<code>Delay</code>	espera <i>N</i> ms antes de <i>ticar</i> o filho
guarda	<code>BlackboardCheckInt</code>	executa o filho só se dois valores forem iguais
	<code>BlackboardCheckDouble</code>	idem, para <code>double</code>
	<code>BlackboardCheckString</code>	idem, para <code>std::string</code>

Tabela 6: Os decoradores registrados pela fábrica, por família. O `SubTree` e o `SubTreePlus` são tecnicamente decoradores, mas pertencem ao Seção VIII.4.

Vale lembrar da Seção III.4 que `DecoratorNode` sobrescreve `executeTick()` para repor o filho a `IDLE` assim que ele conclui — é isso que permite aos decoradores de repetição executarem o mesmo filho várias vezes sem nenhum cuidado adicional.

VI.1 Os que transformam o resultado

`InverterNode` é o mais simples e o mais usado: `SUCCESS` vira `FAILURE`, `FAILURE` vira `SUCCESS`, `RUNNING` passa direto. É o NÃO lógico, e a forma idiomática de escrever “se a condição *não* valer”.

`ForceSuccessNode` e `ForceFailureNode` colapsam as duas conclusões numa só, deixando o `RUNNING` passar. O uso típico do primeiro é tornar opcional um passo dentro de uma `Sequence`: “tente isto; se não der, siga em frente”.

```

1  inline NodeStatus ForceFailureNode::tick()
2  {
3      setStatus(NodeStatus::RUNNING);
4      const NodeStatus child_state = child_node_—>executeTick();
5      switch (child_state)
6      {
7          case NodeStatus::FAILURE:
8              case NodeStatus::SUCCESS: { return NodeStatus::FAILURE; }
9              case NodeStatus::RUNNING: { return NodeStatus::RUNNING; }
10             default: { /* TODO throw? */ }
11         }
12     return status();
13 }
```

Listagem 31: `ForceFailureNode::tick()`, em `include/behaviortree_cpp_v3/decorators/force_failure_node.h`.

Armadilha — o `// TODO throw?` não é decorativo

Três decoradores — `ForceSuccessNode`, `ForceFailureNode` e `KeepRunningUntilFailureNode` — tratam o caso `IDLE` com um comentário e nada mais, devolvendo em seguida `status()`: o próprio estado do decorador, que acabou de ser posto em `RUNNING`. Enquanto todos os outros nós de controle lançam `LogicError` diante de um filho que devolve `IDLE`, estes três **convertem o erro em `RUNNING`**. Um nó de usuário defeituoso pendurado sob um `ForceSuccess` produz uma árvore que fica `RUNNING` para sempre, sem erro e sem progresso — o sintoma mais difícil de diagnosticar que esta biblioteca oferece.

VI.2 Os que repetem

`RepeatNode` e `RetryNode` são o mesmo código com `SUCCESS` e `FAILURE` trocados. O primeiro repete *enquanto o filho tiver sucesso*, até completar `num_cycles`; o segundo repete *enquanto o filho falhar*, até esgotar `num_attempts`.

```
1 NodeStatus RetryNode::tick()
2 {
3     if( read_parameter_from_ports_ )
4     {
5         if( !getInput(NUM_ATTEMPTS, max_attempts_) )
6             throw RuntimeError("Missing parameter ['", NUM_ATTEMPTS, "] in RetryNode");
7     }
8     setStatus(NodeStatus::RUNNING);
9
10    while( try_index_ < max_attempts_ || max_attempts_ == -1)
11    {
12        NodeStatus child_state = child_node_ ->executeTick();
13        switch (child_state)
14        {
15            case NodeStatus::SUCCESS: { try_index_ = 0; haltChild();
16                                     return (NodeStatus::SUCCESS); }
17            case NodeStatus::FAILURE: { try_index_ ++; haltChild(); } break;
18            case NodeStatus::RUNNING: { return NodeStatus::RUNNING; }
19            default: throw LogicError("A child node must never return IDLE");
20        }
21    }
22    try_index_ = 0;
23    return NodeStatus::FAILURE;
24 }
```

Listagem 32: `RetryNode::tick()`, em `src/decorators/retry_node.cpp`.

O `haltChild()` explícito após cada tentativa é a correção da *issue* #228, registrada no `CHANGELOG.rst` como “*Retry and Repeat node need to halt the child*”: sem ele, um filho com estado interno — um `StatefulActionNode`, por exemplo — não seria reiniciado entre as tentativas.

Armadilha — o ID de registro tem erro de grafia, e é ele que vale

O construtor de `RetryNode` chama `setRegistrationID("RetryUntilSuccessful")` — com dois `s`. A fábrica, porém, registra o nó como `registerNodeType<RetryNode>("RetryUntilSuccesful")` — com *um* `s`. Como `instantiateTreeNode()` sobrescreve o ID com aquele usado no registro, **quem vale é a grafia errada**: o XML precisa dizer `<RetryUntilSuccesful>`. O exemplo `t05_crossdoor.cpp` e o comentário do próprio `retry_node.h` usam a forma errada — por necessidade, não por descuido. Escrever o nome corretamente produz “Node not recognized: `RetryUntilSuccessful`”.

Armadilha — num_cycles="-1" com filho síncrono trava o processo

Tanto `Repeat` quanto `RetryUntilSuccessful` aceitam `-1` como “infinito”. Mas o laço de repetição está *dentro* de um único `tick()`: ele só devolve o controle quando o filho responde `RUNNING`. Com um filho síncrono — que nunca devolve `RUNNING` — e limite `-1`, o `tick()` entra num laço infinito e o processo trava, sem consumir memória e sem produzir erro. Use `-1` apenas com filhos assíncronos.

`KeepRunningUntilFailureNode` é o extremo dessa família: converte `SUCCESS` em `RUNNING` e só conclui quando o filho falha. Serve para transformar uma ação pontual em atividade contínua — “fique patrulhando até que algo dê errado”.

VI.3 Os que impõem tempo

`TimeoutNode` arma um temporizador na primeira execução; se ele disparar enquanto o filho ainda estiver `RUNNING`, o filho é *halted* e o decorador devolve `FAILURE`.

```
1  if ( !timeout_started_ )
2  {
3      timeout_started_ = true;
4      setStatus(NodeStatus::RUNNING);
5      child_halted_ = false;
6      if ( msec_ > 0 )
7      {
8          timer_id_ = timer_.add(std::chrono::milliseconds(msec_),
9                                [this](bool aborted) {
10                                     std::unique_lock<std::mutex> lk( timeout_mutex_ );
11                                     if ( !aborted && child()->status() == NodeStatus::RUNNING )
12                                     {
13                                         child_halted_ = true;
14                                         haltChild();
15                                     }
16                                 });
17      }
18  }
19  std::unique_lock<std::mutex> lk( timeout_mutex_ );
20  if ( child_halted_ ) { timeout_started_ = false; return NodeStatus::FAILURE; }
21  else
22  {
23      auto child_status = child()->executeTick();
24      if ( child_status != NodeStatus::RUNNING )
25      {
26          timeout_started_ = false;
27          timeout_mutex_.unlock();
28          timer_.cancel(timer_id_);
29          timeout_mutex_.lock();
30      }
31      return child_status;
32  }
```

Listagem 33: O núcleo de `TimeoutNode::tick()`, em `include/behaviortree_cpp_v3/decorators/timeout_node.h` (condensado).

`DelayNode` é o simétrico: arma o temporizador, devolve `RUNNING` enquanto ele não disparar e só então *tica* o filho, uma vez.

Armadilha — cada `Timeout` e cada `Delay` custa uma `thread`

Ambos têm um `TimerQueue` como *membro*, e o construtor de `TimerQueue` lança uma *thread* dedicada (Seção X.3). Uma árvore com dez `<Timeout>` tem dez *threads* paradas em `condition_variable::wait_until`, criadas na carga e vivas até a destruição da árvore — *mesmo*

que nenhum desses nós seja jamais ticado. Não há *pool* compartilhado.

Armadilha — o `unlock()/lock()` manual sob um `unique_lock`

Repare nas três linhas dentro do `else`: o *mutex* é destravado e retravado à mão, enquanto um `std::unique_lock` ainda pensa que o detém. Funciona porque o `unique_lock` destrava no fim do escopo e o estado, ao final, é o mesmo — mas basta `timer_.cancel()` lançar para o *mutex* ficar destravado com o `unique_lock` tentando destravar de novo, o que é comportamento indefinido. O motivo do malabarismo é real: o *handler* do temporizador toma o mesmo *mutex*, e `cancel()` espera o *handler* correr — segurar o *mutex* durante o `cancel()` seria um *deadlock*.

Armadilha — Delay não se recupera de um `halt`

`DelayNode::halt()` cancela os temporizadores e chama `DecoratorNode::halt()`, mas **não repõe** `delay_started_`. O cancelamento faz o *handler* rodar com `aborted = true`, o que liga `delay_aborted_`. No *tick* seguinte o nó vê `delay_started_` ainda verdadeiro, cai no ramo do aborto e devolve `FAILURE` — uma falha espúria, e só então volta ao estado inicial. Um `<Delay>` sob um nó reativo falha uma vez a cada interrupção.

Regra — `msec="0"` desliga o Timeout

A guarda `if (msec_ > 0)` não arma temporizador nenhum quando o prazo é zero. O nó vira um repassador transparente: *tica* o filho e devolve o resultado dele, para sempre. Não é “prazo zero”, é “sem prazo”.

VI.4 Os que guardam o acesso

`BlackboardPreconditionNode` é um *template* instanciado em três tipos — `int`, `double` e `std::string` — e registrado como `BlackboardCheckInt`, `BlackboardCheckDouble` e `BlackboardCheckString`. Ele executa o filho apenas se `value_A` e `value_B` forem iguais; caso contrário devolve o que a porta `return_on_mismatch` disser.

```
1  T value_A; T value_B;
2  NodeStatus default_return_status = NodeStatus::FAILURE;
3  setStatus(NodeStatus::RUNNING);
4
5  if( getInput("value_A", value_A) && getInput("value_B", value_B) && value_B == value_A )
6  {
7      return child_node_ -> executeTick();
8  }
9  if( child() -> status() == NodeStatus::RUNNING ) { haltChild(); }
10 getInput("return_on_mismatch", default_return_status);
11 return default_return_status;
```

Listagem 34: `BlackboardPreconditionNode<T>::tick()`, em `decorators/blackboard_precondition.h`.

Duas leituras valem a pena. Primeiro, o nó é *reativo*: a comparação é refeita a cada *tick*, e um filho em curso é interrompido assim que a igualdade deixa de valer. Segundo, `return_on_mismatch` é lido com o valor de `getInput()` *ignorado* — se a porta faltar, a variável mantém o `FAILURE` inicial. É um padrão deliberado de valor padrão em código, ao contrário do padrão declarado na porta (Seção IV.6).

Armadilha — as portas de comparação não têm tipo declarado

`providedPorts()` devolve `InputPort("value_A")` e `InputPort("value_B")` — sem parâmetro de *template*. As entradas correspondentes do *blackboard*, portanto, não são travadas em nenhum tipo (Seção IV.5.2), e a conversão acontece só na leitura, dentro do `getInput<T>()` da instanciação. Comparar um `BlackboardCheckInt` contra uma entrada que guarda um `double` não dá erro na carga: dá uma conversão silenciosa no *tick*.

Por que — por que só três tipos, e não um *template* aberto

O registro na fábrica é feito por *string*, em tempo de execução — não há como instanciar `BlackboardPreconditionNode<MeuTipo>` a partir de um nome lido do XML. Cada instanciação precisa existir como código compilado e ser registrada explicitamente, e o construtor da fábrica registra as três que cobrem os casos comuns. Para um tipo próprio, registre a sua: `factory.registerNodeType<BlackboardPreconditionNode<Pose2D>>("BlackboardCheckPose")` — desde que `Pose2D` tenha `operator==`.

VI.5 SimpleDecoratorNode: um decorador sem herança

Para casos triviais, a fábrica oferece `registerSimpleDecorator()`, que recebe uma função com a assinatura `NodeStatus(NodeStatus child_status, TreeNode& self)` e a embrulha num `SimpleDecoratorNode`:

```
1 factory.registerSimpleDecorator("SoSeArmado",
2   [&robo](BT::NodeStatus filho, BT::TreeNode&) {
3     return robo.armado() ? filho : BT::NodeStatus::FAILURE;
4   });
```

Listagem 35: Um decorador de três linhas, sem escrever classe nenhuma.

Armadilha — o filho é *ticado* antes de o *functor* decidir

`SimpleDecoratorNode::tick()` é `return tick_functor_(child()->executeTick(), *this);` — o filho executa *sempre*, e o *functor* só transforma o resultado. Não dá para escrever uma guarda com `registerSimpleDecorator()`: para *impedir* a execução do filho é preciso herdar de `DecoratorNode`. O cabeçalho avisa, em outras palavras: “*SimpleDecoratorNode does not support halting, NodeParameters, nor Blackboards*”.

VI.6 Escrevendo um decorador

O esqueleto é curto, e vale ter os três pontos de atenção à vista:

```
1 class NoMaximoACada : public BT::DecoratorNode
2 {
3     public:
4         NoMaximoACada(const std::string& name, const BT::NodeConfiguration& config)
5             : BT::DecoratorNode(name, config) {}
6
7         static BT::PortsList providedPorts()           // ① publico, senao e ignorado
8         {
9             return { BT::InputPort<unsigned>("periodo_ms", 1000, "intervalo minimo") };
10        }
11
12        BT::NodeStatus tick() override
13        {
```

```

14     unsigned periodo = 1000;
15     getInput("periodo_ms", periodo);           // ② leitura no tick, não no ctor
16
17     auto agora = std::chrono::steady_clock::now();
18     if (agora - ultimo_ < std::chrono::milliseconds(periodo)) {
19         return BT::NodeStatus::FAILURE;        // ainda cedo: não tica o filho
20     }
21     ultimo_ = agora;
22     return child_node_ -> executeTick();        // ③ o pai repoe o filho a IDLE
23 }
24
25 private:
26     std::chrono::steady_clock::time_point ultimo_{};
27 };

```

Listagem 36: Um decorador que limita a taxa de execução do filho.

Regra — um decorador que pode devolver RUNNING precisa de halt()

No exemplo acima não é preciso sobrescrever `halt()`: o `DecoratorNode::halt()` herdado já propaga ao filho e repõe o estado. Mas se o seu decorador guardar estado próprio — um contador, um temporizador, uma posição num ciclo —, sobrescreva `halt()`, zere esse estado e **chame a implementação da base**, como fazem `RepeatNode::halt()` e `RetryNode::halt()`. Esquecer a chamada à base deixa o filho RUNNING pendurado.

Armadilha — `DecoratorNode::halt()` não tolera filho ausente

`haltChild()` desreferencia `child_node_` sem verificar `nullptr`. Um decorador construído em C++ ao qual se esqueceu de chamar `setChild()` produz falha de segmentação no primeiro `halt` — e o destrutor de `Tree` chama `haltTree()`, então basta a árvore sair de escopo. Pelo caminho do XML isso não acontece: a validação exige exatamente um filho para todo decorador (Seção VIII.2.1).

VI.7 No cenário da porta

A subárvore `DoorClosed` usa dois decoradores em cascata. O `<Inverter>` sobre `IsDoorOpen` diz “só siga se a porta *não* estiver aberta” — o NÃO lógico da Seção VI.1. E o `<RetryUntilSuccessful num_attempts="4">` sobre `OpenDoor` dá quatro tentativas à fechadura emperrada. Como `OpenDoor` é uma ação síncrona, as quatro tentativas acontecem *dentro de um mesmo* tick — e é por isso que o valor 4 é seguro ali, enquanto -1 travaria o programa.

VII Nós de ação e de condição

As folhas são onde a árvore toca o mundo. Tudo o que os capítulos anteriores descreveram — controle, decoração, portas — existe para decidir *quando* uma folha executa; esta seção trata do que acontece *dentro* dela.

A decisão que estrutura o capítulo inteiro é uma só: **a ação termina dentro do *tick*, ou continua depois dele?** Da resposta saem quatro classes base distintas, com custos e obrigações bem diferentes.

Classe base	Devolve RUNNING?	Custo	Você implementa
SyncActionNode	nunca (lança se tentar)	nenhum	<code>tick()</code>
StatefulActionNode	sim	nenhum	<code>onStart()</code> , <code>onRunning()</code> , <code>onHalted()</code>
AsyncActionNode	sim	uma <i>thread</i> por execução	<code>tick()</code> , <code>halt()</code>
CoroActionNode	sim	uma pilha de co-rotina	<code>tick()</code> com <i>yield</i>
ConditionNode	não (por convenção)	nenhum	<code>tick()</code>

Tabela 7: As classes base de folha e o que cada uma pede em troca.

Regra — escolha a mais simples que resolva

A ordem de preferência é: `SyncActionNode` se a ação for curta; `StatefulActionNode` se ela for do tipo pedido-e-resposta; `AsyncActionNode` apenas se for preciso bloquear de verdade dentro do `tick()`. O cabeçalho de `AsyncActionNode` é incomumente franco a respeito: *“IMPORTANT: this action is quite hard to implement correctly. Please be sure that you know what you are doing.”*

VII.1 ConditionNode: perguntas, não comandos

Uma condição é uma folha que responde `SUCCESS` ou `FAILURE` e não muda o mundo. Tecnicamente, nada na classe impede efeitos colaterais — o que ela define é apenas que `halt()` não faz nada além de repor o estado, e que `type()` devolve `CONDITION`, ambos `final`.

```
1 class ConditionNode : public LeafNode
2 {
3     public:
4         ConditionNode(const std::string& name, const NodeConfiguration& config);
5         virtual ~ConditionNode() override = default;
6
7         //Do nothing
8         virtual void halt() override final { setStatus(NodeStatus::IDLE); }
9         virtual NodeType type() const override final { return NodeType::CONDITION; }
10 };
```

Listagem 37: `ConditionNode`, em `include/behaviortree_cpp_v3/condition_node.h`.

Regra — uma condição precisa ser barata e sem efeito colateral

A obrigação não vem do compilador, vem do uso: numa `ReactiveSequence`, toda condição à esquerda é reavaliada a *cada tick* (Seção V.4). Uma condição que faz uma chamada de rede ou que altera o *blackboard* transforma a frequência de *tick* da aplicação numa variável do comportamento — e a árvore deixa de ser reproduzível.

Para os casos triviais, `registerSimpleCondition()` dispensa a classe:

```

1 // função livre, sem portas
2 factory.registerSimpleCondition("IsDoorOpen", std::bind(IsDoorOpen));
3
4 // com portas: a lista é passada explicitamente, porque um SimpleConditionNode
5 // não pode ter o seu próprio providedPorts()
6 BT::PortsList ports = { BT::InputPort<double>("limite") };
7 factory.registerSimpleCondition("BateriaAcima", verificaBateria, ports);

```

Listagem 38: As duas formas de registrar uma condição, do exemplo `sample_nodes/crossdoor_nodes.cpp`.

VII.2 SyncActionNode: a ação que termina no tick

É a classe base a usar por padrão. Ela sobrescreve `executeTick()` apenas para transformar o `RUNNING` em erro imediato:

```

1 NodeStatus SyncActionNode::executeTick()
2 {
3     auto stat = ActionNodeBase::executeTick();
4     if( stat == NodeStatus::RUNNING) {
5         throw LogicError("SyncActionNode MUST never return RUNNING");
6     }
7     return stat;
8 }

```

Listagem 39: A proibição do `RUNNING`, em `src/action_node.cpp`.

Em troca, `halt()` é *final* e não faz nada — não há o que interromper, porque quando o `halt()` pudesse ser chamado a ação já terminou.

Armadilha — “síncrono” não significa “rápido”

Nada impede um `SyncActionNode` de dormir dois segundos — e o exemplo `crossdoor_nodes.cpp` faz exatamente isso, com `SleepMS(2000)` dentro de `OpenDoor`. O efeito é que o *tick inteiro* leva dois segundos: nenhuma condição é reavaliada, nenhum irmão progride, e um `Timeout` que englobe a árvore não consegue interrompê-la, porque o *halt* só chega quando o *tick* volta. Se a ação pode demorar mais que o período de *tick* desejado, ela não é síncrona.

O atalho equivalente é `registerSimpleAction()`, que cria um `SimpleActionNode` — uma subclasse de `SyncActionNode` que delega a um *functor* recebendo `TreeNode&`. É por esse parâmetro que o *functor* acessa as portas, como no exemplo `t07` do repositório:

```

1 auto MoveToWrapperWithLambda = [&move_to](TreeNode& parent_node) -> NodeStatus
2 {
3     Point3D goal;
4     // thanks to paren_node, you can access easily the input and output ports.
5     parent_node.getInput("goal", goal);
6     bool res = move_to.go( goal );
7     return res ? NodeStatus::SUCCESS : NodeStatus::FAILURE;
8 };
9
10 PortsList ports = { BT::InputPort<Point3D>("goal") };
11 factory.registerSimpleAction("MoveTo", MoveToWrapperWithLambda, ports );

```

Listagem 40: Embrulhando código legado sem tocar nele, de `examples/t07_wrap_legacy.cpp`.

VII.3 StatefulActionNode: pedido e resposta

Esta é a forma recomendada de escrever uma ação longa, e a que o cabeçalho chama de “*the goto option*”. Em vez de bloquear, você divide a ação em três momentos:

```
1  NodeStatus StatefulActionNode::tick()
2  {
3      const NodeStatus initial_status = status();
4
5      if( initial_status == NodeStatus::IDLE )
6      {
7          NodeStatus new_status = onStart();
8          if( new_status == NodeStatus::IDLE )
9              throw std::logic_error("AsyncActionNode2::onStart() must not return IDLE");
10         return new_status;
11     }
12     if( initial_status == NodeStatus::RUNNING )
13     {
14         NodeStatus new_status = onRunning();
15         if( new_status == NodeStatus::IDLE )
16             throw std::logic_error("AsyncActionNode2::onRunning() must not return IDLE");
17         return new_status;
18     }
19     return initial_status;
20 }
```

Listagem 41: A máquina de estados de `StatefulActionNode::tick()`, em `src/action_node.cpp`.

O contrato é simples: `onStart()` dispara o pedido e devolve `RUNNING` se ele não foi respondido de imediato; `onRunning()` é chamado a cada *tick* seguinte e verifica se a resposta chegou; `onHalted()` limpa o que precisar ser limpo se a ação for cancelada. Nenhuma *thread*, nenhuma pilha extra — a máquina de estados vive no próprio nó.

```
1  class ChamaServico : public BT::StatefulActionNode
2  {
3      public:
4          ChamaServico(const std::string& name, const BT::NodeConfiguration& config)
5              : BT::StatefulActionNode(name, config) {}
6
7          static BT::PortsList providedPorts() { return { BT::InputPort<std::string>("alvo") }; }
8
9          BT::NodeStatus onStart() override
10         {
11             std::string alvo;
12             if (!getInput("alvo", alvo)) { return BT::NodeStatus::FAILURE; }
13             pedido_ = cliente_.enviaAssincrono(alvo); // nao bloqueia
14             return BT::NodeStatus::RUNNING;
15         }
16
17         BT::NodeStatus onRunning() override
18         {
19             if (!pedido_.pronto()) { return BT::NodeStatus::RUNNING; }
20             return pedido_.sucesso() ? BT::NodeStatus::SUCCESS : BT::NodeStatus::FAILURE;
21         }
22
23         void onHalted() override { cliente_.cancela(pedido_); }
24
25     private:
26         Cliente cliente_;
27         Pedido pedido_;
28 };
```

Listagem 42: O esqueleto de uma ação de pedido-e-resposta.

Armadilha — a mensagem de erro cita uma classe que não existe

Os dois `throw` acima dizem "`AsyncActionNode2::onStart()`" e "`AsyncActionNode2::onRunning()`". Não há classe `AsyncActionNode2` na biblioteca — é o nome que `StatefulActionNode` tinha antes de ser renomeada. Procurar por ele no código-fonte, diante do erro, não leva a lugar nenhum. São ainda `std::logic_error` crus, fora da hierarquia `BehaviorTreeException` (Seção III.7).

Regra — `onHalted()` é chamado só se a ação estava `RUNNING`

`StatefulActionNode::halt()` testa o estado antes de chamar `onHalted()`. Uma ação que ainda não começou, ou que já concluiu, não recebe a notificação de cancelamento — o que é o desejado, mas significa que `onHalted()` **não** é um destrutor lógico e não deve ser o único lugar onde recursos são liberados.

VII.4 AsyncActionNode: uma *thread* por execução

Quando não há como evitar bloquear, `AsyncActionNode` move o `tick()` para outra *thread*:

```
1 NodeStatus BT::AsyncActionNode::executeTick()
2 {
3     //send signal to other thread.
4     // The other thread is in charge for changing the status
5     if (status() == NodeStatus::IDLE)
6     {
7         setStatus( NodeStatus::RUNNING );
8         halt_requested_ = false;
9         thread_handle_ = std::async(std::launch::async, [this]() {
10             try { setStatus( tick() ); }
11             catch (std::exception&)
12             {
13                 std::cerr << "\nUncaught exception from the method tick(): ["
14                     << registrationName() << "/" << name() << "]\n" << std::endl;
15                 exptr_ = std::current_exception();
16                 thread_handle_.wait();
17             }
18             return status();
19         });
20     }
21     if( exptr_ ) { std::rethrow_exception(exptr_); }
22     return status();
23 }
```

Listagem 43: `AsyncActionNode::executeTick()`, em `src/action_node.cpp`.

O *tick* da árvore, portanto, não executa a ação: ele a *lança* e devolve imediatamente o estado corrente. Nos *ticks* seguintes, como o estado já é `RUNNING`, nada é lançado — apenas se lê o estado que a outra *thread* vai atualizar quando terminar.

Regra — o contrato do `halt()` cooperativo

O cancelamento não é forçado: `AsyncActionNode::halt()` apenas liga um `std::atomic_bool` e espera a *thread* terminar (`thread_handle_.wait()`). É o seu `tick()` que precisa consultar `isHaltRequested()` periodicamente e voltar. Se ele não consultar, o `halt()` bloqueia a árvore inteira até a ação acabar por conta própria — um `Timeout` sobre uma ação que ignora o pedido de parada não interrompe nada; apenas espera.

```
1 BT::NodeStatus MoveBaseAction::tick()
2 {
3     Pose2D goal;
```

```

4   if ( !getInput<Pose2D>("goal", goal)) {
5       throw BT::RuntimeError("missing required input [goal]");
6   }
7   _halt_requested.store(false);
8   int count = 0;
9   // It is up to you to check periodicall _halt_requested and interrupt
10  // this tick() if it is true.
11  while (!_halt_requested && count++ < 25) { SleepMS(10); }
12
13  return _halt_requested ? BT::NodeStatus::FAILURE : BT::NodeStatus::SUCCESS;
14 }
15
16 void MoveBaseAction::halt() { _halt_requested.store(true); }

```

Listagem 44: O padrão correto, de `sample_nodes/movebase_node.cpp`.

Armadilha — este `halt()` não chama o da classe base

O cabeçalho de `AsyncActionNode` instrui: “*in the overridden halt() method, you can do some cleanup, but do not forget to invoke the base class method AsyncActionNode::halt()*”. O exemplo oficial `movebase_node.cpp` — reproduzido acima — **não a invoca**. A consequência é que o `halt()` devolve o controle sem esperar a *thread*: o nó é posto em IDLE enquanto a *thread* antiga ainda roda e ainda pode chamar `setStatus()`. Se o nó for *ticado* de novo antes de ela terminar, duas *threads* escrevem o mesmo estado. Siga o cabeçalho, não o exemplo.

Armadilha — a exceção chega um *tick* atrasada, e o *handler* espera a si mesmo

O bloco `catch` guarda a exceção em `exptr_` e ela só é relançada na *próxima* chamada a `executeTick()` — o *tick* em que o erro ocorreu devolve `RUNNING` normalmente, e a árvore segue como se nada tivesse acontecido até ser *ticada* de novo. Pior: o `catch` chama `thread_handle_.wait()` sobre o *future* que a própria *thread* está preenchendo — e que pode nem ter sido atribuído ainda, porque a atribuição `thread_handle_ = std::async(...)` só acontece *depois* de a tarefa ser lançada. Chamar `wait()` num `std::future` inválido é comportamento indefinido. A leitura de `exptr_` também não é sincronizada: é um `std::exception_ptr` cru, escrito numa *thread* e lido em outra sem *mutex* nem atômico.

Armadilha — uma *thread* nova a cada execução

`std::async(std::launch::async, ...)` cria uma *thread* do sistema operacional por execução da ação — não há *pool*. Uma ação assíncrona dentro de um `Repeat` de mil ciclos cria mil *threads* ao longo do tempo. Para ações de curta duração e alta frequência, `StatefulActionNode` é ordens de grandeza mais barato.

VII.5 CoroActionNode: suspender em vez de bloquear

A quarta forma usa co-rotinas de Boost: o `tick()` roda numa pilha própria e pode suspender a si mesmo no meio, devolvendo `RUNNING` ao chamador e retomando exatamente daquele ponto no *tick* seguinte.

```

1   void CoroActionNode::setStatusRunningAndYield()
2   {
3       setStatus( NodeStatus::RUNNING );
4       (*_p->yield_ptr)();
5   }
6
7   NodeStatus CoroActionNode::executeTick()
8   {
9       if( !(_p->coro) || !(*_p->coro) ) {
10          _p->coro.reset( new coroutine<void>::pull_type(_p->func) );

```

```

11     return status();
12 }
13 if( status() == NodeStatus::RUNNING && (bool)_p->coro ) { (*_p->coro)(); }
14 return status();
15 }

```

Listagem 45: O ponto de suspensão e a retomada, em `src/action_node.cpp`.

Escrever a ação fica quase idêntico a escrevê-la de forma bloqueante — basta trocar cada espera por `setStatusRunningAndYield()`. É a melhor ergonomia das quatro, e a única que não obriga a explodir a lógica em *callbacks* ou em máquina de estados.

Armadilha — não existe nesta configuração

Toda a classe está dentro de `#ifndef BT_NO_COROUTINES`, e esse macro é definido pelo `CMakeLists.txt` quando Boost não é encontrado. A receita Conan deste *fork* desabilita a busca por Boost deliberadamente e propaga `BT_NO_COROUTINES` aos consumidores (Seção XI.3). Na prática: `CoroActionNode` não existe no binário, o exemplo `t09` e o teste `gtest_coroutines.cpp` não são compilados, e um `#include` de `action_node.h` não declara a classe. É a maior funcionalidade ausente desta configuração.

Armadilha — `CoroActionNode::halt()` descarta a pilha sem desenrolá-la

`halt()` é `_p->coro.reset();` — e só. A co-rotina é destruída no ponto em que estava suspensa. O Boost desenrola a pilha nesse caso, então os destrutores locais rodam; mas o seu `tick()` não recebe aviso nenhum, e qualquer limpeza precisa estar em objetos RAII na pilha da co-rotina, nunca em código após o ponto de suspensão. Se você sobrescrever `halt()`, o cabeçalho lembra de chamar `CoroActionNode::halt()` ao final.

VII.6 Como escolher, na prática

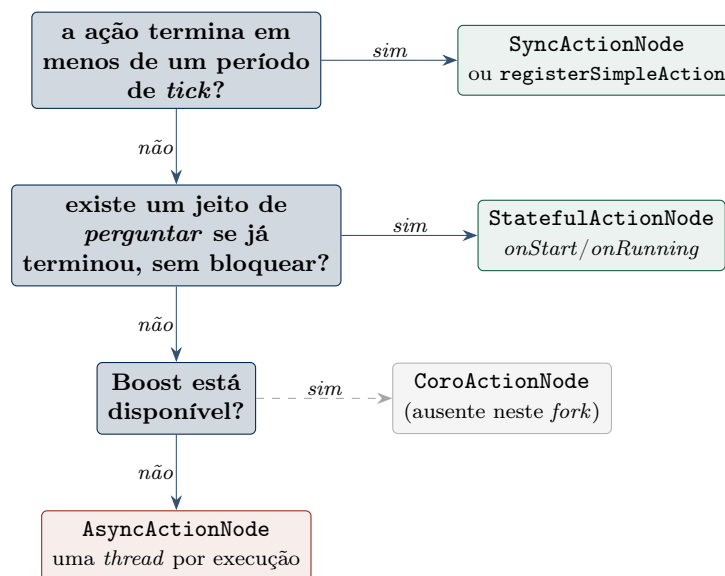


Figura 10: Escolha da classe base de uma ação. O caminho da direita é sempre preferível ao de baixo.

VII.7 Ações auxiliares embutidas

Três ações vêm registradas de fábrica e não precisam de código nenhum:

- `AlwaysSuccess` e `AlwaysFailure` — devolvem sempre o mesmo resultado. Servem de *stub* em árvores em construção e de terceiro filho explícito num `IfThenElse`.
- `SetBlackboard` — escreve um valor textual numa entrada do *blackboard*.

```
1 static PortsList providedPorts()
2 {
3     return { InputPort("value",
4         "Value represented as a string. convertFromString must be implemented."),
5         BidirectionalPort("output_key",
6             "Name of the blackboard entry where the value should be written") };
7 }
8
9 virtual BT::NodeStatus tick() override
10 {
11     std::string key, value;
12     if ( !getInput("output_key", key) ) throw RuntimeError("missing port [output_key]");
13     if ( !getInput("value", value) )   throw RuntimeError("missing port [value]");
14     setOutput("output_key", value);
15     return NodeStatus::SUCCESS;
16 }
```

Listagem 46: `SetBlackboard`, em `include/behaviortree_cpp_v3/actions/set_blackboard_node.h`.

Armadilha — `SetBlackboard` escreve texto, não o tipo de destino

O valor é sempre lido e escrito como `std::string`. Que ele acabe virando um `Position2D` depende da conversão diferida em `Blackboard::set()` (Seção IV.5.2), que só funciona se a entrada *já* tiver sido declarada com tipo por alguma porta tipada. Se o `SetBlackboard` for o primeiro a tocar a chave, ela fica sendo uma *string* — e o `getInput<Position2D>` do nó seguinte a converte a cada leitura, ou falha, conforme a especialização exista. Repare também que `output_key` é declarada *bidirecional*: ela é lida como entrada (para obter o nome da chave) e usada como saída (para escrever nela).

VII.8 Argumentos que não são portas

Nem toda configuração precisa vir do XML. Para parâmetros conhecidos em tempo de compilação ou de *deployment* — um ponteiro para um cliente, um identificador de *hardware* — portas são exagero. O exemplo `t08` mostra as duas saídas.

A primeira é registrar um *builder* próprio, injetando os argumentos por captura:

```
1 NodeBuilder builder_A = [](const std::string& name, const NodeConfiguration& config)
2 {
3     return std::make_unique<Action_A>( name, config, 42, 3.14, "hello world" );
4 };
5 factory.registerBuilder<Action_A>( "Action_A", builder_A);
```

Listagem 47: Injeção por *builder*, de `examples/t08_additional_node_args.cpp`.

A segunda é um método `init()` chamado depois da construção, percorrendo os nós já instanciados:

```

1  auto tree = factory.createTreeFromText(xml_text);
2
3  for( auto& node: tree.nodes )
4  {
5      if( auto action_B_node = dynamic_cast<Action_B*>( node.get() ))
6      {
7          action_B_node->init( 69, 9.99, "interesting_value" );
8      }
9  }

```

Listagem 48: Inicialização a posteriori, do mesmo exemplo.

Regra — prefira o *builder*

A segunda forma depende de o programador lembrar de chamar `init()` antes do primeiro *tick*, e nada verifica isso: esquecer produz um nó com membros não inicializados. A primeira torna a injeção parte da construção e não tem como ser esquecida. Use `init()` apenas quando o argumento só existir depois da árvore — por exemplo, um ponteiro para a própria *Tree*.

VII.9 No cenário da porta

Todas as folhas do exemplo `t05` são síncronas: as sete funções de `crossdoor_nodes.cpp` são registradas com `registerSimpleAction()` e `registerSimpleCondition()`, e simulam duração com `SleepMS()`. É a escolha didática certa e a escolha de engenharia errada — como diz a Seção VII.2, o `SleepMS(2000)` de `OpenDoor` bloqueia a árvore inteira. A Seção XII mostra a mesma porta com `PassThroughDoor` reescrito como `StatefulActionNode`, e o que muda no comportamento observável.

VIII O XML e a fábrica

Esta seção fecha o circuito: como um arquivo de texto vira uma árvore de objetos C++. São três peças — a fábrica, que sabe construir nós a partir de um nome; o formato XML, que descreve a topologia; e o *parser*, que faz a tradução. E uma quarta, opcional, que estende a primeira em tempo de execução: os plugins.

VIII.1 A fábrica

BehaviorTreeFactory é um dicionário de `string` para função construtora, mais um dicionário paralelo de manifestos:

```
1 typedef std::function<std::unique_ptr<TreeNode>(const std::string&,  
2                                             const NodeConfiguration&)> NodeBuilder;  
3 // ...  
4 std::unordered_map<std::string, NodeBuilder>    builders_;  
5 std::unordered_map<std::string, TreeNodeManifest> manifests_;  
6 std::set<std::string>                          builtin_IDs_;
```

Listagem 49: O estado interno, em `include/behaviortree_cpp_v3/bt_factory.h`.

O construtor registra sozinho os nós embutidos — 29 nesta configuração, ou 30 se *ncurses* estiver disponível — e depois copia as chaves para `builtin_IDs_` — é assim que a biblioteca sabe distinguir o que veio de fábrica do que o usuário acrescentou. O catálogo completo está no Seção XIII.

VIII.1.1 `registerNodeType<T>()`

O caminho normal é este, e ele faz cinco verificações em tempo de compilação:

```
1 static_assert(std::is_base_of<ActionNodeBase, T>::value ||  
2             std::is_base_of<ControlNode, T>::value ||  
3             std::is_base_of<DecoratorNode, T>::value ||  
4             std::is_base_of<ConditionNode, T>::value,  
5             "[registerNode]: accepts only classed derived from ...");  
6 static_assert(!std::is_abstract<T>::value,  
7             "[registerNode]: Some methods are pure virtual. "  
8             "Did you override the methods tick() and halt()?");  
9 static_assert(default_constructible || param_constructable,  
10             "[registerNode]: the registered class must have at least one of "  
11             "these two constructors: (const std::string&, const NodeConfiguration&) "  
12             "or (const std::string&).");  
13 static_assert(!(param_constructable && !has_static_ports_list),  
14             "[registerNode]: you MUST implement the static method: "  
15             "PortsList providedPorts();");  
16 static_assert(!(has_static_ports_list && !param_constructable),  
17             "[registerNode]: since you have a static method providedPorts(), "  
18             "you MUST add a constructor sign signature (const std::string&, "  
19             "const NodeParameters&);");
```

Listagem 50: Os `static_assert` de `registerNodeType<T>()` (mensagens condensadas).

As duas últimas formam um par: ou o nó tem *ambos* — construtor com `NodeConfiguration` e `providedPorts()` — ou *nenhum* dos dois. É a regra que impede o erro mais comum de quem começa: declarar portas e esquecer o construtor que recebe a configuração.

Regra — as mensagens de erro mentem sobre um nome

Duas delas falam em `NodeParameters`. Esse tipo não existe na v3 — é o nome que `NodeConfiguration` tinha na v2. Ao ler “*you MUST add a constructor sign signature (const std::string&, const NodeParameters&)*”, entenda `NodeConfiguration`. (O “sign signature” é do original.)

VIII.1.2 O construtor escolhido

`CreateBuilder<T>()` tem três sobrecargas, selecionadas por SFINAE conforme os construtores disponíveis. A do meio é a que reserva uma surpresa:

```
1  template <typename T> inline
2      NodeBuilder CreateBuilder(typename std::enable_if<has_default_constructor<T>::value &&
3                               has_params_constructor<T>::value >::type* = nullptr)
4  {
5      return [(const std::string& name, const NodeConfiguration& config)
6      {
7          // Special case. Use default constructor if parameters are empty
8          if( config.input_ports.empty() && config.output_ports.empty() &&
9              has_default_constructor<T>::value)
10         {
11             return std::make_unique<T>(name);
12         }
13         return std::make_unique<T>(name, config);
14     }];
15 }
```

Listagem 51: A sobrecarga para nós com os dois construtores, em `bt_factory.h`.

Armadilha — com os dois construtores, um nó sem atributos perde o *blackboard*

Se a sua classe tiver *ambos* os construtores e o XML não fornecer *nenhum* atributo de porta, a fábrica usa o construtor de um argumento — e o `NodeConfiguration`, inclusive o ponteiro para o *blackboard*, é descartado. O nó fica sem acesso a portas e sem acesso ao *blackboard*, e qualquer `getInput()` falha com “*NodeConfiguration::input_ports does not contain the key*” — inclusive para portas que teriam valor padrão, porque o padrão só é aplicado quando há o que preencher. A solução é não declarar o construtor de um argumento em nós que tenham portas.

VIII.1.3 Registro em tempo de execução

Quando a lista de portas só é conhecida no momento do registro — porque vem de um arquivo de configuração, por exemplo — há duas formas, mostradas no exemplo `t11`:

```
1  // more verbose way
2  PortsList think_ports = {BT::OutputPort<std::string>("text")};
3  factory.registerBuilder(CreateManifest<ThinkRuntimePort>("ThinkRuntimePort", think_ports),
4                          CreateBuilder<ThinkRuntimePort>());
5
6  // less verbose way
7  PortsList say_ports = {BT::InputPort<std::string>("message")};
8  factory.registerNodeType<SayRuntimePort>("SayRuntimePort", say_ports);
```

Listagem 52: Portas definidas em tempo de execução, de `examples/t11_runtime_ports.cpp`.

A sobrecarga com `PortsList` explícita exige o oposto da anterior: o nó **não** pode ter `providedPorts()` estático, “*to avoid ambiguities*”.

Armadilha — registrar duas vezes o mesmo ID é exceção, não substituição

`registerBuilder()` lança `BehaviorTreeException("ID [...] already registered")`. Não há sobreposição: para trocar a implementação de um ID é preciso `unregisterBuilder()` antes — e esse método *recusa* os IDs embutidos, com `LogicError("You can not remove the builtin registration ID")`. Ou seja, **não é possível redefinir Sequence, Timeout ou qualquer outro nó de fábrica**. Se dois *plugins* registrarem o mesmo nome, o segundo `registerFromPlugin()` lança no meio da carga, deixando a fábrica parcialmente povoada.

VIII.2 O formato XML

O formato é deliberadamente simples: cada tag é um nó, cada atributo é uma porta.

```
1 <root main_tree_to_execute="MainTree">
2
3   <BehaviorTree ID="MainTree">
4     <Sequence name="raiz">
5       <CheckBattery/>
6       <SaySomething message="ola"/>
7       <MoveBase goal="{destino}"/>
8     </Sequence>
9   </BehaviorTree>
10
11 </root>
```

Listagem 53: A estrutura mínima. Um `<root>`, uma ou mais `<BehaviorTree>`.

- `<root>` é obrigatório e deve ser o elemento de topo.
- `main_tree_to_execute` é obrigatório se houver mais de uma `<BehaviorTree>`; opcional se houver exatamente uma.
- Cada `<BehaviorTree>` tem exatamente **um** filho.
- O nome da tag é o ID de registro; o atributo `name` é o nome da instância e é opcional.
- Todo atributo que não seja ID nem `name` é tratado como porta.

Forma compacta e forma explícita. As duas linhas abaixo são equivalentes:

```
1 <SaySomething name="ola" message="Hello World"/>
2 <Action ID="SaySomething" name="ola" message="Hello World"/>
```

Listagem 54: A mesma ação nas duas sintaxes.

A forma explícita usa uma das cinco tags genéricas — `<Action>`, `<Condition>`, `<Control>`, `<Decorator>`, `<SubTree>` — e põe o ID num atributo. É mais verbosa e carrega mais informação: o Groot precisa dela (ou de um `<TreeNodesModel>`) para saber o tipo de cada nó sem consultar o binário.

VIII.2.1 A validação

Antes de instanciar coisa alguma, o *parser* chama `VerifyXML()`, uma função livre de cerca de duzentas linhas que confere aridade e obrigações:

Tag	Exigência
<root>	é o elemento de topo
<BehaviorTree>	exatamente 1 filho
<Decorator>	exatamente 1 filho, e atributo ID
<Action>, <Condition>	nenhum filho, e atributo ID
<Control>	ao menos 1 filho, e atributo ID
<Sequence>, <SequenceStar>, <Fallback>	ao menos 1 filho
<SubTree>	nenhum filho, e atributo ID
<TreeNodesModel>	no máximo um por arquivo
qualquer outra tag	precisa ser um ID registrado ou o ID de uma árvore

Tabela 8: As regras que `VerifyXML()` aplica, em `src/xml_parsing.cpp`.

Armadilha — a aridade só é conferida nas tags genéricas

Repare na tabela: `<Decorator ID="Timeout">` tem a aridade verificada, mas `<Timeout>` não. Da mesma forma, a exigência de “ao menos 1 filho” cobre `Sequence`, `SequenceStar` e `Fallback` — e **não** cobre `ReactiveSequence`, `ReactiveFallback`, `Parallel`, `IfThenElse`, `WhileDoElse` nem `Switch<N>`. Escrever a árvore na forma compacta troca uma parte da validação de carga por exceções em tempo de *tick* (Seção V.8).

Armadilha — uma mensagem de erro que imprime o próprio formato

Dentro da verificação do `<TreeNodesModel>`, a chamada é: `ThrowError(node->GetLineNum(), "Error at line %d: -> The attribute [ID] is mandatory")`. Mas `ThrowError` já formata `"Error at line %d: -> %s"` com o número da linha. O resultado é a mensagem `"Error at line 12: -> Error at line %d: -> The attribute [ID] is mandatory"` — com o `%d` literal, e o prefixo duplicado. Todas as outras chamadas passam apenas o texto.

Armadilha — `<TreeNodesModel>` ou `<TreeNodeModel>`?

O código lê `TreeNodesModel` (com `s`), e é isso que `writeTreeNodesModelXML()` gera. A documentação em `docs/xml_format.md` escreve `<TreeNodeModel>` (sem `s`) em todos os exemplos. Como a validação só percorre recursivamente o que está sob `<BehaviorTree>`, uma tag de nome errado no nível de `<root>` é **silenciosamente ignorada**: nem erro, nem modelo. Um Groot que não enxerga os seus nós costuma ser isto. O mesmo vale para `<Subtree>` minúsculo, que a documentação também usa: o parser exige `<SubTree>`.

VIII.2.2 <include>: dividir em arquivos

Um `<include>` no nível de `<root>` carrega outro arquivo antes de processar o corrente, à maneira de um `#include`:

```

1  <root main_tree_to_execute="BehaviorTree">
2
3      <include path="Check.xml" />
4      <include path="subtrees/Talk.xml" />
5
6      <BehaviorTree ID="BehaviorTree">
7          <Sequence>
8              <CheckStatus/>
9              <OpenGripper/>
10             <CloseGripper/>
11             <SayStuff/>

```

```

12     </Sequence>
13   </BehaviorTree>
14
15 </root>

```

Listagem 55: De `examples/test_files/subtree_test.xml`.

Repare que `<CheckStatus/>` e `<SayStuff/>` não são nós registrados: são os ID das `<BehaviorTree>` definidas nos arquivos incluídos, usados diretamente como tag. É uma forma abreviada de `<SubTree ID="CheckStatus"/>`, e o *parser* a reconhece explicitamente (Seção VIII.3).

Caminhos relativos são resolvidos a partir do diretório do arquivo que inclui — não do diretório de trabalho. O atributo `ros_pkg` permite resolver relativamente a um pacote ROS, e lança `RuntimeError` explicando que a biblioteca precisa ser recompilada com `catkin` se `USING_ROS` não estiver definido (Seção XI.2).

Armadilha — não há detecção de inclusão circular

`loadDocImpl()` chama a si mesma para cada `<include>`, sem manter um conjunto de arquivos já vistos. Dois arquivos que se incluam mutuamente produzem recursão infinita e estouro de pilha — sem mensagem. Não há guarda equivalente ao `#pragma once`.

Armadilha — cada arquivo é validado sozinho, na ordem em que é lido

A validação de um arquivo incluído enxerga apenas as árvores registradas *até aquele momento*. Um arquivo que referencie uma `<BehaviorTree>` definida no arquivo que o inclui falha com `"Node not recognized"`, ainda que o conjunto final seja consistente. A dependência tem de fluir num sentido só — do incluído para o includente — e os `<include>` precisam vir antes.

VIII.3 Da carga à árvore

`XMLParser` guarda os documentos abertos e um mapa de *ID de árvore* para elemento XML. `instantiateTree()` escolhe a árvore principal e chama a recursão:

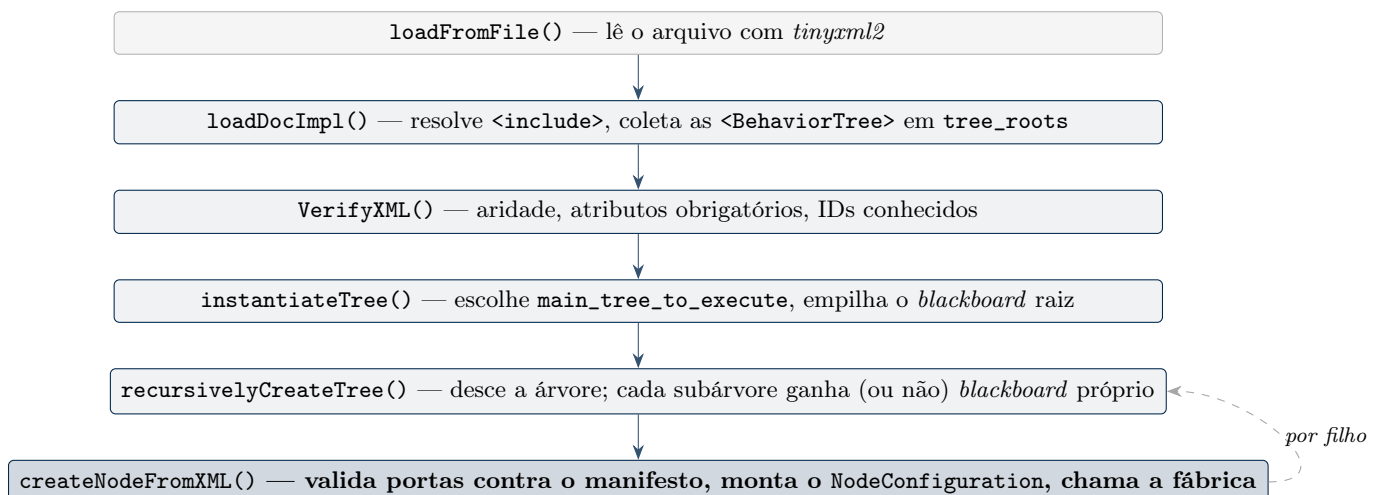


Figura 11: As seis etapas da carga. As três primeiras acontecem uma vez por arquivo; as três últimas, uma vez por nó.

`createNodeFromXML()` é onde a maior parte das verificações úteis acontece, em quatro passos:

1. **Confere cada atributo contra o manifesto.** Um atributo que não corresponda a nenhuma porta declarada produz a mensagem mais amigável de toda a biblioteca: *“Possible typo? In the XML, you tried to remap port ‘X’ in node [ID / nome], but the manifest of this node does not contain a port with this name.”*
2. **Declara os tipos no *blackboard*.** Para cada porta remapeada, registra a `PortInfo` na entrada correspondente — ou, se ela já existir com outro tipo, lança explicando que a entrada *“was initially created with type [X] and, later type [Y] was used somewhere else”*. É esta verificação, feita na carga, que impede o erro de tipo em tempo de execução.
3. **Separa entradas de saídas** conforme a direção declarada, preenchendo `config.input_ports` e `config.output_ports`. Uma porta INOUT vai para os dois.
4. **Aplica os valores padrão** das portas de entrada não fornecidas (Seção IV.6) e chama `factory.instantiateTreeNode()`.

Armadilha — IDs de árvore duplicados são silenciosamente ignorados

`tree_roots` é um `unordered_map` povoado com `insert()`, que **não** substitui uma chave existente. Duas `<BehaviorTree ID=“X”>` — por exemplo, uma no arquivo principal e outra num arquivo incluído — não geram aviso: vale a que foi lida primeiro, e a outra desaparece. Como os `<include>` são processados antes, quem vence é a do arquivo incluído.

Armadilha — uma `<BehaviorTree>` sem ID recebe um nome gerado

Se o atributo faltar, o *parser* inventa `“BehaviorTree_” + suffix_count++`. Isso é útil no caso de arquivo com uma árvore só — mas com duas árvores anônimas o `main_tree_to_execute` teria de citar `BehaviorTree_0`, um nome que depende da ordem de leitura dos arquivos. Dê ID a todas.

VIII.4 Subárvores e a fronteira de dados

Uma subárvore é uma `<BehaviorTree>` usada como nó dentro de outra. Estruturalmente é um decorador; o que a distingue é o tratamento do *blackboard*. A v3.5.6 oferece dois nós com políticas diferentes.

VIII.4.1 `<SubTree>`: isolamento com remapeamento

Por padrão, a subárvore recebe um *blackboard* novo, filho do atual, e **cada atributo do XML vira um remapeamento** de uma chave interna para uma chave externa:

```

1  <BehaviorTree ID="MainTree">
2    <Sequence name="main_sequence">
3      <SetBlackboard output_key="move_goal" value="1;2;3" />
4      <SubTree ID="MoveRobot" target="move_goal" output="move_result" />
5      <SaySomething message="{move_result}" />
6    </Sequence>
7  </BehaviorTree>
8
9  <BehaviorTree ID="MoveRobot">
10   <Fallback name="move_robot_main">
11     <SequenceStar>
12       <MoveBase goal="{target}" />
13       <SetBlackboard output_key="output" value="mission accomplished" />
14     </SequenceStar>

```

```

15     <ForceFailure>
16     <SetBlackboard output_key="output" value="mission failed" />
17     </ForceFailure>
18 </Fallback>
19 </BehaviorTree>

```

Listagem 56: Remapeamento de portas de subárvore, de `examples/t06_subtree_port_remapping.cpp`.

Lê-se: dentro de `MoveRobot`, a chave `target` é a chave `move_goal` do pai, e `output` é `move_result`. Note que os valores *não* levam chaves — num `<SubTree>` clássico, todo atributo é nome de chave externa, nunca literal.

```

1  auto new_bb = Blackboard::create(blackboard);
2  for (const XMLAttribute* attr = element->FirstAttribute(); attr; attr = attr->Next())
3  {
4      if ( strcmp(attr->Name(), "ID") == 0 ) { continue; }
5      new_bb->addSubtreeRemapping( attr->Name(), attr->Value() );
6  }
7  output_tree.blackboard_stack.emplace_back(new_bb);
8  recursivelyCreateTree( node->name(), output_tree, new_bb, node );

```

Listagem 57: A criação do *blackboard* isolado, em `src/xml_parsing.cpp`.

O atributo especial `__shared_blackboard="1"` desliga o isolamento: a subárvore passa a usar o *mesmo* *blackboard* do pai, e nenhum remapeamento é necessário — nem possível.

VIII.4.2 <SubTreePlus>: três formas de ligar

`SubtreePlusNode` generaliza o remapeamento com uma sintaxe que distingue chave de literal, e acrescenta o remapeamento automático. O cabeçalho `decorators/subtree_node.h` traz o exemplo canônico:

```

1  <Sequence>
2    <SetBlackboard value="Hello" output_key="myParam" />
3    <SubTreePlus ID="Talk" param="{myParam}" />
4
5    <SubTreePlus ID="Talk" param="World" />
6
7    <SetBlackboard value="Auto remapped" output_key="param" />
8    <SubTreePlus ID="Talk" __autoremap="1" />
9  </Sequence>

```

Listagem 58: As três formas de ligar uma subárvore, do cabeçalho `subtree_node.h`.

1. `param="{myParam}"` — remapeamento clássico: a chave interna `param` aponta para a chave `myParam` do pai.
2. `param="World"` — *sem* chaves: o valor literal é **escrito** na entrada `param` do *blackboard* novo. Não há ligação com o pai.
3. `__autoremap="1"` — toda chave da subárvore que não tenha sido citada explicitamente é remapeada para a chave de mesmo nome no pai.

Armadilha — o `__autoremap` só enxerga chaves que já existem

O bloco que aplica o `__autoremap` roda *depois* de `recursivelyCreateTree()`, e percorre `new_bb->getKeys()` — as chaves que a construção da subárvore já criou. Uma chave que só passe

a existir no primeiro *tick* — porque nenhum nó a declarou como porta tipada na carga — não é remapeada. O “automático” é “automático para o que o *parser* viu”, e a diferença aparece como uma entrada que não atravessa a fronteira sem que nada tenha sido escrito errado.

Armadilha — o atributo `name` de um `<SubTree>` é ignorado

Para `SubTree` e `SubTreePlus`, o *parser* faz `instance_name = element->Attribute("ID")` — sobrescrevendo qualquer `name` fornecido. A razão é operacional: a recursão localiza a árvore por `node->name()`. O efeito colateral é que, num traço de execução ou no Groot, todas as instâncias de uma mesma subárvore aparecem com o mesmo nome, e não há como distingui-las.

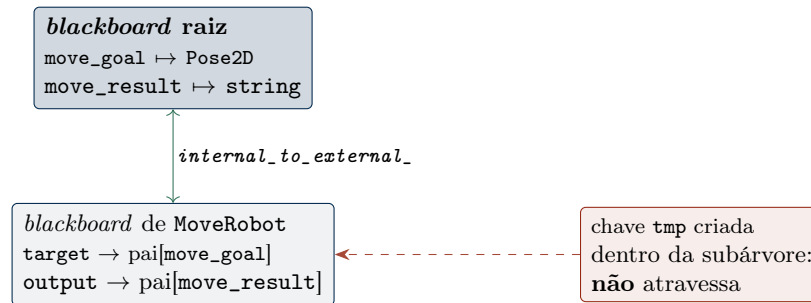


Figura 12: A fronteira entre *blackboards*. Só as chaves listadas no mapa de remapeamento atravessam; todas as outras são locais à subárvore.

VIII.5 Plugins

Um plugin é uma biblioteca compartilhada que exporta uma função de nome fixo. A macro cuida da visibilidade em cada plataforma:

```

1  constexpr const char* PLUGIN_SYMBOL = "BT_RegisterNodesFromPlugin";
2
3  #ifndef BT_PLUGIN_EXPORT
4  #define BT_REGISTER_NODES(factory) \
5      static void BT_RegisterNodesFromPlugin(BT::BehaviorTreeFactory& factory)
6  #else
7  #if defined(__linux__) || defined(__APPLE__)
8  #define BT_REGISTER_NODES(factory) \
9      extern "C" void __attribute__((visibility("default"))) \
10         BT_RegisterNodesFromPlugin(BT::BehaviorTreeFactory& factory)
11  #elif _WIN32
12  #define BT_REGISTER_NODES(factory) \
13      extern "C" void __declspec(dllexport) BT_RegisterNodesFromPlugin(...)
14  #endif
15  #endif

```

Listagem 59: A macro e o símbolo procurado, em `bt_factory.h`.

Do lado do código, é uma função:

```

1  // This function must be implemented in the .cpp file to create
2  // a plugin that can be loaded at run-time
3  BT_REGISTER_NODES(factory)
4  {
5      CrossDoor::RegisterNodes(factory);
6  }

```

Listagem 60: O ponto de entrada de um *plugin*, de `sample_nodes/crossdoor_nodes.cpp`.

Do lado do *build*, é uma biblioteca compartilhada compilada com `BT_PLUGIN_EXPORT` definido:

```
1  # to create a plugin, compile them in this way instead
2  add_library(crossdoor_nodes_dyn SHARED crossdoor_nodes.cpp )
3  target_link_libraries(crossdoor_nodes_dyn PRIVATE ${BEHAVIOR_TREE_LIBRARY})
4  target_compile_definitions(crossdoor_nodes_dyn PRIVATE BT_PLUGIN_EXPORT )
```

Listagem 61: De `sample_nodes/CMakeLists.txt`.

E do lado da aplicação, uma linha: `factory.registerFromPlugin("./libcrossdoor_nodes_dyn.so");`

Armadilha — esquecer `BT_PLUGIN_EXPORT` produz um erro silencioso

Sem a definição, a macro expande para uma função `static` — invisível ao `dlsym`. `registerFromPlugin()` então **não lança**: imprime "ERROR loading library [...]: can't find symbol [BT_RegisterNodesFromPlugin]" em `std::cout` e retorna normalmente. A carga do XML falha depois, com "Node not recognized", e a mensagem que explica o porquê já rolou para fora da tela.

Regra — o *plugin* e a aplicação precisam da mesma biblioteca

O *plugin* é carregado com `RTLD_NOW` | `RTLD_GLOBAL` (Seção X.4) e recebe uma referência à `BehaviorTreeFactory` da aplicação. Se os dois lados tiverem sido compilados contra versões diferentes da biblioteca, ou com *flags* de ABI incompatíveis, o resultado é corrupção silenciosa. Some-se a isso a comparação de `type_info` por ponteiro descrita na Seção IV.5.2, que pode produzir falsos erros de tipo entre os dois lados.

Plugins ROS. `registerFromROSPlugins()` consulta o `pluginlib` por bibliotecas que exportem `bt_lib_plugin` e as carrega todas. Só existe se a biblioteca tiver sido compilada com `catkin`; caso contrário, o método lança explicando isso (Seção XI.2).

VIII.6 O objeto Tree

O resultado da carga é um **Tree**, e vale conhecer os seus quatro membros públicos:

```
1  class Tree
2  {
3  public:
4      std::vector<TreeNode::Ptr> nodes;           // posse de todos os nos
5      std::vector<Blackboard::Ptr> blackboard_stack; // [0] e o do topo
6      std::unordered_map<std::string, TreeNodeManifest> manifests;
7
8      Tree(const Tree& ) = delete;                 // move-only
9      Tree& operator=(const Tree&) = delete;
10
11     void haltTree();
12     TreeNode* rootNode() const;
13     NodeStatus tickRoot();
14     Blackboard::Ptr rootBlackboard();
15     ~Tree();                                     // chama haltTree()
16 };
```

Listagem 62: `Tree`, em `bt_factory.h` (condensada).

`tickRoot()` é a única forma normal de executar a árvore, e faz uma coisa a mais que chamar `executeTick()`:

```

1 NodeStatus tickRoot()
2 {
3     if (!rootNode()) { throw RuntimeError("Empty Tree"); }
4     NodeStatus ret = rootNode()->executeTick();
5     if ( ret == NodeStatus::SUCCESS || ret == NodeStatus::FAILURE){
6         rootNode()->setStatus(BT::NodeStatus::IDLE);
7     }
8     return ret;
9 }

```

Listagem 63: `Tree::tickRoot()`.

É essa reposição a IDLE que fecha o ciclo da Figura 4 no topo da árvore: sem ela, a raiz nunca voltaria a executar depois de concluir uma vez.

Regra — o laço da aplicação

A forma canônica, presente em todos os exemplos, é:

```

1 NodeStatus status = NodeStatus::RUNNING;
2 while( status == NodeStatus::RUNNING ) {
3     status = tree.tickRoot();
4     std::this_thread::sleep_for(std::chrono::milliseconds(10)); // nao remove
5 }

```

O *sleep* não é decorativo: sem ele, a árvore é *ticada* tão rápido quanto o processador permitir, e toda condição é reavaliada milhões de vezes por segundo (Seção II.1).

Armadilha — `haltTree()` passa duas vezes por cada nó

`haltTree()` chama `rootNode()->halt()`, que já propaga para toda a subárvore, e *depois* percorre a árvore inteira com `applyRecursiveVisitor` chamando `halt()` de novo em cada nó — o comentário do fonte admite: “*but, just in case.... this should be no-op*”. Para um `AsyncActionNode` cujo `halt()` espera a *thread*, a segunda passagem é inofensiva; para um `halt()` de usuário com efeito colateral, não é. Escreva `halt()` idempotente.

VIII.7 Gerando o modelo para o Groot

`writeTreeNodesModelXML(factory)` devolve um XML com um `<TreeNodesModel>` descrevendo todos os nós registrados que *não* sejam embutidos e *não* sejam nós de controle — as duas exclusões são explícitas no código. É o que a ferramenta `bt3_plugin_manifest` usa para documentar um *plugin* (Seção IX.6).

VIII.8 No cenário da porta

O arquivo do exemplo `t05` declara duas `<BehaviorTree>` — `MainTree` e `DoorClosed` — e um `main_tree_to_execute="MainTree"`, exatamente como manda a Seção VIII.2. A `DoorClosed` é referida por `<SubTree ID="DoorClosed"/>` sem nenhum atributo: como não há dados a trocar, o isolamento padrão do *blackboard* é gratuito. E é por isso que o exemplo pôde usar variáveis `static` para o estado da porta — assim que esse estado for para o *blackboard*, o `<SubTree>` precisará de remapeamento explícito, como na Seção XII.

IX Observação e depuração

Uma árvore que não se pode observar é indepurável: o comportamento emerge da interação entre dezenas de nós, e nenhum *breakpoint* mostra isso. A biblioteca resolve o problema com um mecanismo só — assinar as transições de estado — e quatro consumidores diferentes para elas.

IX.1 StatusChangeLogger: a base comum

Todo *logger* herda de `StatusChangeLogger`, cujo construtor faz o trabalho inteiro: percorre a árvore e assina a mudança de estado de cada nó.

```
1 inline StatusChangeLogger::StatusChangeLogger(TreeNode* root_node)
2 : enabled_(true), show_transition_to_idle_(true), type_(TimestampType::ABSOLUTE)
3 {
4     first_timestamp_ = std::chrono::high_resolution_clock::now();
5
6     auto subscribeCallback = [this](TimePoint timestamp, const TreeNode& node,
7                                     NodeStatus prev, NodeStatus status) {
8         if (enabled_ && (status != NodeStatus::IDLE || show_transition_to_idle_))
9         {
10             if (type_ == TimestampType::ABSOLUTE)
11                 this->callback(timestamp.time_since_epoch(), node, prev, status);
12             else
13                 this->callback(timestamp - first_timestamp_, node, prev, status);
14         }
15     };
16
17     auto visitor = [this, subscribeCallback](TreeNode* node) {
18         subscribers_.push_back(node->subscribeToStatusChange(std::move(subscribeCallback)));
19     };
20     applyRecursiveVisitor(root_node, visitor);
21 }
```

Listagem 64: O construtor, em `include/behaviortree_cpp_v3/loggers/abstract_logger.h`.

A subclasse implementa apenas `callback()` e `flush()`. Três controles ficam disponíveis para todas: `setEnabled()` liga e desliga a gravação sem desfazer as assinaturas; `setTimestampType()` escolhe entre carimbo absoluto (desde a época) e relativo (desde a construção do *logger*); e `enableTransitionToIdle()` filtra as transições para `IDLE`, que são as mais numerosas e as menos informativas.

Armadilha — o comentário e o construtor discordam sobre o padrão

O cabeçalho anota `// false by default` sobre `showsTransitionToIdle()`, e o construtor inicializa `show_transition_to_idle_(true)`. O padrão real é **mostrar** as transições para `IDLE` — que num traço de `StdCoutLogger` facilmente dobram o volume de saída. Se o seu traço parece ruidoso demais, é isto: chame `enableTransitionToIdle(false)`.

Armadilha — o nome do método tem um `t` a menos

É `setTimestampType()`, não `setTimeStampType()`. Procurar pelo nome correto no cabeçalho não encontra nada, e o erro de compilação (*“no member named setTimeStampType”*) não sugere a grafia real.

Regra — construa os *loggers* depois da árvore e antes do primeiro *tick*

As assinaturas são feitas uma única vez, no construtor, sobre os nós que existiam naquele momento. E o `Signal` não é seguro sob concorrência (Seção III.2): assinar enquanto a árvore está sendo *ticada* é uma

condição de corrida. O *logger* também precisa **continuar vivo** — é ele quem detém os `shared_ptr` das assinaturas.

IX.2 StdCoutLogger: o traço no terminal

O mais direto dos quatro: uma linha por transição, com carimbo de tempo, nome do nó e a transição colorida.

```
1 constexpr const char* whitespaces = "                ";
2 constexpr const size_t ws_count = 25;
3
4 double since_epoch = duration<double>(timestamp).count();
5 printf("[%0.3f]: %s%s %s -> %s",
6         since_epoch, node.name().c_str(),
7         &whitespaces[std::min(ws_count, node.name().size())],
8         toStr(prev_status, true).c_str(),
9         toStr(status, true).c_str());
10 std::cout << std::endl;
```

Listagem 65: `StdCoutLogger::callback()`, em `src/loggers/bt_cout_logger.cpp`.

O alinhamento é feito indexando uma *string* de 25 espaços — um truque compacto que degrada com elegância: nomes com 25 caracteres ou mais simplesmente não recebem espaço nenhum antes da seta.

Armadilha — o `flush()` desarma o guarda de instância única

A classe protege-se contra duas instâncias com um `std::atomic<bool>` estático e lança `LogicError("Only one instance of StdCoutLogger shall be created")` na segunda. Mas `flush()` termina com `ref_count = false;` — e `flush()` é público e virtual. Depois de chamá-lo, nada impede criar um segundo `StdCoutLogger`, e os dois passam a imprimir cada transição em duplicata. A linha em questão é a única de todo o arquivo indentada com tabulação, o que sugere um acréscimo apressado.

Armadilha — os comentários de cor estão trocados

Em `toStr(NodeStatus, colored)`, o `SUCCESS` recebe a sequência `\x1b[32m` — verde — comentada como `// RED`, e o `FAILURE` recebe `\x1b[31m` — vermelho — comentada como `// GREEN`. As cores impressas estão certas; os comentários é que estão invertidos. O mesmo par invertido foi copiado para `tools/bt_log_cat.cpp`.

IX.3 FileLogger: o traço binário

Grava um arquivo com duas partes: um cabeçalho `FlatBuffers` descrevendo a topologia da árvore, e depois um fluxo de transições de 12 bytes cada.

```
1 flatbuffers :: FlatBufferBuilder builder(1024);
2 CreateFlatbuffersBehaviorTree(builder, tree);
3
4 file_os_.open(filename, std::ofstream::binary | std::ofstream::out);
5
6 // serialize the length of the buffer in the first 4 bytes
7 char size_buff[4];
8 flatbuffers :: WriteScalar(size_buff, static_cast<int32_t>(builder.GetSize()));
9 file_os_.write(size_buff, 4);
```

```
10 | file_os_.write(reinterpret_cast<const char*>(builder.GetBufferPointer()), builder.GetSize());
```

Listagem 66: A escrita do cabeçalho, em `src/loggers/bt_file_logger.cpp`.

O cabeçalho é o que torna o arquivo autocontido: ele carrega o UID, o nome de instância, o ID de registro, os filhos e as portas de cada nó, mais os manifestos de todos os nós registrados. Um leitor não precisa do XML nem do binário original para reconstruir a árvore — é o que faz a ferramenta `bt3_log_cat`.

O registro de transição. Cada transição são 12 bytes, montados à mão:

```
1  /** Serialize manually the informations about state transition
2   * No flatbuffer serialization here
3   */
4  inline SerializedTransition SerializeTransition (uint16_t UID, Duration timestamp,
5                                                  NodeStatus prev_status, NodeStatus status)
6  {
7      SerializedTransition buffer;                // std::array<uint8_t, 12>
8      int64_t usec = duration_cast<microseconds>(timestamp).count();
9      int64_t t_sec = usec / 1000000;
10     int64_t t_usec = usec % 1000000;
11
12     flatbuffers :: WriteScalar(&buffer[0], t_sec);
13     flatbuffers :: WriteScalar(&buffer[4], t_usec);
14     flatbuffers :: WriteScalar(&buffer[8], UID);
15     flatbuffers :: WriteScalar(&buffer[10], static_cast<int8_t>(convertToFlatbuffers(prev_status)));
16     flatbuffers :: WriteScalar(&buffer[11], static_cast<int8_t>(convertToFlatbuffers(status)));
17     return buffer;
18 }
```

Listagem 67: `SerializeTransition()`, em `flatbuffers/bt_flatbuffer_helper.h`.

Armadilha — o leiaute de 12 bytes só funciona por acidente

O formato pretendido é: 4 bytes de segundos, 4 de microssegundos, 2 de UID, 1 de estado anterior, 1 de estado novo. Mas `t_sec` e `t_usec` são `int64_t`, e `WriteScalar` escreve **oito** bytes cada. A escrita em `&buffer[0]` ocupa os bytes 0–7; a em `&buffer[4]` ocupa 4–11, sobrescrevendo metade da primeira; e as três escritas seguintes sobrescrevem 8–11. O resultado sai correto *porque* a máquina é *little-endian* e os valores cabem em 32 bits: em cada posição sobrevivem justamente os quatro bytes menos significativos. Num processador *big-endian*, ou passados 68 anos de *uptime*, o arquivo sai corrompido — sem nenhum aviso. Não há `static_assert`, e o comentário do fonte (“*Serialize manually*”) é a única pista de que o leiaute é manual.

Regra — o `buffer` do `FileLogger` custa dados num travamento

O construtor aceita `buffer_size` (padrão 10): as transições são acumuladas em memória e só vão para o disco a cada dez. Se o processo morrer, as últimas até nove transições — justamente as que interessam — não estão no arquivo. Passe 0 para escrita imediata quando estiver caçando um travamento.

IX.4 MinitraceLogger: a linha do tempo

Grava um JSON no formato de *trace* do Chrome, que se abre em `chrome://tracing` como um gráfico de barras temporal — a melhor ferramenta disponível para responder “o que estava demorando”.

```
1  const bool statusCompleted = (status == NodeStatus::SUCCESS || status == NodeStatus::FAILURE);
2  const char* category = toConstStr(node.type());
3  const char* name = node.name().c_str();
```

```

4 |
5 | if (prev_status == NodeStatus::IDLE && statusCompleted) { MTR_INSTANT(category, name); }
6 | else if (status == NodeStatus::RUNNING) { MTR_BEGIN(category, name); }
7 | else if (prev_status == NodeStatus::RUNNING && statusCompleted) { MTR_END(category, name); }

```

Listagem 68: O mapeamento de transição para evento, em `src/loggers/bt_minitrace_logger.cpp`.

A leitura é elegante: um nó que vai de `IDLE` direto ao resultado é um evento *instantâneo*; um que entra em `RUNNING` abre uma *barra*, fechada quando ele conclui. A categoria é o tipo do nó, o que permite filtrar ações, condições e controles separadamente.

Armadilha — a mensagem de erro cita a classe errada

O guarda de instância única do `MinitraceLogger` lança `LogicError("Only one instance of StdCoutLogger shall be created")` — copiado do *logger* de terminal e nunca ajustado. A classe também declara um membro `TimePoint prev_time_` que nada lê nem escreve.

IX.5 PublisherZMQ e o Groot

O quarto *logger* publica o estado da árvore em rede, e é o que dá ao Groot a visualização ao vivo. Ele abre *dois* sockets ZeroMQ:

- um `ZMQ_PUB` na porta **1666**, por onde as atualizações fluem continuamente;
- um `ZMQ_REP` na porta **1667**, servido por uma *thread* própria, que responde a qualquer pedido com o `FlatBuffers` da topologia da árvore.

A separação existe porque as duas informações têm frequências opostas: a topologia é fixa e grande, os estados mudam rápido e são pequenos. O Groot pede a topologia uma vez, ao conectar, e depois só escuta as atualizações.

```

1 | if (!send_pending_)
2 | {
3 |     send_pending_ = true;
4 |     send_future_ = std::async(std::launch::async, [this]() {
5 |         std::this_thread::sleep_for(min_time_between_msgs_);
6 |         flush();
7 |     });
8 | }

```

Listagem 69: A limitação de taxa, em `src/loggers/bt_zmq_publisher.cpp`.

O construtor recebe `max_msg_per_second` (padrão 25) e o converte em um intervalo mínimo. A primeira transição de uma rajada agenda um envio para daí a 40 ms; as demais apenas se acumulam no *buffer*, e todas saem juntas. É um agrupamento simples e eficaz, ao custo de uma *thread* nova a cada janela.

Armadilha — não é compilado nesta configuração

`src/loggers/bt_zmq_publisher.cpp` só entra na lista de fontes se o CMake achar ZeroMQ, e a receita Conan deste *fork* desabilita essa busca (Seção XI.3). Sem ele, não há monitoramento ao vivo pelo Groot — apenas a leitura *a posteriori* do arquivo do `FileLogger`. A ferramenta `bt3_recorder` também deixa de ser construída. O CMake avisa com um `message(WARNING "ZeroMQ NOT found...")`, que se perde no meio da saída de configuração.

Regra — um PublisherZMQ por processo, e portas distintas

O construtor lança se já houver instância viva, e lança de novo se as duas portas forem iguais (“*The TCP ports of the publisher and the server must be different*”). Trocar de árvore em tempo de execução exige destruir o *publisher* antigo antes de criar o novo — o CHANGELOG registra essa correção na versão 3.5.4, sob o título “*Improved switching BTs with active Groot monitoring*”.

IX.6 As ferramentas de linha de comando

O diretório `tools/` produz três executáveis, todos condicionados a `BUILD_TOOLS`:

Executável	Entrada	O que faz
<code>bt3_log_cat</code>	arquivo do <code>FileLogger</code>	imprime a árvore e as transições
<code>bt3_recorder</code>	rede (porta 1666)	grava o que o <code>PublisherZMQ</code> publica
<code>bt3_plugin_manifest</code>	um <code>.so</code> de <i>plugin</i>	lista os nós e as portas que ele registra

Tabela 9: As ferramentas de `tools/`. O `bt3_recorder` exige ZeroMQ; os outros dois, não.

`bt3_plugin_manifest` é o mais útil no dia a dia e o menos conhecido: ele cria uma fábrica vazia, anota os IDs embutidos, carrega o *plugin* e imprime a *diferença* — ou seja, exatamente o que aquela biblioteca acrescenta. É a forma de conferir se um *plugin* exporta o que se espera, sem escrever nenhum XML.

Armadilha — as ferramentas não são instaladas pelo pacote Conan

A receita define `BUILD_TOOLS=False` (Seção XI.3), de modo que o pacote contém apenas a biblioteca e os cabeçalhos. Para usar `bt3_log_cat` é preciso compilar o repositório à parte, com o CMake direto.

IX.7 Depuração sem *logger*

Duas funções cobrem os casos em que instrumentar é exagero:

- `printTreeRecursively(tree.rootNode())` — despeja a estrutura indentada em `std::cout`. Serve para confirmar que o XML produziu a árvore que se esperava, sobretudo com subárvores e `<include>`.
- `tree.blackboard_stack[i]->debugMessage()` — lista as chaves, os tipos e os remapeamentos de um *blackboard* (Seção IV.5). É a forma de descobrir por que uma porta não atravessou a fronteira de uma subárvore — lembrando das limitações registradas na Seção IV.5.

IX.8 Uma estratégia de depuração

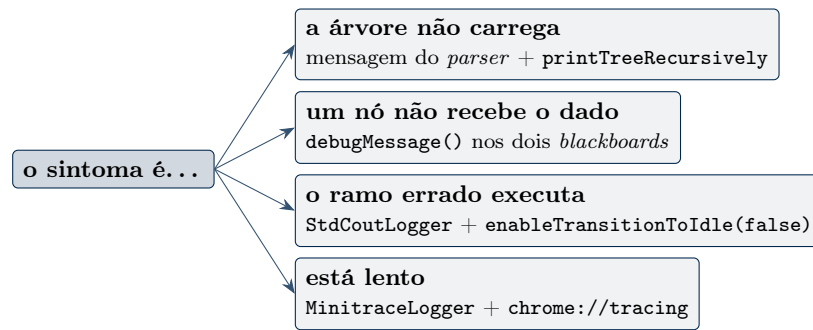


Figura 13: Qual instrumento usar para cada sintoma. Os três primeiros não exigem nenhuma dependência externa.

IX.9 No cenário da porta

O `main()` do exemplo `t05` liga três *loggers* de uma vez — terminal, arquivo binário e *trace* — e um quarto sob `#ifdef ZMQ_FOUND`. Todos são variáveis locais declaradas logo após a criação da árvore e antes do laço de *ticks*, exatamente como manda a Seção IX.1; se fossem temporários, as assinaturas morreriam na mesma linha e nenhum deles imprimiria nada. A Seção XII traz o código completo.

X A infraestrutura de suporte

Sob o núcleo há uma camada de utilitários que não sabe nada de árvores de comportamento (Seção II.2). Parte dela é código próprio; a maior parte, em linhas, é biblioteca de terceiros embutida. Esta seção descreve as duas, porque as duas aparecem nas mensagens de erro, nos tempos de compilação e nas obrigações de licença.

X.1 BT::Any: o valor de tipo apagado

Any é o que o blackboard guarda. Ela envolve um `linb::any` — a implementação de `std::any` embutida — e acrescenta duas coisas que a versão padrão não tem: *normalização* de tipos numéricos e *conversão segura* entre eles.

```
1 // Rational: since type erased numbers will always use at least 8 bytes
2 // it is faster to cast everything to either double, uint64_t or int64_t.
3 explicit Any(const double& value) : _any(value), _original_type(&typeid(double)) {}
4 explicit Any(const uint64_t& value) : _any(value), _original_type(&typeid(uint64_t)) {}
5 explicit Any(const float& value) : _any(double(value)), _original_type(&typeid(float)) {}
6 explicit Any(const std::string& str) : _any(SafeAny::SimpleString(str)),
7                                     _original_type(&typeid(std::string)) {}
8
9 // all the other integrals are casted to int64_t
10 template <typename T>
11 explicit Any(const T& value, EnableIntegral<T> = 0)
12 : _any(int64_t(value)), _original_type(&typeid(T)) {}
```

Listagem 70: A normalização, em `include/behaviortree_cpp_v3/utils/safe_any.hpp` (condensada).

Todo inteiro vira `int64_t`, todo real vira `double`, toda *string* vira `SimpleString` — mas o `type_info` original é guardado à parte. Daí os dois acessores: `type()` devolve o tipo declarado pelo usuário, `castedType()` devolve o tipo realmente armazenado.

Por que — por que normalizar

Sem normalização, um *blackboard* escrito por um nó como `int` e lido por outro como `unsigned` falharia — `any_cast` exige tipo exato. Com a normalização, ambos falam `int64_t` internamente, e a conversão de saída passa por `convertNumber()`, que verifica limites. O custo é uma indireção e um ponteiro extra por valor; o ganho é que o XML pode escrever "42" sem que o autor da árvore precise saber a largura exata do inteiro do outro lado.

`convertNumber()`. Em `utils/convert_impl.hpp` há uma família de sobrecargas selecionadas por SFINAE, uma para cada categoria de conversão numérica — e cada uma verifica o que precisa ser verificado:

Uma conversão que perderia informação lança `std::runtime_error` — por exemplo, “*Value is negative and can't be converted to signed*” (a mensagem diz *signed* onde queria dizer *unsigned*). É esse mecanismo que transforma um erro de tipo do XML em exceção legível, em vez de lixo numérico silencioso.

Armadilha — `Any::type()` desreferencia um ponteiro que pode ser nulo

O construtor padrão faz `_original_type(nullptr)`, e `type()` é `return *_original_type;`, sem verificação. Chamar `type()` num `Any` vazio é comportamento indefinido — e `Blackboard::debugMessage()` faz exatamente isso para entradas que foram declaradas mas nunca escritas (Seção IV.5).

Categoria	Verificação
mesmo tipo, ou destino <code>bool</code>	nenhuma
inteiro para inteiro maior, mesmo sinal	nenhuma (é segura)
real para real de outra largura	truncamento
sem sinal para menor sem sinal	limite superior
com sinal para menor com sinal	limites inferior e superior
com sinal para sem sinal	recusa negativos, e limite superior

Tabela 10: As conversões numéricas de `SafeAny::details::convertNumber` e o que cada uma confere antes do `static_cast`.

X.2 SimpleString: *small object optimization*

Uma `std::string` da `libstdc++` ocupa 32 bytes, o que estoura o armazenamento interno do `linb::any` e força alocação. `SimpleString` existe para caber: são duas palavras — uma união entre um ponteiro e um *buffer* de `sizeof(void*)` bytes, mais o tamanho.

```

1 private:
2     union{
3         char* ptr;
4         char soo[sizeof(void*)] ;
5     } _data;
6     std::size_t _size;
```

Listagem 71: O armazenamento, em `include/behaviortree_cpp_v3/utils/simple_string.hpp`.

Cadeias de até 7 caracteres vivem dentro do próprio objeto; a partir de 8, num `new char[]`. A escolha é feita comparando `_size` com `sizeof(void*)`, tanto na construção quanto no acesso e na destruição.

Armadilha — o `operator=` de `SimpleString` é uma dupla liberação à espera

```

1 SimpleString& operator = (const SimpleString& other)
2 {
3     _data = other._data; // copia a UNIAO: no caso longo, copia o ponteiro
4     _size = other._size;
5     return *this;
6 }
```

Para uma cadeia de 8 caracteres ou mais, os dois objetos passam a apontar para o mesmo *buffer*: o antigo vaza, e o destrutor de ambos executa `delete[]` sobre o mesmo endereço. A biblioteca não exercita esse caminho — o `linb::any` atribui por *copy-and-swap*, o que usa o construtor de cópia, que é correto — mas quem usar `SimpleString` diretamente encontra o defeito. O construtor de cópia e o destrutor estão certos; só a atribuição não está.

X.3 TimerQueue: temporizadores

Uma fila de prioridade de tarefas agendadas, servida por uma *thread* dedicada. É o motor do `TimeoutNode` e do `DelayNode` (Seção VI.3). O cabeçalho credita a origem: um artigo de blog de 2016 sobre filas de temporizador portáteis [2].

As garantias estão documentadas no próprio arquivo, e são incomuns o bastante para merecer transcrição:

```

1 // Guarantees:
2 // - All handlers are executed ONCE, even if canceled (aborted parameter will
```

```

3 // be set to true)
4 // - If TimerQueue is destroyed, it will cancel all handlers.
5 // - Handlers are ALWAYS executed in the Timer Queue worker thread.
6 // - Handlers execution order is NOT guaranteed

```

Listagem 72: O contrato, em `include/behaviortree_cpp_v3/decorators/timer_queue.h`.

A primeira garantia é a que estrutura o código dos dois decoradores: o *handler* recebe um `bool aborted` e **sempre** roda, mesmo cancelado. Cancelar não é remover da fila — isso quebraria a integridade do *heap*; é mover o *handler* para um item novo no topo, com tempo zero, e esvaziar o original.

```

1 // Instead of removing the item from the container (thus breaking the
2 // heap integrity), we set the item as having no handler, and put
3 // that handler on a new item at the top for immediate execution
4 newItem.end = std::chrono::time_point<_Clock, _Duration>();
5 newItem.id = 0; // Means it is a canceled item
6 newItem.handler = std::move(item.handler);
7 item.handler = nullptr;
8 m_items.push(std::move(newItem));

```

Listagem 73: O cancelamento sem quebrar o *heap* (comentário original condensado).

A classe `Queue` interna herda de `std::priority_queue` apenas para expor o contêiner protegido `c` — um truque conhecido, e o único jeito de percorrer os itens para cancelar.

Armadilha — uma *thread* por instância, e o destrutor abusa da fila

O construtor lança a *thread*; o destrutor cancela tudo e agenda um *handler* de zero milissegundo para desligar o laço — o comentário admite: “*Abusing the timer queue to trigger the shutdown*”. Como cada `<Timeout>` e cada `<Delay>` tem a sua própria instância (Seção VI.3), o número de *threads* do processo cresce com o número desses nós na árvore, não com a carga.

X.4 SharedLibrary: carga dinâmica

Deriva do POCO (a licença original está no topo do arquivo) e envolve `dlopen/dlsym` em UNIX e `LoadLibrary` em Windows, com uma implementação por plataforma. É o que sustenta o `registerFromPlugin()` da Seção VIII.5.

```

1 void SharedLibrary::load(const std::string& path, int)
2 {
3     std::unique_lock<std::mutex> lock(_mutex);
4     if (_handle) { throw RuntimeError("Library already loaded: " + path); }
5
6     _handle = dlopen(path.c_str(), RTLD_NOW | RTLD_GLOBAL);
7     if (!_handle)
8     {
9         const char* err = dlerror();
10        throw RuntimeError("Could not load library: " + (err ? std::string(err) : path));
11    }
12    _path = path;
13 }

```

Listagem 74: A carga em UNIX, em `src/shared_library_UNIX.cpp`.

Armadilha — o parâmetro `flags` é aceito e descartado

A interface documenta `SHLIB_GLOBAL` e `SHLIB_LOCAL`, com um comentário longo explicando que `RTLD_LOCAL` quebra RTTI entre bibliotecas. Mas a implementação UNIX declara o segundo parâmetro

sem nome — `load(const std::string& path, int)` — e usa sempre `RTLD_NOW | RTLD_GLOBAL`. Pedir `SHLIB_LOCAL` não tem efeito nenhum. Neste caso, felizmente, o comportamento fixo é o correto para *plugins*.

`prefix()` e `suffix()` devolvem `"lib"` e `".so"` — ou as variantes de macOS, HP-UX e Cygwin — e acrescentam um `"d"` antes da extensão em compilações de depuração. É o que `registerFromROSPlugins()` usa para montar o nome do arquivo a partir do nome do *plugin*.

X.5 StrCat e demangle

`utils/strcat.hpp` traz uma família de sobrecargas variádicas que concatenam `string_view`, `const char*` e `std::string` numa única alocação — o comentário a descreve como *“the fastest possible way to construct a string”*. Todas as exceções da biblioteca a usam (Seção III.7), o que explica a forma `throw RuntimeError("a", b, "c")` onipresente no código.

`utils/demangle_util.h` traduz um `std::type_info` para um nome legível, via `abi::__cxa_demangle` quando disponível. É o que produz as mensagens de erro de tipo do *blackboard* e os nomes de tipo no `<TreeNodesModel>` gerado. Em compiladores sem `cxxabi.h` — MSVC, por exemplo — devolve o nome cru do `type_info`, que já é legível naquele compilador.

X.6 Código de terceiros embutido

Esta é a parte que mais surpreende numa auditoria: em número de linhas, a maioria do repositório não é do projeto.

Componente	Onde	Linhas	Papel
<code>tinycl2</code>	<code>src/private/</code>	2 837	<i>parser</i> XML
<code>FlatBuffers</code>	<code>include/.../flatbuffers/</code>	2 747	serialização dos <i>loggers</i>
<i>expected</i>	<code>utils/expected.hpp</code>	1 957	<code>Optional/Result</code>
<i>string_view</i>	<code>utils/string_view.hpp</code>	1 532	<code>StringView</code>
<i>linb::any</i>	<code>utils/any.hpp</code>	463	base do <code>BT::Any</code>
<code>Minitrace</code>	<code>3rdparty/minitrace/</code>	—	<i>trace</i> do Chrome
<i>filesystem</i>	<code>3rdparty/filesystem/</code>	—	caminhos, para o <code><include></code>
<code>POCO</code> (parcial)	<code>utils/shared_library.h</code>	—	carga dinâmica
<code>cppzmq</code>	<code>src/loggers/zmq.hpp</code>	—	<i>binding</i> C++ do ZeroMQ

Tabela 11: As bibliotecas embutidas. As quatro primeiras somam mais de 9 000 linhas — várias vezes o tamanho do núcleo da `BehaviorTree.CPP`.

Por que — por que embutir em vez de depender

A biblioteca é feita para ser consumida por sistemas robóticos, onde a árvore de dependências já é longa e a versão do compilador raramente é a mais recente. Cada dependência externa é um item a mais na receita do integrador, e três das quatro maiores existem apenas para simular funcionalidades de C++17 (`std::any`, `std::string_view`, `std::expected`) num projeto fixado em C++14. Embuti-las torna a biblioteca compilável com `find_package(Threads)` e mais nada.

Regra — o `tinyxml2` está num *namespace* renomeado

O arquivo em `src/private/tinyxml2.h` declara `namespace BT_TinyXML2`, não `tinyxml2`. É proposital: uma aplicação pode usar a sua própria versão do `tinyxml2` sem colidir com a da biblioteca. O cabeçalho é privado — não é instalado — então isso é invisível ao consumidor.

Armadilha — embutir não dispensa a licença

A biblioteca é MIT, mas o pacote entregue contém código sob outras licenças — POCO (*Boost Software License*), `tinyxml2` (zlib), `FlatBuffers` (Apache 2.0), *expected/string_view* (Boost), `Minitrace` (MIT). A receita Conan copia apenas o `LICENSE` da raiz para `licenses/` (Seção XI.3), o que subdocumenta a distribuição. Numa entrega com exigência de conformidade, os avisos das bibliotecas embutidas precisam acompanhar o binário.

X.7 O que não está aqui

Vale registrar o que a biblioteca deliberadamente *não* oferece, para não procurar em vão:

- **Nenhum relógio ou agendador.** A frequência de *tick* é da aplicação (Seção II.1).
- **Nenhum *pool* de *threads*.** Cada `AsyncActionNode` cria a sua (Seção VII.4); cada `TimerQueue`, a dela.
- **Nenhuma persistência de estado.** A árvore não sabe salvar nem restaurar em que ponto parou; o `FileLogger` grava um traço, não um *checkpoint*.
- **Nenhuma alocação controlada.** Não há *allocator* configurável, e a construção da árvore aloca livremente — o que é irrelevante na carga e relevante em contexto de tempo real rígido.

XI O sistema de compilação

A BehaviorTree.CPP compila com CMake e, neste *fork*, é empacotada com Conan 2. As duas coisas se sobrepõem de um jeito que vale entender antes de integrá-la, porque o `CMakeLists.txt` tem uma característica incomum: o conteúdo do binário depende do que estiver instalado na máquina de quem compila.

XI.1 O `CMakeLists.txt`

O projeto chama-se `behaviortree_cpp_v3` — e esse é também o nome da biblioteca gerada e do diretório de cabeçalhos. O topo do arquivo fixa duas coisas e sonda uma terceira:

```
1 cmake_minimum_required(VERSION 3.5.1) # version on Ubuntu Xenial
2 project(behaviortree_cpp_v3)
3
4 #--- Enable C++14 ---
5 set(CMAKE_CXX_STANDARD 14)
6 set(CMAKE_CXX_STANDARD_REQUIRED ON)
7
8 #--- Include boost to add coroutines ---
9 find_package(Boost COMPONENTS coroutine QUIET)
10 if(Boost_FOUND)
11     # ... 1.59+ define BT_BOOST_COROUTINE2, 1.53+ define BT_BOOST_COROUTINE
12 endif()
13 if(NOT DEFINED BT_COROUTINES)
14     message(STATUS "Coroutines disabled. Install Boost to enable them (version 1.59+ recommended).")
15     add_definitions(-DBT_NO_COROUTINES)
16 endif()
```

Listagem 75: O início de `CMakeLists.txt`.

O padrão C++14 é uma decisão estrutural, não um detalhe: é ela que obriga às implementações embutidas de `any`, `string_view` e `expected` (Seção X.6).

XI.1.1 As opções

Opção	Padrão	Efeito
BUILD_EXAMPLES	ON	compila <code>examples/</code> e <code>sample_nodes/</code>
BUILD_UNIT_TESTS	ON	compila <code>tests/</code> (exige GTest)
BUILD_TOOLS	ON	compila <code>tools/</code>
BUILD_WITH_CURSES	ON	procura <code>ncurses</code> ; se achar, inclui o <code>ManualSelector</code>
BUILD_SHARED_LIBS	ON	biblioteca dinâmica em UNIX

Tabela 12: As opções declaradas em `CMakeLists.txt`. Note que todas são ON por padrão.

Armadilha — `BUILD_UNIT_TESTS=ON` torna o GTest obrigatório

Fora de um ambiente ROS, o `else if(BUILD_UNIT_TESTS)` final faz `find_package(GTest REQUIRED)`. Como a opção é ON por padrão, uma máquina sem GTest **falha na configuração** — e a mensagem fala de GTest, não de que basta desligar uma opção. A primeira coisa a fazer numa integração é passar `-DBUILD_UNIT_TESTS=OFF`.

XI.1.2 A biblioteca que muda de conteúdo

Quatro `find_package` silenciosos determinam *o que entra* no binário:

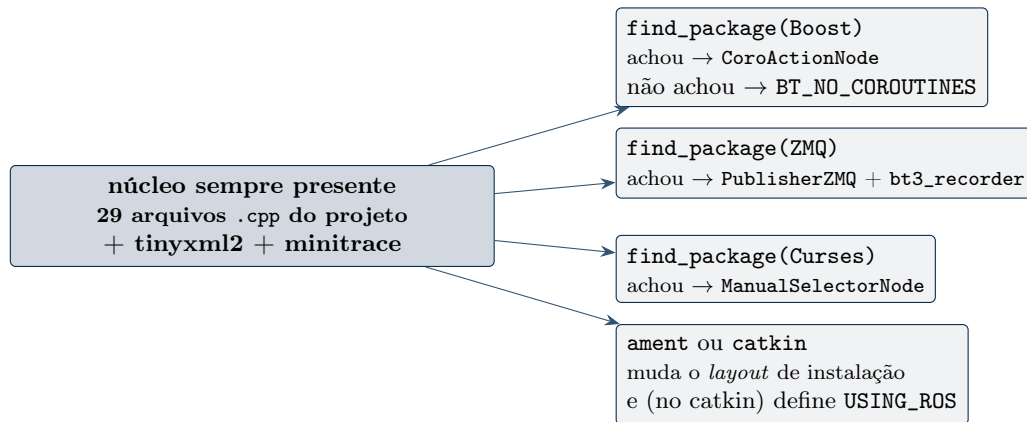


Figura 14: As quatro sondagens que alteram o conteúdo e a ABI da biblioteca. Nenhuma delas é uma opção declarada; todas dependem do que estiver instalado.

Armadilha — a mesma versão do fonte produz binários diferentes

Não há como pedir explicitamente “com co-rotinas” ou “sem ZeroMQ”. O *build* descobre sozinho, e o resultado depende da máquina. Duas consequências práticas: uma imagem de CI com Boost instalado por outro motivo produz uma biblioteca com `CoroActionNode` que a máquina de produção não tem; e um binário compilado numa máquina não é intercambiável com um compilado noutra, mesmo com o mesmo compilador e a mesma *tag*. É exatamente o problema que a receita Conan deste *fork* existe para resolver (Seção XI.3).

XI.1.3 Fontes por plataforma e tipo de biblioteca

```
1 if (UNIX)
2   list(APPEND BT_SOURCE src/shared_library_UNIX.cpp)
3   if (BUILD_SHARED_LIBS)
4     add_library(${BEHAVIOR_TREE_LIBRARY} SHARED ${BT_SOURCE})
5   else()
6     add_library(${BEHAVIOR_TREE_LIBRARY} STATIC ${BT_SOURCE})
7   endif()
8 endif()
9
10 if (WIN32)
11   set(CMAKE_DEBUG_POSTFIX "d")
12   list(APPEND BT_SOURCE src/shared_library_WIN.cpp)
13   add_library(${BEHAVIOR_TREE_LIBRARY} STATIC ${BT_SOURCE})
14 endif()
```

Listagem 76: A seleção da plataforma, em `CMakeLists.txt`.

Armadilha — `BUILD_SHARED_LIBS` é ignorado no Windows

No ramo `WIN32` a biblioteca é sempre `STATIC`, independentemente da opção — o que faz sentido, dado que a biblioteca não exporta símbolos com `__declspec(dllexport)`, mas contraria o nome da opção sem dizer nada. E há um caso pior: numa plataforma que não seja nem `UNIX` nem `WIN32`, *nenhum* `add_library` é executado, e a configuração falha adiante com um erro sobre alvo inexistente.

XI.1.4 Avisos e instalação

O alvo é compilado com `-Wall -Wextra -Werror=return-type` (GCC/Clang) ou `/W3 /WX` (MSVC) — ou seja, **no MSVC todo aviso é erro**. As regras de instalação exportam um alvo com *namespace*:

```
1 INSTALL(TARGETS ${BEHAVIOR_TREE_LIBRARY}  
2   EXPORT BehaviorTreeV3Config  
3   ARCHIVE DESTINATION ${BEHAVIOR_TREE_LIB_DESTINATION}  
4   LIBRARY DESTINATION ${BEHAVIOR_TREE_LIB_DESTINATION}  
5   RUNTIME DESTINATION ${BEHAVIOR_TREE_BIN_DESTINATION} )  
6  
7 install(EXPORT BehaviorTreeV3Config  
8   DESTINATION "${BEHAVIOR_TREE_LIB_DESTINATION}/BehaviorTreeV3/cmake"  
9   NAMESPACE BT::)
```

Listagem 77: A exportação do alvo.

Do lado do consumidor, portanto:

```
1 find_package(BehaviorTreeV3 REQUIRED)  
2 add_executable(minha_app main.cpp)  
3 target_link_libraries(minha_app PRIVATE BT::behaviortree_cpp_v3)
```

Listagem 78: Consumindo a biblioteca instalada, com CMake puro.

Armadilha — o `BT_NO_COROUTINES` não atravessa a exportação

Repare na assimetria. O ZeroMQ é propagado corretamente: `target_compile_definitions(... PUBLIC ZMQ_FOUND)` — portanto o consumidor recebe. Já o `BT_NO_COROUTINES` é definido com `add_definitions()`, que é uma propriedade de *diretório* e **não é exportada** com o alvo. O consumidor compila sem esse macro, `action_node.h` declara `CoroActionNode`, e o erro aparece só na ligação, como um símbolo indefinido de construtor. É a razão de a receita Conan deste *fork* acrescentar o macro à mão em `cpp_info.defines`.

Armadilha — os `cmake_minimum_required` são antigos demais

A raiz pede 3.5.1, com o comentário “*version on Ubuntu Xenial*”; `tools/CMakeLists.txt` pede 2.8. CMake 3.27 emite um aviso de depreciação para políticas anteriores a 3.5, e o CMake 4 **recusa** a configuração de projetos que pedem menos de 3.5. Numa distribuição recente, o *build* pode parar antes de compilar a primeira linha — e a mensagem fala de compatibilidade de políticas, não do arquivo a corrigir.

XI.2 ROS 1 e ROS 2

O `CMakeLists.txt` reconhece três ambientes e escolhe um leiaute de instalação para cada. A ordem importa: `ament` primeiro, `catkin` depois, CMake puro por último.

```
1 find_package(ament_cmake QUIET)  
2  
3 if ( ament_cmake_FOUND )  
4   # Not adding -DUSING_ROS since xml_parsing.cpp hasn't been ported to ROS2  
5   message(STATUS "BehaviourTree is being built using AMENT.")  
6 elseif( CATKIN_DEVEL_PREFIX OR CATKIN_BUILD_BINARY_PACKAGE)  
7   set(catkin_FOUND 1)  
8   add_definitions( -DUSING_ROS )  
9   find_package(catkin REQUIRED COMPONENTS roslib)  
10  message(STATUS "BehaviourTree is being built using CATKIN.")
```

```

11 elif(BUILD_UNIT_TESTS)
12     find_package(GTest REQUIRED)
13 endif()

```

Listagem 79: A detecção do ambiente ROS (condensada).

Armadilha — `ros_pkg` e `registerFromROSPlugins()` não funcionam no ROS 2

O comentário é explícito: o `USING_ROS` não é definido no caminho do `ament`, porque `xml_parsing.cpp` nunca foi portado. Consequência: sob ROS 2, o atributo `ros_pkg` de um `<include>` e o método `registerFromROSPlugins()` lançam a mesma `RuntimeError` de sempre — *“this library was compiled without ROS support. Recompile the BehaviorTree.CPP using catkin”* — que é um conselho impossível de seguir num ambiente ROS 2. Use caminhos relativos e `registerFromPlugin()` explícito.

O `package.xml` declara as dependências dos dois mundos com condições sobre `$ROS_VERSION`, e lista `libzmq3-dev` e `libncurses-dev` como dependências incondicionais — coerente com o fato de que, num *build* ROS, ambas serão encontradas e o binário resultante terá o `PublisherZMQ` e o `ManualSelectorNode`.

XI.3 O empacotamento Conan 2

Este é o acréscimo do *fork*: uma receita que fixa o que o `CMakeLists.txt` deixaria variável. O pacote chama-se `behaviortree.cpp.asa/3.5.6`.

```

1 def generate(self):
2     tc = CMakeToolchain(self)
3     tc.cache_variables["BUILD_SHARED_LIBS"] = bool(self.options.shared)
4     tc.cache_variables["BUILD_EXAMPLES"] = False
5     tc.cache_variables["BUILD_UNIT_TESTS"] = False
6     tc.cache_variables["BUILD_TOOLS"] = False
7     tc.cache_variables["BUILD_WITH_CURSES"] = False
8     # Determinismo: o CMakeLists procura estes pacotes no sistema e, se os
9     # achar, muda o conteúdo/ABI do pacote (coroutines com Boost, logger
10    # ZMQ, layout de instalacao do ament) sem que a receita os declare.
11    tc.cache_variables["CMAKE_DISABLE_FIND_PACKAGE_Boost"] = True
12    tc.cache_variables["CMAKE_DISABLE_FIND_PACKAGE_ZMQ"] = True
13    tc.cache_variables["CMAKE_DISABLE_FIND_PACKAGE_ament_cmake"] = True
14    tc.generate()

```

Listagem 80: O `generate()` da receita, em `conanfile.py`.

Por que — por que desligar tudo em vez de declarar as dependências

Um pacote binário precisa ser reprodutível: o mesmo *package ID* tem de corresponder ao mesmo conteúdo, sempre. Como o `CMakeLists.txt` muda o binário conforme o que houver na máquina, a receita tem duas saídas — declarar `Boost`, `ZeroMQ`, `ncurses` e `ament` como *requires* (e arrastar quatro dependências para todo consumidor) ou desligá-las e documentar a ausência. A receita escolhe a segunda, e as três variáveis `CMAKE_DISABLE_FIND_PACKAGE_*` são a forma padrão do CMake de dizer “não procure, mesmo que esteja aí”.

E o `package_info()` corrige a assimetria de exportação descrita acima:

```

1 def package_info(self):
2     self.cpp_info.libs = ["behaviortree_cpp_v3"]
3     # Sem Boost a lib é compilada com -DBT_NO_COROUTINES; o mesmo define
4     # precisa chegar ao consumidor, senao action_node.h declara
5     # CoroActionNode, que nao existe no binario.
6     self.cpp_info.defines = ["BT_NO_COROUTINES"]
7     if self.settings.os in ("Linux", "FreeBSD"):

```

```
8 | self.cpp_info.system_libs = ["pthread", "dl"]
```

Listagem 81: O `package_info()` da receita.

XI.3.1 O Makefile de conveniência

O *fork* traz ainda um Makefile que encapsula os comandos do Conan. Os alvos úteis:

Alvo	O que faz
<code>make configure</code>	resolve dependências e gera o <i>toolchain</i> em <i>build/</i>
<code>make build</code>	compila localmente, sem tocar no <i>cache</i> do Conan
<code>make create</code>	compila e instala o pacote no <i>cache</i> local
<code>make create-all</code>	faz o mesmo para Debug e Release
<code>make list</code>	lista as revisões no <i>cache</i>
<code>make path</code>	imprime a pasta do pacote no <i>cache</i>
<code>make remove</code>	remove o pacote do <i>cache</i>
<code>make upload</code>	envia o pacote a um <i>remote</i>

Tabela 13: Os alvos do Makefile do *fork*. O padrão é `BUILD_TYPE=Debug` e `SHARED=True`; ambos são sobrescrevíveis na linha de comando.

```
1 | make create BUILD_TYPE=Release    # gera e instala no cache local
2 | make list                        # confere a revisao
3 | make path                        # onde o pacote ficou
```

Listagem 82: O ciclo típico de trabalho no *fork*.

Armadilha — o padrão do Makefile é Debug

`BUILD_TYPE := Debug` está no topo do arquivo. Um `make create` sem argumentos produz um pacote de depuração — mais lento, maior, e possivelmente incompatível de ABI com o resto da aplicação em algumas combinações de MSVC. Para produção, sempre `BUILD_TYPE=Release`, ou `make create-all` para ter os dois no *cache*.

XI.3.2 Consumindo o pacote

```
1 | class MinhaApp(ConanFile):
2 |     settings = "os", "compiler", "build_type", "arch"
3 |     generators = "CMakeDeps", "CMakeToolchain"
4 |
5 |     def requirements(self):
6 |         self.requires("behaviortree.cpp.asa/3.5.6")
```

Listagem 83: O `conanfile.py` de uma aplicação que usa a biblioteca.

```
1 | find_package(behaviortree.cpp.asa REQUIRED)
2 | add_executable(minha_app main.cpp)
3 | target_link_libraries(minha_app PRIVATE behaviortree.cpp.asa::behaviortree.cpp.asa)
```

Listagem 84: E o `CMakeLists.txt` correspondente.

Regra — `shared=True` é o padrão, e importa para *plugins*

A opção `shared` da receita começa em `True`. Isso não é indiferente: um plugin carregado com `RTLD_GLOBAL` precisa compartilhar com a aplicação *a mesma instância* da biblioteca — em particular, os mesmos `type_info` e os mesmos contadores estáticos como o de `UID` (Seção III.3). Com a biblioteca estática ligada duas vezes, cada lado tem os seus, e os sintomas são exatamente os descritos na Seção IV.5.2.

XI.4 Compilando o repositório diretamente

Quando o objetivo é rodar os exemplos, os testes ou as ferramentas — que o pacote Conan não traz — o caminho é o CMake puro:

```
1 cd tmp/BehaviorTree.CPP
2 cmake -B build-cmake -S . \
3     -DCMAKE_BUILD_TYPE=Release \
4     -DBUILD_UNIT_TESTS=OFF      # senao exige GTest
5 cmake --build build-cmake -j
6
7 ./build-cmake/bin/t05_cross_door    # o exemplo do cenario deste manual
8 ./build-cmake/bin/bt3_log_cat bt_trace.fbl
```

Listagem 85: Compilação direta, com tudo ligado.

Os executáveis vão para `build-cmake/bin/` e a biblioteca para `build-cmake/lib/`, conforme os `CMAKE_*_OUTPUT_DIRECTORY` definidos no ramo não-ROS.

XI.5 Resumo das dependências

Dependência	Obrigatória?	O que se perde sem ela
<i>Threads</i> (pthread)	sim	nada compila
libdl	sim (UNIX)	<code>registerFromPlugin()</code>
Boost <i>coroutine</i>	não	<code>CoroActionNode</code>
ZeroMQ	não	<code>PublisherZMQ</code> , <code>Groot</code> ao vivo, <code>bt3_recorder</code>
<i>ncurses</i>	não	<code>ManualSelectorNode</code>
GTest	só para os testes	<code>tests/</code>
catkin / ament	não	<i>layout</i> ROS e <code>ros_pkg</code>

Tabela 14: As dependências e o custo de dispensar cada uma. No empacotamento deste *fork*, apenas as duas primeiras estão presentes.

XII Apêndice: uma aplicação completa

Este apêndice reúne o cenário da porta (Seção II.8) numa aplicação que compila e roda: um arquivo `.cpp`, um `CMakeLists.txt`, um `conanfile.py` e os comandos de *build*. Ele é uma versão elaborada do exemplo `t05_crossdoor` do repositório, com as três correções que os capítulos anteriores recomendaram:

- o estado do mundo não é global — é injetado por *builder* (Seção VII.8);
- a ação longa não bloqueia — é um `StatefulActionNode` (Seção VII.3);
- o resultado atravessa a fronteira da subárvore por remapeamento de porta, e não por variável estática (Seção VIII.4).

Toda a saída transcrita adiante foi produzida pela execução deste programa contra a biblioteca da Seção I.3.

XII.1 A árvore

```
1 <root main_tree_to_execute="MainTree">
2
3   <BehaviorTree ID="PortaFechada">
4     <Sequence name="destrancar_e_passar">
5       <Inverter>
6         <EstaAberta/>
7       </Inverter>
8       <RetryUntilSuccessful num_attempts="4">
9         <Sequence name="uma_tentativa">
10           <Timeout msec="2000">
11             <Destrancar duracao_ms="200"/>
12           </Timeout>
13           <Abrir/>
14         </Sequence>
15       </RetryUntilSuccessful>
16       <Atravessar via="porta" rota="{rota}"/>
17     </Sequence>
18   </BehaviorTree>
19
20   <BehaviorTree ID="MainTree">
21     <Sequence name="missao">
22       <Aproximar goal="1.5;2.0;0" duracao_ms="300"/>
23       <Fallback name="estrategias">
24         <Sequence name="ja_estava_aberta">
25           <EstaAberta/>
26           <Atravessar via="porta" rota="{rota_final}"/>
27         </Sequence>
28         <SubTree ID="PortaFechada" rota="rota_final"/>
29         <Atravessar via="janela" rota="{rota_final}"/>
30       </Fallback>
31       <Falar message="{rota_final}"/>
32       <Fechar/>
33     </Sequence>
34   </BehaviorTree>
35
36 </root>
```

Listagem 86: As duas árvores. A subárvore recebe do pai apenas a chave `rota`.

Três pontos merecem ser lidos com o vocabulário dos capítulos anteriores:

- `<RetryUntilSuccessful>` está com a grafia da biblioteca, com um `s` a menos — é obrigatório (Seção VI.2).
- `<SubTree ID="PortaFechada" rota="rota_final"/>` — o atributo liga a chave *interna* `rota` à chave *externa* `rota_final`, e por isso não leva chaves (Seção VIII.4).
- Dentro da subárvore, `rota="{rota}"` *leva* chaves: ali é um ponteiro para o *blackboard* local (Seção IV.3).

XII.2 O tipo próprio e a sua conversão

```

1 struct Pose2D { double x, y, theta; };
2
3 namespace BT
4 {
5     template <> inline Pose2D convertFromString(StringView str)
6     {
7         auto partes = splitString(str, ',');
8         if (partes.size() != 3) {
9             throw RuntimeError("Pose2D espera tres numeros separados por ','");
10        }
11        Pose2D p;
12        p.x = convertFromString<double>(partes[0]);
13        p.y = convertFromString<double>(partes[1]);
14        p.theta = convertFromString<double>(partes[2]);
15        return p;
16    }
17    // namespace BT

```

Listagem 87: Pose2D e a especialização que o XML precisa (Seção IV.4).

XII.3 Uma ação assíncrona sem *thread*

```

1 class Aproximar : public BT::StatefulActionNode
2 {
3     public:
4         Aproximar(const std::string& name, const BT::NodeConfiguration& config)
5             : BT::StatefulActionNode(name, config) {}
6
7         static BT::PortsList providedPorts()
8         {
9             return { BT::InputPort<Pose2D>("goal", "destino, no formato x;y;theta"),
10                    BT::InputPort<unsigned>("duracao_ms", 300, "quanto o trajeto demora") };
11         }
12
13         BT::NodeStatus onStart() override
14         {
15             Pose2D goal;
16             if (!getInput("goal", goal)) {
17                 throw BT::RuntimeError("[", name(), "] faltou a porta [goal]");
18             }
19             unsigned dur = 300;
20             getInput("duracao_ms", dur);
21             fim_ = std::chrono::steady_clock::now() + std::chrono::milliseconds(dur);
22             std::cout << " [Aproximar] indo para " << goal.x << " " << goal.y
23                 << " " << goal.theta << std::endl;
24             return BT::NodeStatus::RUNNING;
25         }
26
27         BT::NodeStatus onRunning() override
28         {
29             if (std::chrono::steady_clock::now() < fim_) {

```

```

30         return BT::NodeStatus::RUNNING;
31     }
32     std::cout << " [Aproximar] cheguei" << std::endl;
33     return BT::NodeStatus::SUCCESS;
34 }
35
36 void onHalted() override { std::cout << " [Aproximar] cancelado" << std::endl; }
37
38 private:
39     std::chrono::steady_clock::time_point fim_;
40 };

```

Listagem 88: Aproximar: o padrão pedido-e-resposta do Seção VII.3.

XII.4 Nós que precisam do mundo

Destrancar e Abrir precisam de um ponteiro para o estado da porta. Em vez de uma variável global, recebem-no no construtor — e a fábrica o injeta por *builder* (Seção VII.8).

```

1 struct Porta { bool aberta = false; bool trancada = true; int tentativas = 0; };
2
3 class Destrancar : public BT::StatefulActionNode
4 {
5     public:
6     Destrancar(const std::string& name, const BT::NodeConfiguration& config, Porta* porta)
7         : BT::StatefulActionNode(name, config, porta_(porta) {})
8     {
9         static BT::PortsList providedPorts()
10         {
11             return { BT::InputPort<unsigned>("duracao_ms", 200, "quanto a tentativa demora") };
12         }
13
14         BT::NodeStatus onStart() override
15         {
16             unsigned dur = 200; getInput("duracao_ms", dur);
17             fim_ = std::chrono::steady_clock::now() + std::chrono::milliseconds(dur);
18             return BT::NodeStatus::RUNNING;
19         }
20
21         BT::NodeStatus onRunning() override
22         {
23             if (std::chrono::steady_clock::now() < fim_) return BT::NodeStatus::RUNNING;
24
25             porta_->tentativas++;
26             if (porta_->tentativas < 3) {
27                 std::cout << " [Destrancar] emperrou (tentativa "
28                     << porta_->tentativas << ")" << std::endl;
29                 return BT::NodeStatus::FAILURE;
30             }
31             porta_->trancada = false;
32             std::cout << " [Destrancar] destrancada na tentativa "
33                 << porta_->tentativas << std::endl;
34             return BT::NodeStatus::SUCCESS;
35         }
36
37         void onHalted() override {}
38
39     private:
40         Porta* porta_;
41         std::chrono::steady_clock::time_point fim_;
42     };

```

Listagem 89: A fechadura que emperra nas duas primeiras tentativas.

XII.5 Uma ação com entrada e saída

```
1 class Atravessar : public BT::SyncActionNode
2 {
3     public:
4         Atravessar(const std::string& name, const BT::NodeConfiguration& config)
5             : BT::SyncActionNode(name, config) {}
6
7         static BT::PortsList providedPorts()
8         {
9             return { BT::InputPort<std::string>("via", "porta ou janela"),
10                     BT::OutputPort<std::string>("rota", "por onde acabei passando") };
11         }
12
13         BT::NodeStatus tick() override
14         {
15             std::string via;
16             if (!getInput("via", via)) {
17                 throw BT::RuntimeError("[", name(), "] faltou a porta [via]");
18             }
19             setOutput("rota", "atravessei pela " + via);
20             return BT::NodeStatus::SUCCESS;
21         }
22     };
```

Listagem 90: Atravessar: lê via, escreve rota.

XII.6 O main()

```
1 int main()
2 {
3     Porta porta;
4     BT::BehaviorTreeFactory factory;
5
6     // 1a. condicao e acao triviais, sem escrever classe
7     factory.registerSimpleCondition("EstaAberta", [&porta](BT::TreeNode&) {
8         return porta.aberta ? BT::NodeStatus::SUCCESS : BT::NodeStatus::FAILURE;
9     });
10    factory.registerSimpleAction("Fechar", [&porta](BT::TreeNode&) {
11        porta.aberta = false;
12        std::cout << " [Fechar] porta fechada" << std::endl;
13        return BT::NodeStatus::SUCCESS;
14    });
15
16    // 1b. nos com portas, registrados por tipo
17    factory.registerNodeType<Aproximar>("Aproximar");
18    factory.registerNodeType<Atravessar>("Atravessar");
19    factory.registerNodeType<Falar>("Falar");
20
21    // 1c. nos que precisam do mundo: builder proprio
22    factory.registerBuilder<Destrancar>("Destrancar",
23        [&porta](const std::string& name, const BT::NodeConfiguration& config) {
24            return std::make_unique<Destrancar>(name, config, &porta);
25        });
26    factory.registerBuilder<Abrir>("Abrir",
27        [&porta](const std::string& name, const BT::NodeConfiguration& config) {
28            return std::make_unique<Abrir>(name, config, &porta);
29        });
30
31    // 2. carga
32    auto tree = factory.createTreeFromText(xml_text);
33    BT::printTreeRecursively(tree.rootNode());
34
35    // 3. observadores: DEPOIS da arvore, ANTES do primeiro tick
36    BT::StdCoutLogger logger_terminal(tree);
37    logger_terminal.enableTransitionToIdle(false);
38    BT::FileLogger logger_arquivo(tree, "porta_trace.fbl", 0);
```

```

39
40 // 4. o laço
41 BT::NodeStatus status = BT::NodeStatus::RUNNING;
42 while (status == BT::NodeStatus::RUNNING)
43 {
44     status = tree.tickRoot();
45     std::this_thread::sleep_for(std::chrono::milliseconds(10));
46 }
47 std::cout << "\nresultado: " << status << std::endl;
48
49 std::cout << "\n--- blackboard raiz ---" << std::endl;
50 tree.blackboard_stack[0]->debugMessage();
51 std::cout << "--- blackboard da subarvore ---" << std::endl;
52 tree.blackboard_stack[1]->debugMessage();
53
54 return 0;
55 }

```

Listagem 91: Registro, carga, observadores e laço — os quatro passos da Figura 6.

XII.7 Compilando

```

1 cmake_minimum_required(VERSION 3.16)
2 project(porta CXX)
3
4 set(CMAKE_CXX_STANDARD 14)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6
7 find_package(behaviortree.cpp.asa REQUIRED)
8
9 add_executable(porta porta.cpp)
10 target_link_libraries(porta PRIVATE behaviortree.cpp.asa::behaviortree.cpp.asa)

```

Listagem 92: CMakeLists.txt da aplicação, consumindo o pacote Conan.

```

1 from conan import ConanFile
2 from conan.tools.cmake import cmake_layout
3
4 class PortaApp(ConanFile):
5     settings = "os", "compiler", "build_type", "arch"
6     generators = "CMakeDeps", "CMakeToolchain"
7
8     def layout(self):
9         cmake_layout(self)
10
11     def requirements(self):
12         self.requires("behaviortree.cpp.asa/3.5.6")

```

Listagem 93: conanfile.py da aplicação.

```

1 # 1. publicar a biblioteca no cache local (uma vez, no fork)
2 cd tmp/BehaviorTree.CPP
3 make create BUILD_TYPE=Release
4
5 # 2. compilar a aplicacao
6 cd ../../exemplo-porta
7 conan install . --build=missing --s build_type=Release
8 cmake --preset conan-release
9 cmake --build --preset conan-release
10
11 # 3. rodar
12 ./build/Release/porta

```

Listagem 94: Os comandos, do zero ao executável.

Regra — sem Conan, direto contra a árvore de fontes

Para experimentar sem publicar nada, basta compilar contra o repositório já construído:

```
1 g++ -std=c++14 -DBT_NO_COROUTINES \  
2 -I<repo>/include -I<repo>/3rdparty \  
3 porta.cpp -o porta \  
4 -L<repo>/build/Debug/lib -lbehaviortree_cpp_v3 -lpthread -ldl
```

O `-DBT_NO_COROUTINES` é obrigatório e é justamente o macro que a exportação do CMake não propaga (Seção XI.1).

XII.8 A saída

O programa começa imprimindo a árvore que o *parser* construiu — a forma mais rápida de conferir se o XML produziu o que se esperava:

```
1 -----  
2 missao  
3   Aproximar  
4   estrategias  
5     ja_estava_aberta  
6     EstaAberta  
7     Atravessar  
8     PortaFechada  
9       destrancar_e_passar  
10        Inverter  
11         EstaAberta  
12         RetryUntilSuccessful  
13         uma_tentativa  
14         Timeout  
15         Destrancar  
16         Abrir  
17         Atravessar  
18   Atravessar  
19   Falar  
20   Fechar  
21 -----
```

Listagem 95: `printTreeRecursively()`. Note que a subárvore aparece com o nome do seu ID (Seção VIII.4).

Em seguida vem o traço, aqui reproduzido com os carimbos de tempo abreviados e a coloração removida:

```
1 [.8.721]: missao          IDLE  -> RUNNING  
2 [Aproximar] indo para 1.5;2;0  
3 [.8.722]: Aproximar      IDLE  -> RUNNING  
4 [Aproximar] cheguei  
5 [.9.028]: Aproximar      RUNNING -> SUCCESS  
6 [.9.028]: estrategias    IDLE  -> RUNNING  
7 [.9.028]: ja_estava_aberta IDLE  -> RUNNING  
8 [.9.028]: EstaAberta     IDLE  -> FAILURE  
9 [.9.028]: ja_estava_aberta RUNNING -> FAILURE  
10 [.9.028]: PortaFechada   IDLE  -> RUNNING  
11 [.9.028]: destrancar_e_passar IDLE -> RUNNING  
12 [.9.028]: Inverter       IDLE  -> RUNNING  
13 [.9.028]: EstaAberta     IDLE  -> FAILURE  
14 [.9.028]: Inverter       RUNNING -> SUCCESS  
15 [.9.028]: RetryUntilSuccessful IDLE -> RUNNING  
16 [.9.028]: uma_tentativa  IDLE  -> RUNNING  
17 [.9.028]: Timeout        IDLE  -> RUNNING  
18 [.9.028]: Destrancar     IDLE  -> RUNNING  
19 [Destrancar] emperrou (tentativa 1)  
20 [.9.233]: Destrancar      RUNNING -> FAILURE  
21 [.9.233]: Timeout        RUNNING -> FAILURE  
22 [.9.233]: uma_tentativa  RUNNING -> FAILURE
```

```

23 | [..9.233]: uma_tentativa          IDLE  -> RUNNING
24 | [..9.233]: Timeout                IDLE  -> RUNNING
25 | [..9.233]: Destrancar             IDLE  -> RUNNING
26 | [Destrancar] emperrou (tentativa 2)
27 | [..9.439]: Destrancar             RUNNING -> FAILURE
28 | [..9.439]: Timeout                RUNNING -> FAILURE
29 | [..9.439]: uma_tentativa          RUNNING -> FAILURE
30 | [..9.439]: uma_tentativa          IDLE  -> RUNNING
31 | [..9.439]: Timeout                IDLE  -> RUNNING
32 | [..9.439]: Destrancar             IDLE  -> RUNNING
33 | [Destrancar] destrancada na tentativa 3
34 | [..9.644]: Destrancar             RUNNING -> SUCCESS
35 | [..9.644]: Timeout                RUNNING -> SUCCESS
36 | [Abrir] aberta
37 | [..9.644]: Abrir                  IDLE  -> SUCCESS
38 | [..9.644]: uma_tentativa          RUNNING -> SUCCESS
39 | [..9.644]: RetryUntilSuccessful   RUNNING -> SUCCESS
40 | [..9.644]: Atravessar             IDLE  -> SUCCESS
41 | [..9.644]: destrancar_e_passar    RUNNING -> SUCCESS
42 | [..9.644]: PortaFechada           RUNNING -> SUCCESS
43 | [..9.644]: estrategias            RUNNING -> SUCCESS
44 | [Falar] atravessei pela porta
45 | [..9.644]: Falar                  IDLE  -> SUCCESS
46 | [..9.644]: Fechar                 IDLE  -> RUNNING
47 | [Fechar] porta fechada
48 | [..9.644]: Fechar                 RUNNING -> SUCCESS
49 | [..9.644]: missao                 RUNNING -> SUCCESS
50 |
51 | resultado: SUCCESS

```

Listagem 96: O traço do `StdCoutLogger`, com as transições para `IDLE` suprimidas.

XII.9 Sete coisas para reparar nesse traço

O traço é a melhor revisão possível do manual inteiro. Vale percorrê-lo com sete perguntas.

1. Por que Fechar passa por `RUNNING` e Abrir não? Porque `Fechar` foi registrado com `registerSimpleAction()`, e `SimpleActionNode::tick()` chama `setStatus(RUNNING)` antes de invocar o *functor* (Seção VII.2). `Abrir` é uma classe derivada de `SyncActionNode`, cujo `tick()` vai direto ao resultado. Os dois são igualmente síncronos; a diferença é só de instrumentação.

2. Por que EstaAberta aparece duas vezes? São duas instâncias distintas do mesmo ID — uma na `Sequence ja_estava_aberta`, outra sob o `Inverter` da subárvore. O `StdCoutLogger` imprime o `name()`, não o `UID()` (Seção III.3), e como nenhuma das duas recebeu atributo `name`, as duas herdaram o ID (Seção III.1). Nomeie as instâncias se quiser distingui-las.

3. Por que uma_tentativa vai de `FAILURE` para `IDLE` e de volta a `RUNNING` no mesmo milissegundo? É o `haltChild()` do `RetryNode` entre tentativas (Seção VI.2): ele devolve o filho a `IDLE` para que ele possa ser executado de novo. A transição para `IDLE` está suprimida na saída, mas a volta a `RUNNING` denuncia que ela ocorreu.

4. Por que as três tentativas levam 200 ms cada e não bloqueiam? Porque `Destrancar` é um `StatefulActionNode`: cada `onRunning()` custa uma comparação de relógio, e o tempo passa entre *ticks*,

no *sleep* do laço do `main()`. Com um `SyncActionNode` que dormisse 200 ms, o traço seria idêntico — mas a árvore inteira ficaria parada, e o `<Timeout msec="2000">` não teria como interromper nada (Seção VII.2).

5. Por que o Timeout nunca disparou? Porque $200\text{ ms} < 2\,000\text{ ms}$. Ele está ali como rede de segurança, e o seu custo é uma *thread* viva durante toda a execução (Seção VI.3).

6. Por que Falar imprime o valor escrito *dentro* da subárvore? Porque *Atravessar* escreveu na chave `rota` do *blackboard* da subárvore, que o `<SubTree rota="rota_final"/>` remapeou para `rota_final` no *blackboard* do pai. O `debugMessage()` confirma os dois lados:

```
1  -- blackboard raiz --
2  rota_final (std::string) -> full
3  -- blackboard da subarvore --
4  rota (std::string) -> remapped to parent [rota_final]
```

Listagem 97: Os dois *blackboards* ao final da execução.

7. Por que os carimbos de tempo são números de dez dígitos? Porque o padrão de `StatusChangeLogger` é `TimestampType::ABSOLUTE`, ou seja, segundos desde a época (Seção IX.1). Para um traço legível, chame `logger.setTimestampType(BT::TimestampType::RELATIVE)` — lembrando da grafia do método.

XII.10 Exercícios de leitura

Três modificações de uma linha, cada uma ilustrando um capítulo. As três foram executadas; o que se descreve é o resultado observado, não o esperado.

1. Troque `<Sequence name="missao">` por `<ReactiveSequence>`. O resultado surpreende: a saída é *praticamente idêntica*, e a missão termina normalmente. A razão é instrutiva. O *Aproximar* de fato passa a ser *ticado* a cada passagem, mas `StatefulActionNode::tick()` só chama `onStart()` quando o estado é `IDLE` e só chama `onRunning()` quando é `RUNNING`; em `SUCCESS`, ele devolve o próprio estado sem executar nada (Seção VII.3). E a *ReactiveSequence* não repõe a `IDLE` os filhos *à esquerda* do que está `RUNNING` — apenas os *à direita* (Seção V.4). O único efeito visível no traço é a ordem das duas primeiras linhas: com *Sequence*, o *missao* aparece como `RUNNING` *antes* do filho; com *ReactiveSequence*, *depois* — porque os nós reativos são os únicos que não chamam `setStatus(RUNNING)` no início do `tick()` (Seção V.1). A lição é que “reativo” não basta para tornar uma ação reexecutável: é preciso que alguém a devolva a `IDLE`.

2. Troque `num_attempts="4"` por `"2"`. As duas tentativas falham, o *RetryUntilSuccessful* devolve `FAILURE`, a subárvore falha e o *Fallback* cai na terceira estratégia — o traço termina com `[Falar]` *atravessei pela janela* e a missão ainda assim devolve `SUCCESS` (Seção V.5).

3. Remova o `rota="rota_final"` do `<SubTree>`. A chave deixa de atravessar a fronteira. A árvore executa normalmente até o *Falar*, e então aborta com exceção não capturada:

```
1 terminate called after throwing an instance of 'BT::RuntimeError'
2   what(): [Falar] getInput() failed because it was unable to find the key
3         [message] remapped to [rota_final]
```

Note que **nada disso é detectado na carga**: o remapeamento ausente não é erro de validação, e a chave simplesmente não existe quando é procurada (Seção VIII.4). Note também que o `main()` deste exemplo não tem `try/catch` em volta do laço — num programa real, teria (Seção III.7).

XIII Apêndice: catálogo dos nós embutidos

Este apêndice lista os ID de registros que `BehaviorTreeFactory` registra sozinha no construtor, com as portas de cada um. É a referência a consultar ao escrever XML.

Regra — o catálogo não foi transcrito: foi extraído

As tabelas abaixo não vêm da leitura do `bt_factory.cpp`. Elas foram geradas por um programa de vinte linhas que constrói uma fábrica vazia e imprime `factory.manifests()` — ou seja, exatamente o que o *parser* enxerga em tempo de carga (Seção XIII.6). Onde a declaração no cabeçalho diverge do manifesto, é o manifesto que está registrado aqui.

XIII.1 Nós de controle

ID	Porta	Tipo	Padrão
Sequence	—	—	—
SequenceStar	—	—	—
ReactiveSequence	—	—	—
Fallback	—	—	—
ReactiveFallback	—	—	—
IfThenElse	—	—	—
WhileDoElse	—	—	—
Parallel	success_threshold failure_threshold	unsigned unsigned	nenhum 1
Switch2	variable	std::string	nenhum
	case_1...case_2	std::string	nenhum
Switch3	variable, case_1..3	std::string	nenhum
Switch4	variable, case_1..4	std::string	nenhum
Switch5	variable, case_1..5	std::string	nenhum
Switch6	variable, case_1..6	std::string	nenhum
ManualSelector [†]	repeat_last_selection	bool	false

Tabela 15: Nós de controle. [†] registrado apenas se a biblioteca foi compilada com *ncurses* — não é o caso nesta configuração (Seção V.9).

Aridades a lembrar: `SwitchN` exige $N + 1$ filhos (os casos mais o *default*); `IfThenElse` aceita 2 ou 3; `WhileDoElse` exige exatamente 3, apesar da mensagem em contrário (Seção V.7).

XIII.2 Decoradores

Armadilha — quatro portas obrigatórias sem valor padrão

`num_cycles`, `num_attempts`, `msec` e `delay_msec` não têm padrão. Omiti-las no XML não é erro de carga: é uma `RuntimeError` no primeiro *tick* daquele nó, com a mensagem “*Missing parameter [...]*”. Num ramo raro da árvore, isso pode demorar meses para aparecer.

ID	Porta	Tipo	Padrão
Inverter	—	—	—
ForceSuccess	—	—	—
ForceFailure	—	—	—
KeepRunningUntilFailure	—	—	—
Repeat	num_cycles	int	nenhum
RetryUntilSuccessful	num_attempts	int	nenhum
Timeout	msec	unsigned	nenhum
Delay	delay_msec	unsigned	nenhum
BlackboardCheckInt	value_A	(sem tipo)	nenhum
	value_B	(sem tipo)	nenhum
	return_on_mismatch	NodeStatus	nenhum
BlackboardCheckDouble	idênticas às de BlackboardCheckInt		
BlackboardCheckString	idênticas às de BlackboardCheckInt		
SubTree	—	—	—
SubTreePlus	—	—	—

Tabela 16: Decoradores. SubTree e SubTreePlus têm type() igual a SUBTREE, mas herdam de DecoratorNode.

Armadilha — o manifesto de SubTree está vazio

A tabela não tem erro de omissão. SubtreeNode declara `__shared_blackboard` e SubtreePlusNode declara `__autoremap`, mas em ambos o método `providedPorts()` está na seção `private`, e por isso a detecção por SFINAE falha e a lista chega vazia à fábrica (Seção IV.1). Os dois atributos *funcionam* mesmo assim, porque o *parser* os lê diretamente do XML (Seção VIII.4) — mas eles não aparecem no manifesto, não aparecem no Groot e não são validados.

XIII.3 Ações

ID	Porta	Direção	Tipo
AlwaysSuccess	—	—	—
AlwaysFailure	—	—	—
SetBlackboard	value	entrada	(sem tipo)
	output_key	bidirecional	(sem tipo)

Tabela 17: Ações embutidas. Nenhuma delas devolve RUNNING.

XIII.4 Resumo por tipo

XIII.5 IDs que não são o nome da classe

Nem todo ID coincide com o nome da classe C++, e a diferença importa porque é o ID que vai no XML.

NodeType	Quantidade	Quais
Control	13	7 sem portas, Parallel, Switch2..6
Decorator	11	4 sem portas, 4 de parâmetro, 3 de <i>blackboard</i>
SubTree	2	SubTree, SubTreePlus
Action	3	AlwaysSuccess, AlwaysFailure, SetBlackboard
Condition	0	nenhuma condição vem de fábrica
total	29	

Tabela 18: Contagem por tipo, conforme `factory.manifests()`. Os dois nós de subárvore contam como `SubTree`, e não como `Decorator`, por causa da ordem dos testes em `getType<T>()` (Seção III.5).

Classe C++	ID no XML	Observação
RetryNode	RetryUntilSuccessful	grafia com um s a menos
SequenceStarNode	SequenceStar	sufixo Node removido
SwitchNode<2>...<6>	Switch2...Switch6	instanciações do <i>template</i>
BlackboardPreconditionNode<int>	BlackboardCheckInt	nome inteiramente diferente
BlackboardPreconditionNode<double>	BlackboardCheckDouble	idem
BlackboardPreconditionNode<std::string>	BlackboardCheckString	idem
TimeoutNode<>	Timeout	<i>template</i> com parâmetros padrão
SubtreeNode	SubTree	T maiúsculo no XML
SubtreePlusNode	SubTreePlus	idem
ManualSelectorNode	ManualSelector	sufixo removido

Tabela 19: IDs de registro que diferem do nome da classe. O primeiro é o único em que a diferença é um erro de digitação (Seção VI.2).

XIII.6 Como este catálogo foi verificado

O programa abaixo é a fonte das tabelas. Ele não depende de nenhum XML: constrói uma fábrica vazia e imprime o que ela registrou sozinha, ordenado alfabeticamente. Rodar isto contra a sua própria compilação é a forma de saber, sem ambiguidade, quais nós estão disponíveis — inclusive os que dependem de *ncurses* ou de um plugin.

```

1  #include "behaviortree_cpp_v3/bt_factory.h"
2  #include <map>
3  #include <iostream>
4
5  int main()
6  {
7      BT::BehaviorTreeFactory factory;
8      // std::map (e nao unordered_map) so para a saida sair ordenada
9      std::map<std::string, BT::TreeNodeManifest> ordenado;
10     for (const auto& it : factory.manifests()) ordenado.insert(it);
11
12     std::cout << "total de IDs registrados: " << ordenado.size() << "\n\n";
13     for (const auto& it : ordenado)
14     {
15         const auto& m = it.second;
16         std::cout << m.registration_ID << " [" << BT::toStr(m.type) << "]"
17             << " portas=" << m.ports.size() << "\n";
18
19         std::map<std::string, BT::PortInfo> portas;
20         for (const auto& p : m.ports) portas.insert(p);
21         for (const auto& p : portas)
22         {
23             std::cout << "   - " << p.first
24                 << " dir=" << BT::toStr(p.second.direction())
25                 << " tipo=" << (p.second.type()
26                     ? BT::demangle(p.second.type())

```

```

27         : std::string("(nenhum)")
28         << " default=" << (p.second.defaultValue().empty()
29                             ? std::string("(nenhum)")
30                             : p.second.defaultValue())
31         << "\n";
32     }
33 }
34 return 0;
35 }

```

Listagem 98: scripts/catalogo.cpp — o extrator do catálogo.

```

1  g++ -std=c++14 -DBT_NO_COROUTINES \
2  -I<repo>/include -I<repo>/3rdparty \
3  scripts/catalogo.cpp -o catalogo \
4  -L<repo>/build/Debug/lib -lbehaviortree_cpp_v3 -lpthread -ldl
5
6  LD_LIBRARY_PATH=<repo>/build/Debug/lib ./catalogo

```

Listagem 99: Compilando e rodando o extrator.

A saída de referência — a que produziu as tabelas deste apêndice — começa assim:

```

1  total de IDs registrados: 29
2
3  AlwaysFailure [Action] portas=0
4  AlwaysSuccess [Action] portas=0
5  BlackboardCheckDouble [Decorator] portas=3
6      - return_on_mismatch dir=Input tipo=BT::NodeStatus default=(nenhum)
7      - value_A dir=Input tipo=(nenhum) default=(nenhum)
8      - value_B dir=Input tipo=(nenhum) default=(nenhum)
9  ...
10 Parallel [Control] portas=2
11     - failure_threshold dir=Input tipo=unsigned int default=1
12     - success_threshold dir=Input tipo=unsigned int default=(nenhum)
13 ...
14 SubTree [SubTree] portas=0
15 SubTreePlus [SubTree] portas=0

```

Listagem 100: Trecho da saída do extrator (as três primeiras entradas e a contagem).

Regra — o mesmo programa serve para auditar um *plugin*

Basta acrescentar `factory.registerFromPlugin("./meu_plugin.so")` antes do laço, ou usar a ferramenta `bt3_plugin_manifest`, que faz exatamente isso e ainda subtrai os nós embutidos da saída (Seção IX.6).

Referências

- [1] Faconti, D., and Colledanchise, M., “BehaviorTree.CPP: a C++ library to build Behavior Trees,” <https://github.com/BehaviorTree/BehaviorTree.CPP>, 2021. Versão 3.5.6, licença MIT. Acesso em 2026.
- [2] Figueira, R., “Portable C++ Timer Queue,” <http://www.crazygaze.com/blog/2016/03/24/portable-c-timer-queue/>, 2016. Origem do `TimerQueue` embutido na biblioteca.

Glossário

ação assíncrona

Ação que pode devolver `RUNNING` e continuar em curso entre *ticks*. Na v3 há três realizações: `AsyncActionNode` (*thread* própria), `StatefulActionNode` (máquina de estados) e `CoroActionNode` (co-rotina)

ação síncrona

Ação que conclui dentro do próprio *tick* e por isso nunca devolve `RUNNING` (`SyncActionNode`). Bloqueia a árvore enquanto executa, e por isso deve ser curta

apagamento de tipo

A técnica pela qual o *blackboard* guarda valores de qualquer tipo numa única estrutura (`BT::Any`), recuperando o tipo original apenas na leitura. É o que permite ao XML falar de `double` e `Pose2D` com a mesma sintaxe

blackboard

O dicionário *chave* → valor com tipo apagado (`Any`) pelo qual os nós trocam dados. Cada subárvore isolada tem o seu, encadeado ao do pai por um mapa de remapeamentos

decorador

Nó de um único filho que modifica o resultado, a frequência ou a duração da execução dele — inverter, repetir, forçar um resultado, impor prazo

FlatBuffers

Formato de serialização binária, embutido no repositório, usado para descrever a topologia da árvore no cabeçalho dos registos do `FileLogger` e nas mensagens do `PublisherZMQ`

Groot

O editor e monitor gráfico do projeto. Lê a árvore pelo `<TreeNodeesModel>` do XML e acompanha a execução ao vivo pelas mensagens do `PublisherZMQ`

halt

O cancelamento explícito de um nó que está `RUNNING`. Percorre a subárvore devolvendo cada nó a `IDLE`; é o único mecanismo pelo qual uma ação assíncrona é interrompida de fora

ID de registo

A *string* pela qual um nó é instanciável a partir do XML. **Não precisa coincidir com o nome da classe C++** — e em vários nós embutidos não coincide, inclusive com erro de grafia (`RetryUntilSuccessful`)

manifesto

O trio *tipo de nó*, *ID de registo* e *lista de portas* que a fábrica guarda para cada nó registrado (`TreeNodeManifest`). É o que permite ao *parser* validar atributos do XML antes de instanciar qualquer coisa

Minitrace

Biblioteca embutida que grava eventos no formato de *trace* do Chrome (`chrome://tracing`). É o que o `MinitraceLogger` alimenta para produzir a linha do tempo de execução dos nós

nó com memória

Nó de controle que guarda em qual filho parou e retoma dali no *tick* seguinte, sem reexecutar os filhos já concluídos. `Sequence`, `SequenceStar` e `Fallback` têm memória — o que as distingue é quando essa memória é zerada

nó reativo

Nó de controle que reavalia *todos* os filhos anteriores a cada *tick*, em vez de retomar do filho que ficou `RUNNING`. `ReactiveSequence` e `ReactiveFallback` são os dois casos canônicos

NodeStatus

O valor de retorno de todo *tick*: `SUCCESS`, `FAILURE`, `RUNNING` ou `IDLE`. `IDLE` é estado, não resultado — nenhum `tick()` de usuário deve devolvê-lo

plugin

Biblioteca compartilhada que exporta o símbolo `BT_RegisterNodesFromPlugin` e registra os seus nós na fábrica em tempo de execução, sem que o executável precise conhecê-los em tempo de compilação

porta (*port*)

A declaração estática de uma entrada ou saída de um nó. Uma porta de entrada é o argumento de uma função; uma de saída, o valor de retorno. O conjunto delas é devolvido pelo método estático `providedPorts()`

remapeamento

A ligação entre o nome da porta, fixo no código, e a chave do *blackboard*, escolhida no XML. A sintaxe `{chave}` num atributo diz “leia/escreva nesta entrada”; sem chaves, o atributo é um literal

subárvore

Uma `<BehaviorTree>` inteira usada como se fosse um nó dentro de outra. Por padrão recebe um *blackboard* próprio, isolado do pai, e só enxerga o que for explicitamente remapeado

tick

A única forma de executar uma árvore. Um *tick* é uma chamada de `executeTick()` propagada da raiz para baixo, segundo a lógica de cada nó de controle; ela retorna sempre um `NodeStatus` e nunca bloqueia por conta própria

Índice remissivo

ActionNodeBase, 15
Any, 65
AsyncActionNode, 6, 13, 14, 18, 41, 44, 45, 58, 69

BehaviorTreeFactory, 2, 49, 57, 85
BlackboardPreconditionNode, 38

ConditionNode, 10, 15, 19
ControlNode, 10, 19
CoroActionNode, 3, 13, 46, 71, 72

DecoratorNode, 10, 13, 16, 17, 19, 35, 39
DelayNode, 37, 66

FileLogger, 62, 69
ForceFailureNode, 35, 36
ForceSuccessNode, 35, 36

IDs de registro
 AlwaysFailure, 47, 86
 AlwaysSuccess, 47, 86
 BlackboardCheckDouble, 86
 BlackboardCheckInt, 86
 BlackboardCheckString, 86
 Delay, 86
 Fallback, 85
 ForceFailure, 86
 ForceSuccess, 86
 IfThenElse, 85
 Inverter, 86
 KeepRunningUntilFailure, 86
 ManualSelector, 85
 Parallel, 85
 ReactiveFallback, 85
 ReactiveSequence, 85
 Repeat, 86
 RetryUntilSuccessful, 86
 Sequence, 85
 SequenceStar, 85
 SetBlackboard, 47, 86
 SubTree, 86
 SubTreePlus, 86
 Switch2, 85
 Switch3, 85
 Switch4, 85
 Switch5, 85
 Switch6, 85
 Timeout, 86
 WhileDoElse, 85
IfThenElseNode, 18, 33

InverterNode, 35

KeepRunningUntilFailureNode, 36, 37

LeafNode, 10

ManualSelectorNode, 34, 73
MinitraceLogger, 62

ParallelNode, 28, 31
portas
 delay_msec, 85, 86
 failure_threshold, 85
 msec, 85, 86
 num_attempts, 36, 85, 86
 num_cycles, 36, 85, 86
 output_key, 47, 86
 repeat_last_selection, 85
 return_on_mismatch, 38, 86
 success_threshold, 85
 value, 86
 value_A, 38, 86
 value_B, 38, 86
 variable, 33, 85
PortInfo, 20, 25, 26, 54
PublisherZMQ, 3, 14, 18, 73

ReactiveFallback, 9, 31
ReactiveSequence, 9, 31
RepeatNode, 16, 36
RetryNode, 36, 82

SetBlackboard, 26
Signal, 59
SimpleActionNode, 42, 82
SimpleDecoratorNode, 39
SimpleString, 65, 66
StatefulActionNode, 18, 36, 41, 44, 45, 48, 76, 82, 83
StatusChangeLogger, 14, 59, 83
SubtreeNode, 10, 17, 18, 21, 86
SubtreePlusNode, 21, 55, 86
SwitchNode, 33
SyncActionNode, 10, 13, 32, 41, 42, 82, 83

Timeout, 30
TimeoutNode, 37, 66
TimerQueue, 6, 37, 69
Tree, 2, 9, 11, 16, 25, 40, 48, 57
TreeNode, 2, 4, 6, 10, 12, 15, 21, 27

WhileDoElseNode, 18, 33

XMLParser, 53